

Instructor notes — all sessions

Generative AI: Foundations & Applications · TCE Madurai · mastery guide, six prep packs, six run sheets, lab facilitation guide

Contents

1. Instructor Mastery Guide
2. Session 1 — Instructor Prep Pack
3. Session 1 — Run Sheet
4. Session 2 — Instructor Prep Pack
5. Session 2 — Run Sheet
6. Session 3 — Instructor Prep Pack
7. Session 3 — Run Sheet
8. Session 4 — Instructor Prep Pack
9. Session 4 — Run Sheet
10. Session 5 — Instructor Prep Pack
11. Session 5 — Run Sheet
12. Session 6 — Instructor Prep Pack
13. Session 6 — Run Sheet
14. Lab Facilitation Guide — all six labs

Instructor Mastery Guide

Generative AI: Foundations and Applications · TCE Madurai · 19–20 September 2026

What this is. The layer above the six prep packs. The prep packs tell you what to say on each slide and what the sharp student will ask about it. This document is for the ten days before, and its job is different: make the material *yours*, at mechanism level, so that any question — from a first-year who wandered in, from the best student in the room, from a faculty member who thinks this is statistics with a marketing budget — gets a correct, confident, honest answer without you reaching for notes.

Everything here agrees with the decks and with the verified September-2026 API facts in `pre-delivery-review-2026-09-08.md` (item 13). Where it goes beyond the decks it goes to textbook level, and where the honest answer is “contested” or “unknown” it says so. Slides are referred to by title, never by number — the S1 deck was cut from 31 to 29 slides this week and numbers drift.

Part 0 — How to use this guide, and the ten-day plan

The reading order

Read Part 1 first, twice — it is the one page that answers most questions. Then Part 2 one session per sitting, each followed by that session's prep pack and a full drive of its deck. Part 3 (the question bank) is for the last four days: read it in the evenings, cover the answers, say yours aloud, compare. Parts 4 and 5 are for the two days before and the mornings of. Part 6 is the reading list — ten to fifteen papers and pages, one to two per evening, abstracts and figures only unless something grabs you.

Do not try to memorise the question bank. Try to make every answer *derivable* from Part 1. If you can reduce a question to the causal chain, you can answer it; if you cannot, that is the section of Part 2 to re-read.

The ten-day schedule (Wed 9 – Fri 18 Sep)

Assumes about ninety minutes an evening on weekdays and two longer blocks on the weekend. The live checks are placed where they buy the most: early enough to fix what they find, late enough to still be true on the day.

DAY	READ / REHEARSE	LIVE CHECKS
Wed 9	Part 1 (twice). Part 2 §S1.	Regenerate every instructor/demo key in AI Studio (Standard keys are rejected from September; new keys are auth keys). Run <code>validate-live.py</code> on the fresh key. Resolve both aliases: <code>client.models.get("gemini-flash-lite-latest")</code> and <code>client.models.get("gemini-flash-latest")</code> — write the resolved names into <code>fact-check.md</code> . Confirm the Lab 4 embed fix (one <code>Content</code> per chunk, <code>output_dimensionality=768</code> , count assertion) is in the notebook and the validator.
Thu 10	S1 prep pack. Drive the S1 deck end to end in	Run Lab 1 on the fresh key. Test the temperature stretch cell: <code>temperature</code> , <code>top_p</code> , <code>top_k</code> are deprecated on Gemini 3.x

DAY	READ / REHEARSE	LIVE CHECKS
	presenter mode (<code>S</code>); time it. Open every depth panel once.	(changelog 21 Jul 2026) and were still accepted on 3 Aug; confirm 0.0 gives near-identical runs and 1.5 visibly varies. If the parameter is now ignored or rejected, rewrite the teaching note (Part 4 has the line). Run the Tamil <code>count_tokens</code> cell and note the real ratio.
Fri 11	Part 2 §S2. S2 prep pack. Drive the S2 deck; play all three spot-the-lie rounds without reading the answers.	Run Lab 2 end to end. Write your own ten-question test set (you need one as the worked example). Run the eval twice and confirm the score moves.
Sat 12	Part 2 §S3 and §S4. S3 and S4 prep packs. Drive both decks.	Run Lab 3 with your own receipt and handwriting photo; confirm <code>response_schema</code> parses on the lite alias. Run Lab 4 on the fresh key with your own documents , then with <code>labs/session-4/sample-os-notes.txt</code> ; confirm <code>chunk_vecs.shape[0] == len(chunks)</code> and that the Drive-mount/reload path works. Run the three playground queries from the deck against the sample document and check the trap query scores low.
Sun 13	Part 2 §S5 and §S6. S5 and S6 prep packs. Drive both decks. Part 5 (delivery craft) once.	Pre-pull <code>gemma4:e4b</code> on the laptop you will present from (<code>ollama pull gemma4:e4b</code> ; keep <code>gemma3:4b</code> as the fallback). Turn Wi-Fi off and run it. Run Lab 5 end to end, including the manual loop and the guarded loop; confirm the <code>function_response</code> format still round-trips. Run Lab 6; confirm the indirect payload in the Cell 5 demo still lands on the lite alias and that the direct attacks mostly bounce.
Mon 14	Part 3, sections “no ML background” and “strong students”. Say answers aloud before reading them.	Dry-run all six notebooks with <code>MOCK=True</code> (Cell 1 of each) so you know exactly what the offline path looks like before you ever need it in the room.
Tue 15	Part 3, “faculty / sceptics” and “about the tools”. Part 6: read the two Gemini docs pages (thinking, embeddings) properly, not skimmed.	Re-run <code>validate-live.py</code> . Check <code>pricing</code> and <code>models</code> pages once; update <code>fact-check.md</code> if a number moved.

DAY	READ / REHEARSE	LIVE CHECKS
Wed 16	Part 3, “careers” and “curveballs”. Part 4 (failure playbook) — read it as a checklist and imagine each one happening.	Publish the materials link, open it from a phone on mobile data and from a laptop on the college’s network if you can get on it. Confirm the Session 1 lab-kit slide’s link opens the course home (note: <code>certificate.html</code> is deliberately not deployed — the rubric students see is the Course Plan’s capstone section).
Thu 17	Full rehearsal of S1 and S2 aloud, standing, timed. S3 talk segment only.	Resolve both aliases again with <code>client.models.get</code> (the flash alias hot-swaps per release — 3.8 Flash went GA on 2 Sep). Screenshot your AI Studio rate-limit page for each project you will use; that screenshot is the honest answer to “what are the limits?”.
Fri 18	Full rehearsal of S4, S5, S6 aloud, timed. Re-read Part 1 and the “night before” checklist. Sleep.	Fresh-key dry run of Lab 1 Cell 1–3 and Lab 4 Cell 1–5 only (five minutes). Charge everything. Load the decks, the six notebooks and the sample document onto the presenting laptop and a USB stick.

Delivery-day checklist (both mornings)

Laptop on mains, not battery. Both decks for the day opened in separate tabs, each tested with `S`, `D`, `O`, `F` and the theme toggle on the projector. Presenter mode: the pace badge reads on-time at slide 1. Hotspot phone charged and tethered once as a test. AI Studio open in a tab, signed in with the instructor account, rate-limit page ready. A fresh Colab runtime with Lab N Cell 1 already executed (SDK installed) so your own demo does not spend the busiest five minutes installing. Whiteboard marker that works. Your own ten-question test set, two photos (receipt, handwriting) and the sample OS notes on the laptop. Stopwatch for the S6 demos. The five API-key steps written on the board before students open laptops (Day 1 only).

The night before Day 2

Ask yourself which pairs left Day 1 without a working key, and plan to seat them beside a working pair. Confirm your own Lab 4 artefacts (`chunk_vecs.npy`, `chunks.json`) are in Drive and that the reload cell in Lab 5 and Lab 6 finds them. Run `ollama run gemma4:e4b` once more with Wi-Fi off. Re-read the S6 timing: the slide titled “Six sessions, one throughline” carries the whole last hour — roughly 30 minutes red-team and harden, roughly 35 minutes demos, then the close — and its presenter budget is 76 minutes, so the pace badge stays honest through the block — trust it. Count the pairs: at three minutes each, more than twelve pairs means announcing 60–90-second lightning demos *before* Part A, not after. Decide your hard stop for demos and write it on the board.

Part 1 — The mental model that answers 80% of questions

Hold one causal chain in your head. Every slide in six sessions is either a link in it or a technique for shaping its input or measuring its output.

text → **tokens** → **embeddings** → **[attention + MLP] × L layers** → **logits** → **softmax(z/T)** → **sampled token** → **append** → **repeat**

Read it as a machine with one moving part. *Text* is turned into *tokens* by a fixed, learned vocabulary (BPE-style sub-word pieces). Each token is looked up in a learned table to get an *embedding*, a vector of a few thousand numbers, with position information added because attention itself cannot see order. That vector rides its own track (the residual stream) through *L* identical blocks; in each block, *attention* lets every token's vector borrow from every earlier token's vector according to learned relevance weights, and the *MLP* adds what the model “knows” about whatever the vector now represents. After the last block, the vector at the final position is scored against every row of the output table, giving one raw number per vocabulary token — the *logits*. *Softmax*, with the logits divided by temperature *T* first, turns those into a probability distribution. One token is *sampled* from it, appended to the input, and the whole thing runs again. A 500-word answer is roughly 650 trips through the machine.

Training is the only place the numbers inside change. The objective is cross-entropy on next-token prediction: for each position in a huge corpus, compute the distribution, take $-\log$ of the probability assigned to the token that actually came next, average, and nudge every parameter by gradient descent to make that number smaller. *Pretraining* does this on trillions of tokens of raw text and produces a base model that only continues. *SFT* (supervised fine-tuning, “instruction tuning”) does the same loss on curated question → answer pairs so the continuation looks like an assistant's reply. *RLHF* or *DPO* then tunes toward human preference rankings — a reward model plus policy optimisation in RLHF; a direct classification-style loss on preferred-vs-rejected pairs in DPO. All three are the same machine with different data and a slightly different loss; none adds a new component.

Inference runs the frozen file. Nothing in the file changes while anyone chats. There is no lookup, no index, no memory between calls. The **context window** — system prompt, conversation so far, documents pasted, tool results, and the answer being written — is the only working memory the model has, and it is measured in tokens because that is what the machine reads.

That last sentence is the hinge. **Sessions 2 to 6 are all techniques for either shaping what goes into the window or measuring what comes out.** Prompting (S2) shapes the input directly. Structured output (S3) constrains the output at the decoder. RAG (S4) chooses which of *your* text goes into the window. Tools (S5) let the model write a request whose result your code puts back into the window. Evals (S2, everywhere) measure the output against a test set. Guardrails and defence in depth (S6) filter the input and validate the output, and — because filtering text cannot be complete — cap what the output is allowed to *do*.

Eight reduction moves

When a question arrives, map it onto the chain before answering. These are the moves.

1. **“Where in the chain does that happen?”** Counting letters fails at *tokens*. Arithmetic fails at *sampling from a distribution* (there is no carry operation). Word order lives in *positional encoding*. Word meaning lives in *embeddings* plus *MLP*. Facts live in the *MLP weights*, smeared across millions of parameters. Style, refusals and manners come from *SFT/RLHF*. If you can name the link, you can name the failure and the fix.
2. **“Is this about training or inference?”** Half of all confusion is here. “Does it learn from me?” — inference, no. “Why doesn't it know last week's news?” — training ended before then. “Can I fix a wrong fact?” — not at inference (that is RAG's job), and not by editing the file (retraining or fine-tuning). Free-tier text may be used to train *future* models offline; that is a different thing from the model learning mid-chat.
3. **“Is this about the distribution or about one sample?”** Different answers to the same prompt, temperature, top-p, “it was right yesterday”, “run the eval three times” — all sampling. The model produces a distribution; everything downstream is dice. Temperature 0 is argmax and is nearly, not perfectly, deterministic.
4. **“What is in the window?”** Memory, RAG, long context, “lost in the middle”, tool results, injected documents, cost — all questions about the contents and size of the window. If it is not in the window it does not exist for this call. If it is in the window, the model cannot tell who wrote it.
5. **“Is that a claim about behaviour or about mechanism?”** “It reasons”, “it understands”, “it is creative” are behaviour claims; the deck makes mechanism claims. Answer at the level asked, then offer the other. Behaviour claims are settled by evals; mechanism claims by

the chain.

6. **“What is the loss, and does it measure what you care about?”** Pretraining minimises cross-entropy on next-token prediction. That loss says nothing about truth, safety or usefulness. Every “why does it hallucinate / why is it sycophantic / why is it confident” question is answered by: the objective rewarded plausible continuation, not correctness. Engineers therefore measure evals, not loss.
7. **“Where is the boundary, and is it mechanical or learned?”** SQL has a parser boundary between code and data; a transformer has none — the instruction/data distinction is a *trained preference*. Every security question reduces to this: text filtering raises attacker cost, only a mechanical boundary (what the tool is *allowed* to do, who must click) caps damage.
8. **“What compounds?”** Reliability across steps (p^n), context growth across turns, cost across users, n^2 across context length. Any “why does it get worse as it gets bigger/longer/more autonomous” question is one of these four multiplications.

If a question does not reduce to any of these, it is probably a question about products, policy or philosophy — answer it honestly as such, and say which parts are contested.

Part 2 — Concept mastery, per session

Each session: the concepts the deck claims, one level deeper than the deck and the prep pack; what the deck simplifies and how to unsimplify it; the canonical misconceptions; and what the lab evidences — including what a *failing* lab teaches.

Session 1 — How Machines Learned to Talk

Concepts, one level deeper

Discriminative vs generative. A discriminative model learns $P(\text{label} \mid \text{input})$ — a boundary in feature space. A generative model learns a distribution over the data itself; a language model specifically learns $P(\text{next token} \mid \text{all previous tokens})$, which by the chain rule is a full distribution over sequences: $P(x_1 \dots x_n) = \prod P(x_i \mid x_1 \dots x_{i-1})$. That factorisation is why “autocomplete” and “generative model of all text” are the same claim. Same neural-network mathematics, different training target. If a student asks whether classifiers are obsolete: no — fraud, ranking, face unlock, spam. And note that a generative model can be *used* discriminatively (ask it to output a label), which is what few-shot classification in S2 does.

Next-token prediction and the bigram toy. The deck’s tally table (“Those odds aren’t magic — you just count words”) is a genuine Markov chain of order 1: $P(\text{next} \mid \text{current})$ estimated by counts. Its failure is not that it is small but that it conditions on one token, so it cannot carry a subject across a clause. Raising the order to n makes the table size vocabulary^n and makes almost every context unseen — the sparsity wall. An LLM replaces the table with a *function* over the whole context that generalises to sequences never seen. Historical anchor for the faculty: Markov 1906, Shannon’s n -gram experiments 1948, neural language models Bengio et al. 2003, transformers Vaswani et al. 2017.

Logits, softmax, temperature. The last layer emits one unbounded real number per vocabulary entry. Softmax with temperature: $p(i) = \exp(z_i/T) / \sum_j \exp(z_j/T)$. Because T divides *inside* the exponent, it rescales the *gaps* between logits: $T < 1$ stretches gaps, so the exponential amplifies the leader (sharper); $T > 1$ compresses gaps (flatter); $T \rightarrow 0$ is argmax ; $T \rightarrow \infty$ is uniform. Worked example the room can check: logits (2, 1, 0). At $T=1$: $\exp = (7.39, 2.72, 1.00)$, sum 11.11 $\rightarrow (0.665, 0.245, 0.090)$. At $T=0.5$: logits become (4, 2, 0) $\rightarrow \exp (54.6, 7.39, 1) \rightarrow (0.867, 0.117, 0.016)$. At $T=2$: (1, 0.5, 0) $\rightarrow (2.72, 1.65, 1) \rightarrow (0.506, 0.307, 0.186)$. Top- k keeps the k largest before renormalising; top- p keeps the smallest prefix of the sorted distribution whose mass $\geq p$. The deck’s temperature widget computes exactly $\text{softmax}(\log p / T)$, which is the same thing with log-probabilities as logits.

Why T=0 is not perfectly deterministic. GPU reductions are not associative in floating point; batch composition changes summation order; near-ties flip. Also MoE routing can be batch-dependent. Say “close to reproducible, never guaranteed” and move on.

Parameters, training, loss. A parameter is one learned real number. $y = wx + b$ has two; frontier models have 10^{11} – 10^{12} . Training: loss = $-\log p(\text{actual next token})$, averaged over the batch; backpropagation gives $\partial \text{loss} / \partial \text{parameter}$ for every parameter; gradient descent (Adam in practice) steps each one a little downhill. The three numbers on the depth panel (“How anyone knows the training is working”) are exact: $-\ln 0.9 = 0.105$, $-\ln 0.1 = 2.303$, $-\ln 0.001 = 6.908$. **Perplexity** = $\exp(\text{mean loss})$, interpretable as the effective branching factor — “how many equally likely options is it choosing among”. A uniform guess over a 50k vocabulary has perplexity 50,000; good models on clean English sit in the low single digits. Cross-entropy is the right loss because minimising it is equivalent to maximising the likelihood of the training data under the model — and equivalently to minimising KL divergence from the empirical data distribution.

Training vs inference; no memory. Training changes parameters; inference does not. Chat “memory” is the application re-sending history. Free-tier data *may* be used to train future versions offline — that is a data-use policy, not the model updating. The Gemini API’s free tier is documented as using prompts to improve products; paid tiers are not. This is the privacy warning and it is real.

Tokens. Modern tokenisers are learned sub-word vocabularies (BPE and variants; Gemma’s published vocabulary is 256k entries, Gemini’s is not disclosed but is of that order). Merges are learned from frequency in the training corpus, which is English-heavy, so Latin-script English gets long merges and most other scripts get fragmented toward bytes. Consequences: cost per meaning, speed (more loops per sentence), and effective context (fewer sentences fit). The on-slide tokenizer is a rule-based toy; the lab’s `count_tokens` is real, and on Gemini the Tamil-to-English ratio is narrower than the toy suggests — measure it Thursday and quote your own number. Sub-words beat whole words because a whole-word vocabulary explodes and has no way to represent an unseen word; sub-words trade sequence length for coverage.

Embeddings. A learned table, one row per token, d_{model} wide (thousands of dimensions). Direction carries meaning; the geometry supports analogy arithmetic (king – man + woman $\approx$ queen) because training pushes tokens used in similar contexts toward similar vectors (the distributional hypothesis). The deck now covers this in the depth panel on “You saw this machine on the pre-class page”; S4 re-teaches it with the real embedding API. Distinguish *token* embeddings (rows of the input table, inside the LLM) from *text* embeddings (one vector per passage from a separate embedding model, pooled) — students conflate them.

Attention. For each token, three learned projections: query $Q = xW_Q$, key $K = xW_K$, value $V = xW_V$. Scores = $QK^T / \sqrt{d_k}$, causal mask sets future positions to $-\infty$, softmax per row, output = weights $\cdot V$. The output is *added* to the residual stream. Why the $\sqrt{d_k}$: for random d -dimensional vectors the dot product has variance proportional to d , so without scaling the logits would grow with d and softmax would saturate, killing gradients. Multi-head: h independent (Q, K, V) sets on d_{model}/h -wide slices, concatenated, projected. Why n^2 : n queries $\times$ n keys. Why it is order-blind: a set of vectors with no position information gives identical attention weights for any permutation, hence positional encoding — sinusoidal in 2017, RoPE (rotary) today, which rotates Q and K by a position-dependent angle so their dot product depends on relative distance. The “T in GPT” is Transformer; the paper is Vaswani et al. 2017.

Scale, scaling laws, Chinchilla, emergence. Kaplan et al. 2020 found loss falls as a power law in parameters, data and compute separately. Hoffmann et al. 2022 (Chinchilla) fit $L(N, D) = E + A/N^\alpha + B/D^\beta$ and showed compute-optimal training scales N and D together — about 20 tokens per parameter at the budgets studied — so a 70B model on 1.4T tokens beat a 280B model on 300B tokens. Modern practice over-trains relative to Chinchilla (far more than 20 tokens/parameter) because inference cost, not training cost, dominates the lifetime bill. “Emergence” (abilities appearing at scale) is real as a phenomenon and contested as a *discontinuity*: Schaeffer et al. 2023 showed several sharp jumps become smooth curves under continuous metrics. Say exactly that.

Base $\rightarrow$ SFT $\rightarrow$ RLHF/DPO. Base models continue text. SFT trains on demonstrations with the same next-token loss. RLHF (Ouyang et al. 2022, InstructGPT): collect human rankings of outputs, train a reward model to predict preference, optimise the policy with PPO against that reward with a KL penalty toward the SFT model. DPO (Rafailov et al. 2023) skips the reward model: it derives a closed-form loss on (preferred, rejected) pairs that is equivalent under the same KL-regularised objective, so preference tuning becomes ordinary supervised training. Refusals, tone and “I’m sorry, I can’t” come from this stage, not from pretraining.

Reasoning models / test-time compute. Same loop; trained (largely by reinforcement learning on verifiable tasks) to emit a long hidden chain of tokens before the visible answer. The scratchpad is real tokens billed at output rates — this is why the flash alias moving to a

thinking model in August burned an order of magnitude more output quota (28,089 hidden vs 2,770 visible tokens in the measured session). The correct framing: a third dial (scale, finishing school, thinking time), each buying capability at a different cost.

Model families; MoE, distillation, quantization. MoE: many expert MLPs per layer plus a router that activates a few per token; total parameters large, active parameters small, so parameter count stopped predicting cost. Distillation: train a small student to match a large teacher's *full output distribution* (soft targets carry more information than hard labels). Quantization: store weights in fewer bits; $\text{params} \times \text{bits} \div 8 = \text{bytes}$. Closed weights vs open weights is a licensing/distribution fact, not an architectural one.

Famous failures. All four are the chain seen from different angles: letters (tokens), arithmetic (sampling, no algorithm), news (training cutoff), citations (the objective rewards plausible). Say "one mechanism, four symptoms."

Context window and the KV cache. The window is the input length the model was trained and served for. The app re-sends everything, but the server does not recompute everything: keys and values for already-processed tokens are cached. Prefill (parallel, sets time-to-first-token) vs decode (sequential, sets tokens/second). Cache memory per token $\approx 2 \times L \times d_{\text{kv}} \times \text{bytes}$; with grouped-query attention d_{kv} is much smaller than d_{model} , which is one of the tricks that made 1M windows serveable. Context caching as a product is renting a stored prefix; it only helps if the stable part of the prompt comes first.

The API call. HTTPS POST with the key in a header; gateway checks identity and quota (429 = over quota, 5xx/503 = capacity); the model runs on the provider's accelerators; tokens stream back. Latency is the loop running.

What the deck simplifies, and how to un-simplify it if asked

The tokenizer widget is rule-based; the real one is learned BPE and the ratio varies by model. The attention percentages are illustrative of one head, one layer; real models run dozens of heads over dozens of layers and no single picture is "the" attention. The 21-neuron network animation shows the choreography, not the architecture — a real block is attention plus MLP plus normalisation, repeated. The "fit the line" widget is two parameters and one loss; real training is the same idea with 10^{11} parameters and a stochastic gradient over minibatches, but the loop is honestly the same. The scale slider implies one axis; there are three (parameters, data, compute) and the depth panel says so. "Finishing school in three steps" hides that modern pipelines interleave many rounds of SFT and preference tuning and add RL on verifiable tasks for reasoning. The context-window widget shows a fixed 128k/1M wall; real serving has a hard limit plus a soft quality decline before it ("lost in the middle").

Misconceptions and the one-sentence correction

It searches the internet. Nothing is looked up; the weights are frozen and the answer is computed (point at "Wait — is it just searching a giant database?"). *It learns from my chat.* The parameters never change during inference; the app re-sends the history (point at "Training vs using: the cookbook rule"). *Bigger is always better.* Three dials, and inference cost is the constraint that decides products (point at "The loop is old. The scale is new." and its panel). *Temperature is creativity.* It is one division inside softmax; the model is not more imaginative at $T=2$, only less selective. *Hallucination is a bug they will fix.* It is the same machinery that produces correct answers; the objective rewards plausibility.

What the lab evidences, and what a failing lab teaches

Lab 1's Checkpoint 1 evidences the API-call chain: the model is not on the laptop, the key is identity, the response is computed. Part C's five prompts evidence that one machine does explain/summarise/translate/extract/roleplay — one loop, many tasks. Part D evidences "same trick, different kitchens": identical prompt, different finishing school, different length and tone; the alias comparison also evidences thinking cost (slower, more verbose from the flash alias). Part E is not evidence of anything yet; it is the test set S2 needs. The stretch cells evidence temperature (variance), tokenizer inequity (`count_tokens`), and the bigram model (the toy from the deck, runnable).

A 429 during Part B teaches the gateway link in the chain better than any slide — narrate it as "you just hit the quota check". A student whose temperature-1.5 runs look identical teaches that the parameter is deprecated on 3.x and that sampling behaviour is provider-controlled; say so and point at the review. A Tamil token count that is only $\sim 2\times$ English teaches "measure, don't assume" — Gemini's tokenizer is comparatively multilingual. A student who cannot create a key because they have a Google Cloud account teaches that AI Studio no longer auto-creates a project for existing GCP users; they must import one. None of these is a broken lab; each is a link of the chain made visible.

Session 2 — Talking to AI, and Catching Its Lies

Concepts, one level deeper

Prompt anatomy as distribution shaping. Role, task, context, format, examples, constraints each condition the next-token distribution. Mechanistically a prompt is just earlier tokens that every later position attends to; a role line shifts which regions of the training distribution the continuation resembles, a format line raises the probability of the requested tokens, a constraint line lowers the probability of invented continuations. Over-stuffing dilutes attention because softmax weights sum to one — every irrelevant token competes for the same unit of attention mass — and costs tokens per call.

In-context learning (few-shot). Brown et al. 2020 named it: behaviour changes with zero weight updates because the examples are in the window. Why it works is still studied — one line of evidence treats attention as implementing something like gradient descent on the in-context examples; another shows the model is pattern-matching to the demonstrated format. For the room: examples pin format, label set and edge cases far more precisely than description, and two to five is the practical sweet spot because each example costs tokens and the marginal gain flattens.

Chain-of-thought. Wei et al. 2022. Intermediate tokens are computation the model can condition on; a single forward pass has fixed depth, so multi-step arithmetic that does not fit in one pass can fit across many. It is not “thinking harder”; it is more serial compute and an auditable trace. The ₹472 example: $500 \times 0.8 = 400$; $400 \times 1.18 = 472$. Direct answering tends to produce a plausible-looking number because no position in the network performs the carry.

Format requests vs enforcement. “Reply ONLY with JSON” raises the probability of JSON; it does not make non-JSON unreachable. S3’s `response_schema` masks illegal tokens at the decoder, which does. Keep the distinction crisp because the S3 payoff depends on it.

Hallucination as calibration failure. The model’s expressed confidence (tone) and its actual accuracy are different quantities, and the training objective never aligned them. Pretraining rewards the most plausible continuation; SFT/RLHF reward answers humans preferred, and humans prefer confident, complete answers — so unwarranted confidence is actively selected for. The two cases are real and checkable: *Mata v. Avianca* (S.D.N.Y., 2023, six fabricated citations, \$5,000 sanction) and *Moffatt v. Air Canada* (2024 BCCRT 149, airline liable for its chatbot’s invented bereavement-fare policy).

Test set, scorer, score. Three separable components. Change any one and the number changes, which is why a score is meaningless without the whole configuration: “7/10 with normalised-contains on my ten cricket questions at T=0, averaged over three runs”. Run at T=0 to remove sampling variance; run three times to measure what remains; average and report spread.

Scorer design and its failure modes. Exact match: precision high, recall terrible on free text. Normalised contains (lowercase, strip punctuation, substring): forgiving, but a short or generic expected string false-positives (“1992” appears in many answers), and it will pass an answer that contains the fact *and* a contradiction. LLM-as-judge: handles paraphrase, introduces bias — Zheng et al. 2023 (MT-Bench) measured position bias (prefers the first candidate), verbosity bias (prefers longer), and self-enhancement bias (prefers its own family’s outputs). Mitigation: swap order and require agreement; hand-grade 20–30 items and measure agreement with the judge; below ~80% you are measuring the judge.

Precision, recall, F1. Precision = $TP/(TP+FP)$; recall = $TP/(TP+FN)$; $F1 = 2PR/(P+R)$, the harmonic mean, which punishes imbalance. Accuracy misleads under class imbalance (the 5%-escalation bot scoring 95% by escalating nothing). Which to optimise is a consequence question: screens buy recall, auto-actions buy precision.

Eval sample size — the $\pm 1/\sqrt{n}$ rule. A pass/fail score over n items is a binomial proportion with standard error $\sqrt{p(1-p)/n} \leq 0.5/\sqrt{n}$. Two standard errors ($\approx 95\%$ band) at $p = 0.5$ is therefore exactly $\pm 1/\sqrt{n}$: $n=10 \rightarrow \pm 0.32$, $n=100 \rightarrow \pm 0.10$, $n=1,000 \rightarrow \pm 0.03$. The deck’s table (± 30 , ± 10 , ± 3 points) is that arithmetic rounded. Paired comparison helps because on the items both prompts get right or both get wrong the difference is zero and carries no information; only the discordant items count, and the noise on the difference is set by their number, not by n . The model’s run-to-run variance (T=0, three runs) and the test-set variance (which ten questions) are different quantities with different fixes.

Eval-driven development. Prompt → eval → read failures → fix one thing → re-run, until the score plateaus, then grow the test set. Changing one thing at a time is ordinary experimental control. The eval set becomes the regression test in S6.

What the deck simplifies, and how to un-simplify it

The quality meter on the makeover is a teaching device, not a measurement; say so. The arena shows five questions and the widget scores B 4–2 while its depth panel says “4–3” — if a student notices, the honest answer is that the panel's argument is about *any* one-question gap on $n=5$ and the number on screen is the one that counts (the review flagged it for correction). The eval demo is scripted so the score moves 7 → 8 reliably; real runs move less predictably. The spot-the-lie game presents lies as clean fabrications; real hallucinations are usually *partly* right — a real award, wrong year — which is harder to catch and is exactly why the lab wants deep-cut questions. “Three runs, average” is a minimum; a real regression suite runs more and tracks the distribution.

Misconceptions and the one-sentence correction

A better prompt fixes hallucination. It reduces frequency; it cannot change an objective that rewards plausibility (point at “Hallucination is not a bug”). *A high score means it works.* It means it works on that test set with that scorer at that temperature. *Confident tone signals correctness.* The tone meter is identical on every round including the lies (point at “One of these is a confident lie”). *The scorer is neutral infrastructure.* Exact-match rejects “AR Rahman” for “A. R. Rahman” (point at “Same answer, three verdicts”). *Ten questions settle A vs B.* ± 30 points; it is a smoke test (point at the arena's depth panel).

What the lab evidences, and what a failing lab teaches

Part A evidences that each upgrade narrows the distribution — the documented row per iteration is the artefact. Part B evidences the three components and, through Checkpoint 2's per-failure diagnosis (model / scorer / question), that the number is a joint property of all three. Part C evidences paired comparison and that the winner must be reported with n .

A 10/10 baseline teaches that the test set is too easy to teach anything — swap in harder questions. A student who “fixes” a failing test by loosening `expected` until it passes is committing the exact crime the session exists to prevent; catch it at the sweep and name it. A score that moves between runs at $T=0$ teaches the non-determinism footnote. A judge that grades an obviously wrong answer as correct teaches judge bias more vividly than the panel does. An expected answer that turns out to be wrong (“Roja was 1993” — it was 1992) teaches that an eval lies with total confidence when its labels are wrong, which is why you read failures.

Session 3 — AI Beyond Text

Concepts, one level deeper

Patches are tokens (ViT). Dosovitskiy et al. 2020: split the image into fixed patches (16×16 pixels classically), flatten each to a vector of $16 \cdot 16 \cdot 3 = 768$ numbers, multiply by one learned matrix into `d_model`, add a positional embedding, feed to a standard transformer encoder. The whole contribution was that no convolutional inductive bias was needed at scale. In a multimodal LLM the image encoder's output vectors are projected into the LLM's embedding space and interleaved with text tokens, so “what is in this photo” is next-token prediction with picture-tokens earlier in the context. Consequences: token count scales with resolution (double each side, $4 \times$ patches); providers tile large images and charge per tile — on the order of a few hundred tokens per tile, with the exact tile size and count on the pricing/vision docs page (quote the page, not a number from memory); anything smaller than a patch is averaged into its neighbours, so small text fails suddenly; and attention pools rather than enumerates, so counting is estimation.

Audio. Waveform → short frames (the deck says ~20 ms) → spectral features → encoder → tokens, then the same stack. Speech-to-text is near-human on clean read speech and degrades on accents, code-switching (Tamil-English), noise, and rare proper nouns — the failures are silent, which is why “lectures → notes” is a useful workflow, not a solved problem (the deck says exactly this; the S3 cheatsheet's “solved” was flagged for correction).

Diffusion. Forward process: add Gaussian noise to a clean image over T steps by a fixed schedule — nothing learned. Training (Ho et al. 2020, DDPM): sample an image, a step t and noise ϵ ; the network takes the noised image and t and is trained to predict ϵ with a simple

mean-squared error. Sampling: start from pure noise, repeatedly subtract a fraction of the predicted noise (plus a little fresh noise in stochastic samplers), 20–50 steps in modern schedulers. Why coarse-before-fine: low-frequency structure survives more noise than high-frequency detail, so it is recoverable earlier in the reverse process. Text conditioning: the prompt is embedded by a text encoder and injected via cross-attention; classifier-free guidance runs the network with and without the prompt and extrapolates away from the unconditional prediction (the “guidance scale”). Latent diffusion (Rombach et al. 2022): a VAE compresses the image about 8× per side, denoising happens in the latent grid — 8× per side is 64× fewer positions, and four latent channels against three RGB channels gives $64 \times 3/4 = 48\times$ fewer numbers per step, which is the deck’s “ $\approx 48\times$ ”; say “about fifty times less work” and do not fight over the constant. Autoregressive image generation (tokens over a learned image codebook) is a competing family; both coexist.

Voice cloning and the family password. A few seconds of audio suffices for a speaker-embedding-conditioned TTS model to reproduce timbre and prosody. Detection is an arms race; the defence that does not depend on detecting the fake is an out-of-band shared secret. This is a genuine, current fraud pattern in India; deliver it straight.

One API call. `contents=[img, question]` — the SDK serialises the image as inline bytes or a file reference; the model receives interleaved image and text tokens. Everything from S1–S2 transfers because the machine is the same.

Constrained decoding. `response_mime_type="application/json"` + `response_schema` compiles the schema to a grammar (a finite-state machine over tokens, or a pushdown automaton for nesting); at each decode step tokens that cannot extend a valid prefix are masked to $-\infty$ before softmax. Malformed shape is unreachable; wrong *values* are not. Two cautions the depth panel gives: shape $\neq$ truth (a number field guarantees a number, not the right number), and over-constraint can hurt quality because a terse schema removes room to reason — put a `reasoning` field before the answer fields since the model fills them in order and can only condition on what it has already written. And an explicit null/unknown outcome must exist in the schema, or the model is forced to invent.

Vision failure modes. Counting (pooling, not enumeration — same disease as arithmetic), spatial precision (position is encoded weakly after pooling), blurred text (the prior completes what pixels do not determine — hallucination in pixels; the grounding line “if unreadable, say so” reduces it), identity (refused by design; KYC face-match is a separate regulated system).

What the deck simplifies

The patchify demo shows a small grid; real encoders use finer patches and often multi-resolution tiling. The diffusion canvas is a linear pixel blend — the coarse-to-fine ordering is real, the pixels are not; say which part is honest before anyone asks. “3 s of audio” is a round number for a capability that exists at that order of magnitude; do not defend the exact figure. The video landscape line (Sora 2, Veo 3.1, Kling 3.0, \$0.10–0.75/s) is mid-2026 reporting and belongs in the volatile table; say “order of magnitude, as reported”.

Misconceptions and the one-sentence correction

Vision is a different kind of AI. Patches become tokens and enter the same attention loop (point at “Same loop. New kinds of tokens.”). *Diffusion draws like a person.* It subtracts predicted noise, repeatedly; shapes resolve before details (point at “Diffusion: a picture emerges from static”). *If it can see it, it can count it.* One forward pass estimates; there is no tally (point at “Where vision quietly fails”). *Asking for JSON gives valid JSON.* A prompt requests; a schema constrains (point at “All of it is one API call” and its panel). *Capability is permission.* It can read a CAPTCHA; that does not mean your product may (point at “Will it read it? Place your bets”).

What the lab evidences, and what a failing lab teaches

Part A’s ladder (describe → read → count → infer → surprise) evidences the failure gradient in one photo: description good, reading good, counting shaky, inference confident. Part B evidences that the test of extraction is `json.loads` succeeding, not the output looking right. The `response_schema` cell evidences constrained shape — and Checkpoint 3 asks the student to check the *values* against the image, which is the shape- $\neq$ -truth lesson. Part C evidences that handwriting and Tamil are where quality is uneven. Part D is designed to fail: the confident invention is the checkpoint.

A blurred total that the model “reads” is the lesson, not a bug — that is Part D’s checkpoint. A miscounted crowd is the ViT panel made concrete. A `json.loads` crash on the prompt-only cell followed by success on the schema cell is the best possible outcome. A schema cell that returns a null total on an unreadable receipt evidences correct design. A Tamil handwriting transcription at 40% is a data point for the “Indian languages” question in Part 3 — collect it.

Session 4 — Giving AI Your Own Knowledge

Concepts, one level deeper

The villain: it cannot know, and it will not say so. No mechanism in the chain represents “this is outside my training data”; the MLP adds whatever direction the question's vector triggers. Hedging is a trained behaviour (RLHF), not knowledge of ignorance — a model that apologises first and then invents a syllabus has hedged and hallucinated in the same reply.

The three taxes on pasting. Window (hard limit), meter (input tokens billed every call), middle (Liu et al. 2023, “Lost in the Middle”: accuracy on a fact placed mid-context drops relative to the same fact at the start or end, across models and lengths). Arithmetic for the slider: a 380-page textbook at roughly 500 words a page is ~190k words ≈ 250k tokens; at Gemini 3.5 Flash's \$1.50 per million input tokens that is about $0.38 \approx ₹36 * \text{perquestion}^*, \text{before the answer. On 3.5 Flash - Lite (0.30/1M)}$ it is ~₹7 per question — still absurd at 30 questions a day per student.

Semantic search. Embed each chunk once with an embedding model (a transformer whose token vectors are pooled — mean-pooled or a designated summary token — into one vector per passage, then trained contrastively so that paraphrases land close and unrelated text lands far). Embed the query with the *same* model. Score = cosine similarity = $(a \cdot b) / (|a| |b|)$. Normalise every vector to unit length once and cosine becomes the dot product, so `chunk_vecs @ q` scores every chunk in one matrix multiply, and `argsort` gives top-k. Forget normalisation and you rank partly by vector length, which correlates with passage length, not relevance. Query/document asymmetry: good APIs accept a task type so that questions and declaratives are embedded into a shared region. Dimensions are a dial: many current models are trained with Matryoshka-style objectives so the first 768 of 3072 dimensions carry most of the signal — this is why `output_dimensionality=768` is safe and why the lab uses it. Never mix vectors from two embedding models; store the model name beside the index.

gemini-embedding-2 specifics (September 2026, verified). Default output dimensionality is **3072**; the lab passes `output_dimensionality=768` to match the deck's stepper. Passing a bare list of strings as `contents` returns **one aggregated embedding for the whole list**; to get one vector per chunk each input must be wrapped as its own `Content` object. The SDK (google-gemai 2.22) has an explicit branch that folds a string list into one `Content`, so the old code returned `(3, 3072)` for 60 chunks and `search()` silently returned the same three chunks for every query. The fix asserts `len(res.embeddings) == len(texts)`. `gemini-embedding-001` is *not* shut down — earliest shutdown 14 May 2028; 14 July 2025 was its release date; embedding-2 (GA 22 Apr 2026) supersedes it. Price: \$0.20 per million input tokens.

Chunking. Too small: the fragment loses referents (“...and the end-semester exam” of what?). Too large: one vector must summarise several topics and sits near none (pooling's cost); the right sentence is diluted and the prompt fills with irrelevant text. Paragraph plus overlap on natural boundaries is the default that wins. Upgrades in order of payoff: structure-aware recursive splitting; small-to-search/big-to-read (index small, return the parent section); metadata stored at index time (source, section, page, date) so you can filter *before* ranking; a one-line context header prepended to each chunk before embedding. The free test: print ten random chunks and read them cold.

Vector databases. An approximate-nearest-neighbour index (HNSW graphs, IVF partitions, product quantisation) trades a little recall for sub-linear search over millions of vectors, plus persistence, filtering and multi-user access. Below ~100k vectors, brute-force numpy is exact and fast enough; adding infrastructure earlier adds latency and failure modes with no accuracy gain.

RAG end to end (Lewis et al. 2020 coined the name; the DPR bi-encoder is Karpukhin et al. 2020). Retrieve → augment → generate. The grounded template's three load-bearing lines: ONLY the context (suppresses parametric memory), cite the chunk (makes errors checkable), and the escape hatch (a legal way to refuse — without it the model fills silence with fiction). Reranking: the bi-encoder embeds question and chunk separately (fast, good recall, mediocre ranking); a cross-encoder reads them together (accurate, slow) and is affordable only on the top 20. Hybrid search merges BM25 (exact identifiers, course codes, surnames) with semantic (paraphrase). Query rewriting fixes the vocabulary gap at the source.

Where RAG breaks; two scores. Retrieval failures (vocabulary gap, answer split across chunks) versus generation failures (stale index cited confidently; model ignores context). Recall@k = fraction of questions whose gold chunk is in the top k — the ceiling on answer accuracy. MRR = mean of 1/rank of the first correct chunk (rank 1 → 1.0, rank 5 → 0.2); high recall with low MRR is the reranker signature.

Faithfulness = given the right chunk, did the answer come from it? Debug order: retrieval → chunks → prompt → model. Put unanswerable questions in the test set and check the refusal fires.

RAG vs long context vs fine-tuning. Fine-tuning moves parameters toward a behaviour; it is poor at injecting facts (facts are smeared across the MLP and new ones interfere with old), freezes at training time, and gives no citations. RAG updates by re-embedding one file and cites. Long context wins for small, static corpora queried rarely; RAG survives on cost, latency, freshness and citations; real systems combine both.

What the deck simplifies

The playground's similarity scores are authored ranges; the lab computes real ones. "Cosine similarity, 1 = same, 0 = unrelated" hides that real embedding models rarely produce values near 0 — unrelated text often scores 0.2–0.4, so absolute thresholds must be calibrated per model (the trap query "every bar under 0.35" is a property of the widget; your own threshold comes from your own data). "One numpy line" hides normalisation, task type and pooling, which the depth panel restores. The RAG stepper shows top-3 straight into the prompt; production inserts rerank and often a query-rewrite step. "Fine-tuning teaches behaviour; RAG provides knowledge" is a slogan; the precise version is that parameter updates are a poor, slow, uncitable channel for facts, not an impossible one.

Misconceptions and the one-sentence correction

Fine-tuning is how you add knowledge. It changes behaviour; facts go in the window via retrieval (point at "RAG vs paste-it-all vs fine-tuning"). *1M context makes RAG obsolete.* Window, meter, middle — and citations and freshness — still favour retrieval for anything large or changing (point at "Just paste all my notes — let's price that"). *A citation means the answer is right.* A stale index cites its old chunk confidently (point at "Where RAG breaks in the wild"). *Chunking is a detail.* It causes more failures than model choice (point at "Chunking: how you cut the book"). *You need a vector database.* A numpy array is one with fewer moving parts (point at "Vector databases, in plain words").

What the lab evidences, and what a failing lab teaches

Part A evidences the ingest pipeline and — with the fixed `embed()` — that the store has one row per chunk. Part B evidences that retrieval is inspectable: the top chunks are printed with scores. Part C evidences grounding and citation. Part D is designed to fail twice: the out-of-corpus question must trigger the escape hatch; the split-across-chunks question must produce a half-truth, which is the overlap/neighbour lesson.

A store shaped `(3, 768)` for 60 chunks is the aggregated-embedding bug and teaches that a wrong API contract can fail silently — the assertion exists precisely so it fails loudly. Garbage extraction from a scanned PDF teaches that RAG starts with text, and that S3's vision extraction is the tool for images of text; swap the document at minute five. Search returning junk teaches chunking before prompting. A model that answers a question not in the documents teaches that the ONLY line is a preference, not a wall — strengthen it, lower temperature, put context before the question, and note that this is exactly why S6 exists. A pair with no documents teaches nothing until they use the sample file; have it ready.

Session 5 — Making AI Do Things

Concepts, one level deeper

The trick. The model emits a structured `functionCall` part (name plus JSON arguments) instead of prose; the turn ends; your code decides whether to run it, runs it, appends a `functionResponse` part; the model continues. Tool use is a trained behaviour — models are fine-tuned on examples of emitting these calls — which is why fluency and tool competence are separate skills. Strictly true for tools *you* define; provider-hosted tools (code execution, search grounding) run on the provider's side and are a different trust boundary — concede this before a sharp student finds it.

The wire format. Your Python function becomes a JSON schema: name, description (the docstring), parameters with types and descriptions, required list. That JSON is the model's entire knowledge of the tool; it cannot read the body. Hence "the docstring is the

prompt” and a wrong tool choice is a writing bug. The SDK’s automatic function calling builds the declaration from type hints and docstring, runs the loop for you, and hides the four roles (user → model functionCall → user functionResponse → model text); Lab 5 Part C switches it off to show them. Models can return several calls in one turn when the calls are independent; run them in parallel. Every tool result is re-sent on every later turn, so return the smallest useful result.

MCP. The Model Context Protocol (Anthropic, November 2024, since adopted broadly) standardises the declaration format and the transport so one server exposing tools and resources can be used by any client model — N+M integrations instead of N×M. Mechanically nothing changes: the model still writes requests, a host still executes. Name it, do not demo it.

The agent loop and its leash. `while model wants a tool and steps < MAX_STEPS: assert name in ALLOWED; validate args; execute; append result`. Without the cap the termination condition is a probabilistic system holding your key. The on-slide sketch already carries the guard (the “loop with no exit condition” wording in older notes is stale). Context grows monotonically with every call and result, so long runs get more expensive and less obedient (lost-in-the-middle again); production summarises old steps or writes intermediate state to files.

Reliability compounds: p^n . If each step succeeds independently with probability p , n steps succeed with probability p^n . $0.95^{10} = 0.599$; $0.95^{20} = 0.358$; $0.99^{20} = 0.818$; $0.99^{100} = 0.366$. Two 90%-reliable agents in series: 0.81 before any work is done. The independence assumption is generous — in practice errors correlate and the tail is worse. Fixes in order: fewer steps (workflow), validation between steps (which converts a silent failure into a caught one and resets the chain), human approval on side effects, retries on cheap idempotent steps. The Monte-Carlo widget draws n Bernoulli trials per agent and shows the death step; that is a fair simulation of the model.

Workflow vs agent. Anthropic’s “Building effective agents” (December 2024) makes the same distinction: workflows are systems where the LLM and tools are orchestrated through predefined code paths; agents are systems where the LLM directs its own process. The decision rule: do you know the steps in advance? Multi-agent chains multiply p^n ; parallel fan-out with a judge does not, because independent attempts do not compound — and the judge is a single validation step.

The escalation ladder. Prompt → few-shot → RAG → tools → fine-tuning, climbed only when the eval proves the cheaper rung failed; rung zero is “you did not need a model”. Fine-tuning last because it is the most expensive, least reversible, and worst at facts.

Local models: the arithmetic. File size = parameters × bits ÷ 8: $4B \times 16/8 = 8\text{ GB (bf16)}$, $\times 8/8 = 4\text{ GB (Q8)}$, $\times 4/8 = 2\text{ GB (Q4)}$. Add 1–2 GB headroom for the KV cache, which grows with context. Tokens per second on a laptop is bounded by memory bandwidth, not compute: each generated token reads every active parameter once, so 2 GB of weights over ~50 GB/s gives ~25 tokens/s ceiling. Quantization degrades the hard tail first (multi-step reasoning, exact formats, rare languages). A 70B model at Q4 is ~35 GB — a hardware conversation. Gemma 4 E4B is a ~4B-active open-weight model (Apache 2.0, March 2026 per the deck — volatile-table it); `gemma3:4b` still runs if the pull fails.

What the deck simplifies

The agent log is scripted for reliability; a real run varies its order and sometimes over-calls. “Nobody scripted the order” is true and also the risk — say both. The tool stepper’s “validate: args parse as maths, not DROP TABLE” is a real defence (the notebook’s AST allow-list) and also a demo; production adds limits, timeouts, authorisation, logging. The p^n slider assumes independence, which flatters real agents. The Ollama “race” is a simulation if the live demo fails — label it. The S5 cheatsheet’s `eval()` -behind-a-character-filter snippet is *not* what the notebook does (the notebook uses an AST walker) — if a student copies the cheatsheet version, `g**g**g` gets through; point them at the notebook.

Misconceptions and the one-sentence correction

The model executes code. It writes a request; your code executes (point at “The model never executes anything. It writes requests.”). *Agents are the default architecture.* Do you know the steps? Then a workflow — most business AI (point at “Workflow or agent? One question decides”). *More agents means more capability.* $0.9 \times 0.9 = 0.81$ before any work (point at “Multi-agent: more agents, more compounding”). *Tool output is trustworthy.* It is text entering the window from something you did not write — the S6 bridge (point at “Five ways tool use goes sideways”). *Local models are toys.* A distilled 4B on one narrow task with an eval often beats a general giant on that task; production is hybrid (point at “Local vs API: an engineering trade, not a religion”).

What the lab evidences, and what a failing lab teaches

Part A evidences delegation: with the tool the GST answer is exact (₹422.46; total ₹2,769.46); without it the digits wobble. Part B evidences chaining without scripting. Part C evidences the wire format — the raw `function_call` printed is the whole trick made visible. Part D evidences the decision framework on paper. Stretch 3 evidences trajectory eval (tool-choice accuracy including the no-tool case); Stretch 4 evidences the leash.

A model that calls `calculator` for “who wrote the Thirukkural?” teaches over-tooling — sharpen the docstring and add “answer directly when no tool is needed”. Arguments like `"eighteen percent of 2347"` crashing the AST parser teach validate-inside-the-tool and readable errors (the model self-corrects on a good error string). A loop that hits `MAX_STEPS` teaches why the cap exists. An Ollama demo that fails on college Wi-Fi teaches nothing useful — pre-pull it; if it still sulks, run the labelled simulation and say so.

Session 6 — Breaking, Securing, Shipping

Concepts, one level deeper

Every capability is a surface. RAG reads → documents attack. Tools act → hijacks act. Vision reads → images carry text instructions (an image with “ignore previous instructions” printed in it is a real payload class).

Prompt injection, precisely. System prompt, user message, retrieved chunks and tool results arrive as one token sequence. The distinction “instructions vs data” is a learned preference from SFT/RLHF (and, on newer models, explicit instruction-hierarchy training), not a mechanical boundary. A sufficiently persuasive sequence outvotes the preference. Direct injection: the user types the attack. Indirect injection (Greshake et al. 2023): the attacker never talks to the bot; they plant text where the pipeline will fetch it — a web page, a shared PDF, white-on-white text, a wiki edit, a tool result. OWASP Top 10 for LLM Applications lists prompt injection as LLM01 in both the 2023/24 and 2025 editions. There is no escape character because there is no parser: SQL quoting, HTML entity encoding and shell argument arrays all work because a mechanical parser separates code from data, and a transformer has no such stage.

Why direct attacks bounce and indirect ones land. Current models are trained hard against a *user* asking them to misbehave (the Lab 6 Cell 3 attacks mostly bounce — that is the finding, not a broken exercise). They are much weaker when the instruction arrives inside content the *developer* told them to trust (“Answer using ONLY this context”), because grounding instructions and injected instructions are the same kind of token. Lab 6 Cell 5 measured the model replying exactly “PWNED” from a poisoned chunk on the same key that refused the direct attacks. Teach that contrast explicitly; it is the most important security fact in the course.

Jailbreaks and leaks. A jailbreak wraps a refused request in roleplay/hypothetical framing so the refusal preference does not fire — an arms race against safety training. A prompt leak extracts the system prompt; assume the prompt is public and never put a secret in it.

Defence in depth. Delimit and label untrusted text; instruction hierarchy in the system prompt; output validation (format, no secrets, allow-listed values); least privilege plus a human gate on anything that writes, spends or sends. The first three raise attacker cost; only the fourth caps damage. The lethal trifecta (Simon Willison, 2025): catastrophic exfiltration needs access to private data, exposure to untrusted content, and a channel out — remove any one leg and the attack class dies regardless of prompt cleverness. This is a design property you can verify; “our filter catches it” is not. Practical order: narrowest tool set; human click on side effects; a tool’s output never expands permissions; run with the asking user’s privileges, not the app’s; treat every tool result as hostile.

Match trust to blast radius. Read-only actions can run unsupervised; side effects get a gate. Autonomy is a dial. This is S5’s leash made official.

Notebook → product: cost. September-2026 prices per million tokens: Gemini 3.5 Flash \$1.50 in / \$9.00 out (the deck’s slider constants, now the previous-generation Flash); Gemini 3.5 Flash-Lite (the lab model) \$0.30 / \$2.50; Gemini 3.8 Flash \$0.75 / \$3.75 introductory to 31 Dec; gemini-embedding-2 \$0.20 per million input tokens. At ₹95 per dollar (the deck uses 95.5): Flash-Lite input ₹28.5 per million tokens, output ₹237.5 per million. A typical RAG query — 2,000 input tokens (system prompt + three chunks + question) and 300 output tokens — costs on Flash-Lite $2,000 \times 0.30 + 300 \times 2.50 = \$0.0006 + \$0.00075 = \$0.00135 \approx ₹0.13$; on 3.5 Flash $2,000 \times 1.50 + 300 \times 9.00 = \$0.003 + \$0.0027 = \$0.0057 \approx ₹0.54$. At ten queries a day per user for 30 days: Flash-Lite $\approx ₹39/\text{user}/\text{month}$, 3.5 Flash $\approx ₹162/\text{user}/\text{month}$. Embedding a 60-chunk corpus of ~200 tokens each is 12,000 tokens $\approx \$0.0024 \approx ₹0.23$, once. Output costs 6× input on 3.5 Flash and ~8×

on Flash-Lite because output is decode (sequential, one pass per token) and input is prefill (parallel, once) — so capping output length is usually the highest-leverage line. Context caching sells a stored prefix at roughly a tenth of input price, which is why the slider shows ~90% input savings and why the stable part of the prompt must come first. Batch endpoints are around half price for non-urgent work. Thinking models bill hidden scratchpad tokens at output rates. Cost per user per month = queries × (in × in-rate + out × out-rate); run it at 100 and 100,000 users.

Speed, reliability, observability. Latency is two numbers: TTFT (prefill, set by prompt length) and tokens/s (decode, set by output length). Streaming exposes only the first. Report p50 and p99, never the mean — with LLMs the tail is structural because output length varies. Retry with exponential backoff *and jitter* on 429/5xx, capped; timeouts with a deliberate fallback; idempotency keys on anything with a side effect (a retried “send” sends twice). Log request id, prompt version, model id, token counts, TTFT and total latency, cost, and thumbs; the S2 eval set runs before every prompt change and its score goes in the log. Model id matters because an alias can move under you — the flash alias did in August.

Responsible AI and jurisdiction. Bias (your eval failure rows are a bias audit in miniature), provenance (watermarks, content credentials), privacy (whose data is in the window), accountability (Air Canada). EU AI Act: risk-tiered, applies where the output is used, obligations phasing in through 2027. US: sectoral plus NIST AI RMF (Govern, Map, Measure, Manage). India: the DPDP Act is data-protection law — consent and purpose limitation land on your prompts, index and logs. Gulf: data residency shapes architecture — the S5 local-model argument in law. Not legal advice; dates move; the architectural consequences (residency, retention, PII redaction before the prompt) do not.

The capstone rubric. Working demo 40 · honest eval numbers 25 · failure analysis 15 · technique fit 10 · presentation 10. The certificate page and the S6 slide agree. Security and cost are not separate lines; they are graded through the failure analysis (a found hole and its fix is a failure mode with a mitigation), technique fit (a human gate on a side-effect tool is “the right tool for the job”), and the ship checklist that Part D audits.

What the deck simplifies

The attack and defence widgets are scripted for classroom reliability; real behaviour is probabilistic and model-version-dependent — have a second payload ready. “0–2 layers falls, 3–4 holds” is a teaching threshold, not a measured curve. The cost slider uses 3.5 Flash’s prices and ₹95.5; relabel it aloud as “previous-generation Flash, September 2026” and give the Flash-Lite number from this guide as the lab’s actual rate. “Works in Colab is the easy 20%” is a proportion of effort, not a measurement. The jurisdiction slide compresses four legal regimes into four cards; say once that none of it is legal advice.

Misconceptions and the one-sentence correction

Injection is a bug to patch. No parser boundary exists; you layer defences and cap damage (point at “Defense in depth” and its panel). *A classifier or regex will stop it.* Text filtering raises cost; the attacker adapts; only the tool boundary caps damage. *If the direct attacks bounce, my app is safe.* Direct attacks test the model’s refusal of users; your app also feeds it text it did not write (point at “Indirect injection: the document attacks your RAG”). *Cost is one number.* Output is 6–8× input; caching changes input; thinking changes output. *Compliance is a legal problem for later.* Residency, retention and redaction are architecture decisions made on day one (point at “Your users decide whose law you’re under”).

What the lab evidences, and what a failing lab teaches

Part A evidences that direct attacks mostly bounce on a current model and that this proves the model resists users, not that the app is safe. Part B evidences that layers change *which class* still lands, not a clean before/after. Part C — the classmate swap — evidences indirect injection against a real app; Cell 5 evidences it against a poisoned chunk. Part D evidences the ship checklist as a roadmap.

A room where every direct attack bounces teaches exactly what the notebook comment says: vendors patched direct jailbreaks hard; write it down as a finding and move to Cell 5. An indirect payload that *also* bounces teaches that safety training has improved on that model version — try the second payload (roleplay framing inside the document, or an instruction phrased as a helpful note to the assistant), and if that bounces too, say plainly that the class of attack is mitigated on this model today and not closed, which is the honest position. A

`TypeError` from a blocked response (`candidates_token_count` is `None` , `r.text` is `None`) teaches that the API itself has a

safety layer that returns nothing — the fixed `ask()` handles it. A pair whose capstone does not run teaches that the checkpoint is the hole plus the fix; red-team the naive bot instead.

Part 3 — The question bank

Answers are two to six sentences, mechanism first. **[contested]** marks answers where the honest reply is that the field disagrees; **[unknown]** marks answers where nobody knows. “Point at” names the slide or widget that carries the answer on screen.

3.1 From students with no ML background

1. Is ChatGPT just a very big autocomplete? The core loop is the same operation as autocomplete — score every possible next token, pick one, repeat — so yes as an architectural claim. What is not the same is the machinery that produces the scores: a learned function over the whole context with hundreds of billions of parameters instead of a lookup of the last word. “Just” is doing no work in that sentence; the capability ceiling is set by the function, not the loop. Point at “ChatGPT is autocomplete at scale.”

2. Where does it get the answer from if it isn’t searching? It computes it. The training process compressed patterns from trillions of tokens into the parameters, and at inference the prompt flows through those parameters to produce a distribution over next tokens. Nothing is looked up; that is why it can write a pirate poem about jigarthanda that no one has written, and why it can be confidently wrong. Point at “Wait — is it just searching a giant database?”

3. Does it remember our conversation? No. The model has no state between calls. The app re-sends the entire conversation into the context window each turn, which is why long chats get slower and eventually “forget” the start. Point at “The context window: its entire working memory.”

4. Does it learn from what I type? Not while you use it — the parameters are frozen. On the Gemini API free tier your prompts *may* be used to improve future models offline, which is a separate thing and is the reason not to paste private data. Paid tiers do not use your data that way. Point at the quiz on “Training vs using: the cookbook rule.”

5. Why did my friend get a different answer to the same question? The model produces a probability distribution over the next token and samples from it. Two runs draw different dice. Temperature controls how peaked the distribution is; at temperature 0 you get the most likely token every time and near-identical answers. Point at the temperature widget.

6. What is a token? A learned chunk of text — roughly three-quarters of an English word, and a few characters of Tamil because the vocabulary was fitted to English-heavy data. The model reads, counts and bills in tokens. Point at “Models don’t read words. They read tokens.”

7. What is a parameter? One learned number. $y = wx + b$ has two, w and b ; a frontier model has hundreds of billions to a trillion. A model file is those numbers and nothing else — no sentences, no facts written down anywhere. Point at “What’s inside a model? Just knobs.”

8. Why can’t it count the r’s in strawberry? Because it never sees letters. The tokenizer hands it `str·aw·berry` or similar as integer ids, and nothing in the input distinguishes the r’s. Newer models pass because labs drilled them on that specific question; the mechanism is unchanged. Point at “Famous failures.”

9. Why is it bad at multiplication when it can write essays? There is no carry operation anywhere in the network. It predicts digits that look plausible given the digits before, which works for numbers it has seen and fails for 847×923 . The fix is a calculator tool — Session 5.

10. Does it know what happened yesterday? No; training ended months before release. Anything after the cutoff has to be put into the window — search or your documents — which is what RAG and tools do.

- 11. Why does it sometimes make things up?** The training objective rewards the most plausible continuation, never truth. A fluent, confident, wrong sentence scores as well as a fluent, confident, right one, and the model has no internal check that distinguishes them. That is why Session 2 measures instead of trusting. Point at “One of these is a confident lie.”
- 12. What is “temperature” really?** A single division. The model's raw scores are divided by T before they are turned into probabilities, so $T < 1$ exaggerates the leader and $T > 1$ flattens the field. It is not a creativity setting; the model is not more imaginative at $T = 2$, only less selective. Point at the temperature widget's depth panel.
- 13. What does “training” cost?** Hundreds of crores of rupees in compute for a frontier model, months on thousands of accelerators, once. Inference — using it — costs fractions of a rupee per call, forever. The two moments are different in every way. Point at “Training vs using.”
- 14. Why do I need an API key?** It is your identity and your quota. The gateway checks it before the model runs; a 429 means you exceeded the quota, not that anything is broken. Treat it like a password. Point at “What actually happens when you call an API.”
- 15. Is the AI on my laptop?** No. Your laptop sends an HTTPS request; a data centre runs the model; tokens come back. That is why a ₹15,000 laptop and a workstation get the same answer. The one exception is Session 5's Ollama demo, where a small model really does run on the instructor's machine.
- 16. What is an embedding, in one sentence?** A list of numbers that places a piece of text in a space where nearby points mean similar things — “GPS for meaning.” Session 4 turns it into search.
- 17. How does it know that “marks to clear the subject” means “50% aggregate to pass”?** The embedding model was trained so that paraphrases land close together in vector space. The two sentences share no words but share a direction. Point at “Keyword search misses meaning.”
- 18. What does RAG stand for and what does it do?** Retrieval-Augmented Generation: find the right few paragraphs from your documents, staple them into the prompt, and have the model answer only from them with citations. It is the pattern behind most “chat with your documents” products. Point at “RAG, end to end.”
- 19. If the model can call a calculator, does it run code on my computer?** No. It writes a request — “call calculator with 2347×0.18 ” — and *your* code decides whether to run it. Control stays with you. Point at “The model never executes anything.”
- 20. Can I run this offline at home?** Yes, with an open-weight model through Ollama — `ollama run gemma4:e4b` — on an 8 GB laptop. It is slower and less capable than the API and it is yours. Point at “Ollama: a model in your pocket.”
- 21. What is the difference between ChatGPT and GPT?** ChatGPT is the app; GPT is the model inside it. Like WhatsApp and the phone network. In the lab you skip the app and talk to the model directly. Point at “AI, ML, GenAI, LLM.”
- 22. Why do I have to write ten questions with one-word answers?** Because Session 2 needs a test set with answers a program can check. One-word answers make the scorer simple and honest; sentences make every score an argument.

3.2 From strong students

- 23. Where exactly does temperature enter the maths?** $p(i) = \exp(z_i/T) / \sum_j \exp(z_j/T)$. Dividing the logits inside the exponential rescales the gaps between them; the exponential then amplifies or dampens the leader. $T \rightarrow 0$ recovers argmax; $T \rightarrow \infty$ recovers uniform. Top-k and top-p truncate the candidate set before renormalising.
- 24. Why is temperature 0 not deterministic?** Floating-point addition on accelerators is not associative; batching changes summation order; near-tied logits flip. Mixture-of-experts routing can also depend on batch composition. Close to reproducible, never guaranteed — which is why evals run three times even at $T = 0$.
- 25. What is the loss, exactly, and why cross-entropy?** $-\log p(\text{actual next token})$, averaged. Minimising it maximises the likelihood of the training corpus under the model, equivalently minimises KL divergence from the empirical distribution. Confidently wrong is punished far harder than unsure: $-\ln 0.001 = 6.9$ versus $-\ln 0.1 = 2.3$. Perplexity is $\exp(\text{loss})$, the effective branching factor. Point at the depth panel on “What's inside a model?”

- 26. Why divide by $\sqrt{d}$ in attention?** Dot products of d -dimensional vectors with unit-variance components have variance proportional to d . Without scaling, logits grow with d , softmax saturates toward one-hot, and gradients vanish. Dividing by $\sqrt{d}$ keeps the logits $O(1)$ regardless of head width.
- 27. Attention is permutation-invariant — so how does it know word order?** It does not; position is added to each embedding before attention runs. Sinusoidal encodings in the 2017 paper; RoPE today, which rotates queries and keys by a position-dependent angle so their dot product encodes relative distance. Point at the depth panel on “You saw this machine on the pre-class page.”
- 28. Why n^2 and what do people do about it?** n queries against n keys. 1k tokens $\rightarrow 10^6$ pairs; 1M tokens $\rightarrow 10^{12}$ pairs. Mitigations: sliding-window attention, grouped-query attention (shared K/V across heads, which also shrinks the KV cache), sparse patterns, and — at the application level — retrieval so you never send the textbook.
- 29. What is the KV cache and why does it matter for cost?** Keys and values for already-processed tokens do not change, so they are computed once and cached. Prefill (all input tokens, parallel) sets time-to-first-token; decode (one token at a time against the cache) sets tokens per second. Output is sequential and unbatchable, which is why every provider prices output 6–8× input. Context caching rents a stored prefix. Point at the depth panel on “The context window.”
- 30. How big is the KV cache?** Per token, roughly $2 \times \text{layers} \times d_{\text{kv}} \times \text{bytes per value}$ (2 for K and V). With multi-head attention $d_{\text{kv}} = d_{\text{model}}$; with grouped-query attention it is a small fraction, which is one of the reasons million-token windows became serveable. For a long conversation this is gigabytes per user, which is what you are paying for.
- 31. Where in the network do facts live?** Mostly in the MLP blocks, which are about two-thirds of the parameters and behave like key-value memories: the first matrix asks “is this vector about X?”, the nonlinearity keeps the yeses, the second matrix adds the stored direction. Facts are smeared across many neurons by superposition, which is why you cannot edit one out. Point at Learning Guide Part 9.
- 32. What is superposition?** Packing more features than dimensions by using directions that are only nearly orthogonal. In 12,288 dimensions two random unit vectors are at $90^\circ \pm 0.5^\circ$, so millions of near-orthogonal directions coexist. Consequence: single neurons are polysemantic, which is why interpretability is hard.
- 33. What is the Chinchilla result and does it still hold?** Hoffmann et al. 2022: for a fixed compute budget, scale parameters and training tokens together — about 20 tokens per parameter at the budgets studied — and a smaller, better-fed model beats a larger, under-trained one. The shape holds; the constant has been revised, and modern models deliberately over-train well past 20 tokens/parameter because inference cost dominates the lifetime bill. Point at the depth panel on “The loop is old. The scale is new.”
- 34. Is emergence real? [contested]** The phenomenon — abilities present at scale and absent below — is well documented. Whether it is *discontinuous* is contested: Schaeffer et al. 2023 showed several sharp jumps become smooth under continuous metrics. Say “capability follows compute smoothly; some thresholded metrics make it look sudden.”
- 35. How does RLHF actually work, and what is DPO?** RLHF: humans rank model outputs; a reward model is trained to predict the ranking; the policy is optimised (PPO) to maximise reward minus a KL penalty toward the SFT model. DPO derives a closed-form loss on (preferred, rejected) pairs that optimises the same KL-regularised objective without a separate reward model or RL loop, so preference tuning becomes supervised training. Both shape tone, refusals and helpfulness; neither adds knowledge.
- 36. How do reasoning models differ architecturally?** They do not. They are trained — largely by reinforcement learning on tasks with checkable answers — to emit a long hidden chain of tokens before the visible answer. The chain is billed as output. Same loop, a new habit, and the cost rule follows directly. Point at “Reasoning models: think longer, not just train bigger.”
- 37. What is MoE and why did parameter count stop predicting cost?** Each layer has many expert MLPs and a router that activates a few per token. Total parameters can be a trillion while active parameters per token are a few percent, so serving cost tracks active parameters. Point at the depth panel on “Why ChatGPT $\neq$ Gemini $\neq$ Claude.”
- 38. How does distillation work?** A small student is trained to match the large teacher’s full output distribution (soft targets) rather than just the correct label. The soft targets carry the teacher’s uncertainty structure, which is more information per example. Nearly every Flash/Haiku/mini is a distilled sibling.

- 39. Why does few-shot work with no weight update? [contested]** The examples are in the context and every later token attends to them. Mechanistically, the field has evidence both that attention layers can implement something like gradient descent on the in-context examples and that much of the effect is format and label-space matching. For the room: it works, it is bounded by the window, and it disappears when the examples leave the context.
- 40. Why does chain-of-thought help?** A forward pass has fixed depth, so a computation that needs more sequential steps than the network has layers cannot fit in one pass. Emitting intermediate tokens spreads the computation across passes, and each token is a scratch value the next pass can read. It is more serial compute, not more intelligence.
- 41. Is a normalised-contains scorer sound?** No scorer is sound on free text. Contains is high-recall and low-precision: a generic expected string false-positives, and an answer containing the fact plus a contradiction passes. Exact match is the reverse. A judge is flexible and biased. Pick the cheapest scorer that does not lie about *your* task and read the failures.
- 42. Where does $\pm 1/\sqrt{n}$ come from?** A pass/fail score is a binomial proportion with standard error $\sqrt{p(1-p)/n} \leq 0.5/\sqrt{n}$. Two standard errors at $p = 0.5$ is $1/\sqrt{n}$. $n=10 \rightarrow \pm 0.32$, $n=100 \rightarrow \pm 0.10$, $n=1,000 \rightarrow \pm 0.03$. Paired comparison shrinks the noise because concordant items contribute zero to the difference.
- 43. What are the known biases of an LLM judge?** Position (prefers the first candidate in a pairwise comparison), verbosity (prefers longer), self-enhancement (prefers its own family's style); Zheng et al. 2023 measured all three on MT-Bench. Fixes: swap order and require agreement, hand-grade 20–30 items and measure agreement, and never let the model under test be the judge.
- 44. How does constrained decoding actually enforce a schema?** The schema is compiled to a grammar; a state machine tracks the position in it; at each step tokens that cannot extend a valid prefix have their logits set to $-\infty$ before softmax. Malformed shape has probability exactly zero. It says nothing about values. Point at the depth panel on “All of it is one API call.”
- 45. Why is cosine similarity a dot product after normalisation?** $\cos(a,b) = a \cdot b / (|a||b|)$. If $|a| = |b| = 1$ the denominator is 1. Normalise once at index time and query time, and `chunk_vecs @ q` scores every chunk in one matrix multiply. Point at the depth panel on “Embeddings, now for whole paragraphs.”
- 46. Why can you truncate a 3072-dimensional embedding to 768?** Models like gemini-embedding-2 are trained with a Matryoshka-style objective that concentrates signal in the leading dimensions, so a prefix is a usable lower-resolution embedding. You lose a little recall and gain 4× on storage and search time. Never truncate one that was not trained for it.
- 47. Bi-encoder versus cross-encoder?** A bi-encoder embeds question and passage separately, so all passage work happens at index time — fast for a million chunks, mediocre at fine ranking. A cross-encoder reads question and passage together and scores true relevance — accurate, too slow for a corpus, ideal for re-ordering 20 candidates. That asymmetry is the whole case for retrieve-wide-then-rerank. Point at the depth panel on “RAG, end to end.”
- 48. What are recall@k and MRR, and why two numbers?** Recall@k: fraction of test questions whose gold chunk appears in the top k — the ceiling on answer accuracy. MRR: mean of $1/\text{rank}$ of the first correct chunk. High recall, low MRR means you find it and bury it — add a reranker. Faithfulness is the third number and belongs to generation. Point at the depth panel on “Where RAG breaks in the wild.”
- 49. Why does BM25 still matter if embeddings are better?** Embeddings blur exact identifiers — course codes, error numbers, surnames, “Section 4(b)” — that a keyword index matches exactly. Hybrid search merges both rankings. Neither is redundant.
- 50. What does the function-calling wire format look like?** Your function becomes a JSON schema (name, description, parameters with types and descriptions, required list). The model emits a `functionCall` part with the name and arguments; the turn ends; you append a `functionResponse` part; the model continues. Four roles, all in context. Point at the depth panel on “Function declarations: the description IS the prompt.”
- 51. Can the model call several tools in one turn?** Yes, when the calls are independent it can emit several `functionCall` parts in one response and expects all results back. Running them in parallel is free latency.
- 52. What is p^n hiding?** Independence. Real step failures correlate (a bad early retrieval poisons every later step), so the true chain reliability is usually worse than p^n . Validation between steps breaks the chain by converting silent failures into caught ones.

- 53. Why is memory bandwidth the bottleneck for local inference?** Generating one token reads every active parameter once. A 2 GB Q4 model on a laptop with ~50 GB/s memory bandwidth cannot exceed ~25 tokens/s no matter the CPU. Quantization halves the bytes and therefore roughly doubles the speed. Point at the depth panel on “Ollama: a model in your pocket.”
- 54. What does quantization break first?** The hard tail — long multi-step reasoning, exact output formats, rarer languages. Chat and summarisation survive Q4 well. Test with your eval set, not with a casual chat.
- 55. Why is there no escape character for prompt injection?** Every other injection defence relies on a parser with a mechanical code/data boundary — SQL quoting, HTML entities, shell argument arrays. A transformer receives one flat token sequence, and “these tokens are instructions” is a trained preference that a persuasive sequence can outvote. Instruction-hierarchy training is the current attempt and it is an open problem. Point at the depth panel on “Defense in depth.”
- 56. What is the lethal trifecta?** Willison’s framing: exfiltration needs private data access, exposure to untrusted content, and a channel out, all at once. Remove any leg and that attack class dies regardless of prompt cleverness. A RAG bot over private notes that ingests untrusted PDFs is safe *until* you add an “email this” tool.
- 57. Why do direct attacks bounce but the poisoned document lands?** Safety training targets a *user* asking the model to misbehave, and it is now good at refusing that. The poisoned chunk arrives inside content the developer told the model to trust (“answer ONLY from this context”), so the injected instruction and the grounding instruction are the same kind of token. Lab 6 Cell 5 measured exactly this.
- 58. Why is output priced 6× input?** Input is prefill: all tokens in parallel, once. Output is decode: one sequential pass per token, holding the KV cache in memory, unbatchable per request. The asymmetry is physics, and it is why capping output length is the highest-leverage cost line.
- 59. p50 vs p99 — why not the mean?** Output length varies, so LLM latency has a structural long tail. The median is the designed experience; the 99th percentile is the complaint. A mean of 2.4 s hides a 30 s p99.
- 60. What is idempotency and why does it matter here?** A retried operation that has the same effect as running it once. Reads are naturally idempotent; “send email” is not, so a retry after a timeout sends two. Anything with a side effect needs an idempotency key or a pre-check before it is safe to retry.

3.3 From faculty and sceptics

- 61. Isn’t this just statistics?** The objective is statistical — maximum likelihood on next-token prediction — and saying so is accurate, not a dismissal. What is not “just” is the function class: a deep network with attention that learns compositional structure and generalises to sequences never seen. The honest framing for a CSE faculty room: it is statistics with a function approximator large enough that the behaviour needs empirical study, which is why evaluation is a first-class topic in this course. **[contested]** on how much of the capability is memorisation versus generalisation.
- 62. Why teach a proprietary API instead of the theory?** The course teaches the theory — tokens, attention, loss, sampling, embeddings, retrieval — and uses one API as the lab bench because a free key lets forty students run real models in fifty minutes on any laptop. Every notebook has one `MODEL` variable; the mechanics transfer to any provider or to a local model, which Session 5 demonstrates live. The pre-class page trains a real GPT from scratch on Thirukkural in the browser for anyone who wants the raw version.
- 63. Will this be obsolete in a year?** The model names, prices and free-tier limits will be — the course keeps those in a volatile table and re-verifies them before every run. Tokens, embeddings, attention, sampling, training-vs-inference, evaluation, retrieval, tool use and the injection failure mode are consequences of how the technology works, not product decisions, and have been stable since 2017–2020. Point at Learning Guide §7.3.
- 64. What about Indian languages?** Tokenisers fitted on English-heavy corpora fragment Tamil into more tokens per sentence — more cost, more latency, less effective context — and training data is thinner, so quality is uneven; the Lab 1 stretch measures the token ratio with `count_tokens` and Gemini’s is narrower than the toy on screen suggests. Handwriting and speech in Tamil are where Lab 3 finds failures. Multilingual tokenizers and Indic-specific models are improving this; it is a live research area and a good final-year project (the Tamil scheme assistant track). **[contested]** on how fast the gap closes.

65. Does it understand? [contested] Not a question this course can settle, and it says so. What can be said: it has no persistent state, no goals, no world model that anyone has located, and it produces confident right and confident wrong answers through identical machinery. Redirect to something testable: whether it produces correct answers on *your* test set is measurable; whether that constitutes understanding is philosophy of mind.

66. Can this be used for setting or grading our exams? Setting: yes, with a human editing every item and an eval on the output — it is good at generating variants and bad at guaranteeing correctness. Grading: only with a rubric, a scorer you have audited on 20–30 hand-graded scripts, and a human on the blast radius (marks are a side effect that bites). The S2 judge-bias panel is the exact reason: an unaudited AI grader prefers longer answers and whichever came first.

67. Is prompt engineering a real skill? As a job title, no — it is being absorbed. As a skill, yes in the unglamorous sense: specifying a task precisely, anticipating a failure mode with a constraint, and measuring the difference. The measuring is the real skill; the course calls it eval-driven development and it outlives any prompt.

68. Why not teach LangChain? Because the course builds RAG as a numpy array and tools as plain Python functions so students understand what a framework hides before it hides it. Once you have written the 60 lines, LangChain is a convenience you can debug; if you have only ever used the framework, its failures are opaque. It is named as an honest next step, not a gap.

69. Is RAG dead now that context windows are a million tokens? [contested] For a small static corpus queried rarely, pasting is fine and the course says so. For anything large, changing, or needing citations, three taxes remain: input cost scales with tokens every call, latency scales with prefill, and models attend worst to the middle. Real systems increasingly do both — retrieve, then hand a large context to the model. Point at “Just paste all my notes — let’s price that.”

70. Are agents hype? [contested] The loop is real and the demos are real; the failure arithmetic is also real — $0.95^{10} \approx 0.60$. Most business AI that ships is a workflow with fixed steps and a model inside each. “Agent” earns its name when the path is genuinely unpredictable and the leash is in place. Point at “Why long agent chains collapse.”

71. Who is liable when it hallucinates? In the one adjudicated consumer case, the company: *Moffatt v. Air Canada*, 2024, the tribunal held the airline to its chatbot’s invented policy. In *Mata v. Avianca*, the lawyers who filed the fabricated citations were sanctioned. “The AI decided” has not worked as a defence. Not legal advice; the architectural consequence is a human on anything that bites.

72. How do we assess students fairly if they use AI? Assess artefacts AI cannot produce for them: a test set they wrote from a subject they know cold, a documented failure and its fix, a number with an n , a demo that survives one unrehearsed input. The capstone rubric is built that way. The course’s own rule is “AI assistants are allowed; your eval set judges their code too.”

73. What is the evidence this pedagogy works? Honest answer: the course has a designed evaluation — checkpoints, a graded capstone with a rubric — and no controlled study. The claim it makes is narrower: every student leaves with six working artefacts and can explain each to a neighbour, which the checkpoint sweep tests directly. **[unknown]** whether it improves placement outcomes; the placement roadmap says so explicitly.

74. Aren’t you overstating the risks (injection, deepfakes)? OWASP has ranked prompt injection first in two consecutive editions; voice-clone fraud is a current pattern in India; the poisoned-document attack lands live in Lab 6 on a current model. The course also says what *does* work — a family password, a human gate, the trifecta — so the risk beats end in an action, not in anxiety.

75. How does this map to our syllabus outcomes? `ASSESSMENT.md` maps six course outcomes to a 52-question bank across Bloom levels and includes a model paper on the 2/6/15-mark pattern; `COURSE-PLAN.md` maps each session to the original ten topics. The “Responsible AI: the four questions” slide is written for this audience.

76. Is the free tier really free, and is it appropriate to require students to sign up? No card is required on the designed path; the trade is that free-tier prompts may be used for training, which the course tells students on Day 1 and designs around (own notes, no private data). Availability and quotas are not guaranteed, and the course has a fallback: instructor-driven demonstration and, if built, a mock path. Nobody is asked to create extra accounts or share keys.

77. What about energy and water use? [contested on magnitudes] Training a frontier model consumes energy on the order of a small town for months; inference per query is small individually and large in aggregate. Published per-query figures vary by an order of magnitude

and depend on model size, batching and hardware; quote a range or none. The engineering levers the course teaches — smaller models for routine work, caching, capped output — reduce it directly.

78. Is the “autocomplete can pass your exam” hot take an attack on our exams? It is a claim about assessment design: if a next-token predictor scores well, the paper measured recall and pattern more than transfer. The counter-argument the room usually gives — that recall is a prerequisite for transfer and exams are the cheap way to check it — is a good one, and the close names the strongest counter-argument aloud with credit. Point at “If autocomplete can pass your exam...”

3.4 About the tools — Gemini, Colab, keys, quotas, privacy (September 2026 facts)

79. What is the difference between an auth key and a standard key? From September 2026 the Gemini API rejects requests from “Standard” keys; new keys created in AI Studio are auth keys automatically. Students creating keys fresh are fine. Instructor keys created before the change must be regenerated — that is the first item on the ten-day plan.

80. A student has a Google Cloud account and “Get API key” does nothing. AI Studio no longer creates a default project for accounts that already have Google Cloud; the student must import an existing project first. Walk them to the project selector; if they have no usable project, pair them with a working machine and both type.

81. Which model does the lab use and why? `gemini-flash-lite-latest`, an alias currently serving Gemini 3.5 Flash-Lite (no newer Lite exists; resolve it live with `client.models.get` the week before). Lite is chosen because it runs with *minimal* thinking by default, so the free output quota lasts the lab; because it is verified for vision, function calling, streaming, `response_schema` and `max_output_tokens`; and because an alias serves both new accounts (where `gemini-2.5-flash` returns 404) and old ones.

82. Why not `gemini-flash-latest`? That alias is hot-swapped to each new Flash release — 3.7 Flash on 13 Aug, 3.8 Flash on 2 Sep — and those are thinking models. A measured session billed 28,089 hidden reasoning tokens against 2,770 visible, 91% of output, and on the free tier that drains output quota roughly an order of magnitude faster. Keep it as the S1 slow-and-verbose comparison and as a fallback, not as the default.

83. Can thinking be turned off? Not with `thinking_budget` — numeric budgets are rejected on Gemini 3.x. `thinking_level="minimal"` exists on 3.5 and 3.6 Flash but not on 3.7/3.8. Flash-Lite’s default is minimal thinking, which the docs describe as not guaranteeing thinking is off; you may pin `thinking_config=types.ThinkingConfig(thinking_level="minimal")` on the lite alias to be explicit.

84. Why do the official docs look different from the notebook? The docs now default to the Interactions API (`client.interactions.create(...)`, GA June 2026). `generate_content` is fully supported and is what every notebook uses. Same model, different client surface; nothing in the labs needs to change.

85. The temperature cell shows no difference between 0 and 1.5. `temperature`, `top_p` and `top_k` were deprecated on Gemini 3.x in the 21 July 2026 changelog and were still accepted on 3 August. If on the day the parameter is ignored, say: the provider has taken the sampling dials away from you, which is exactly the “names change, mechanics don’t” lesson — the softmax is still there, you just no longer hold the knob. Run the eval three times instead to show variance.

86. Why does my embedding store have three rows for sixty chunks? You passed a bare list of strings; `gemini-embedding-2` returns one *aggregated* embedding for the whole list. Wrap each text as its own `types.Content(parts=[types.Part.from_text(text=t)])` to get one vector per chunk, and keep the assertion `len(res.embeddings) == len(texts)`.

87. Why 768 dimensions when the default is 3072? `output_dimensionality=768` matches the deck’s stepper, cuts storage and search time by 4×, and loses little because the model’s leading dimensions carry most of the signal. Whatever you pick, use the same value for chunks and queries and store it beside the index.

88. Is `gemini-embedding-001` shut down? No. It remains available with an earliest shutdown of 14 May 2028; 14 July 2025 was its *release* date. It is superseded by `gemini-embedding-2` (GA 22 April 2026), which the lab uses.

- 89. What are the free-tier rate limits?** Google no longer publishes per-model numbers; the docs defer to your project's live limits in AI Studio. The materials say “~10 requests/min, a few hundred a day, varies” as a hedge. The honest answer in the room is the screenshot of the rate-limit page you took the day before.
- 90. What does a 429 mean and what do I do?** Quota exceeded at the gateway; nothing is broken. The notebooks' `ask()` retries with 20/40/60-second backoff. If a whole room hits it at once, stagger: half run while half write test questions.
- 91. What does a 503 mean?** Capacity on the provider's side (“high demand”). Wait and retry; if persistent, the flash alias is the fallback at the cost of thinking tokens.
- 92. Is my data used for training?** On the free tier of the Gemini API, prompts and responses may be used to improve Google's products; on paid tiers they are not. Hence: own notes, no ID numbers, no confidential documents, and a `getpass` prompt for the key so it never sits in a cell.
- 93. Where should the API key live?** In `getpass` during the lab; in Colab secrets or an environment variable for anything saved; in Streamlit secrets or an env var when deployed; never in a notebook you push. A key in a public repository is scraped within minutes.
- 94. Colab disconnected at lunch and my embeddings are gone.** Colab's runtime disk is ephemeral. Lab 4 mounts Drive and saves `chunk_vecs.npy` and `chunks.json` there; Lab 5 and Lab 6 have a reload cell. If a pair skipped the mount, re-embedding 60 chunks is about 12,000 tokens and three batched calls — cheap, but rate-limited if the whole room does it at once.
- 95. My PDF extracted nothing.** It is a scanned image; `pypdf` reads text layers, not pixels. Swap to a text PDF or the sample document at minute five, or run the pages through Session 3's vision extraction first. The lesson is retrieval, not parsing.
- 96. Prices for the cost slide?** Per million tokens, September 2026: Gemini 3.5 Flash \$1.50 in / \$9.00 out (the slider's constants, now previous-generation); 3.5 Flash-Lite \$0.30 / \$2.50; 3.8 Flash \$0.75 / \$3.75 introductory to 31 December; gemini-embedding-2 \$0.20 input. About ₹95 per dollar. A typical Lab-4 RAG query costs about ₹0.13 on Lite.
- 97. Does `response_schema` guarantee valid JSON?** It guarantees the *shape* by masking illegal tokens at the decoder; `json.loads` will not throw on the text. It does not guarantee the request succeeds (a blocked response returns no text), the values are right, or the model is honest about unreadable input — give the schema a null outcome and parse defensively.
- 98. Why does the S1 handout compare against the flash alias instead of ChatGPT?** Because the college network or a missing account may block the web UIs, and the alias comparison teaches the same lesson without a login — plus the thinking-cost lesson for free: the same prompt comes back slower and longer.
- 99. Can students run Ollama in the lab?** No — it is an instructor demonstration; student labs use the API. At home, 8 GB RAM runs a 3–4B model; without a machine, the same model runs in a free Colab runtime.
- 100. Is automatic function calling still supported on the lite alias?** Verified 3 August; re-verify in the ten-day plan. Lab 5 Part C disables it deliberately to show the raw `function_call`, and the guarded loop builds the `function_response` by hand with `types.Part.from_function_response`.

3.5 About careers and placements

- 101. Will this get me a job?** Twelve hours does not make an AI engineer; it makes someone with six working artefacts who can explain how a model works without hand-waving. Most campus offers are general software roles; this makes you the memorable candidate in that pile. DSA preparation is not replaced. Point at `PLACEMENT-ROADMAP.md`.
- 102. What should go on my resume from this?** A deployed URL, a measured number, and the specific problem: “retrieval over 400 pages of regulations, 84% on a 100-question test set against 61% ungrounded, ~₹40/month.” Not a list of fields. Never claim fine-tuning unless you did it; it collapses under one question about training-data format.
- 103. What is a forward deployed engineer?** The role whose working week is exactly the capstone shape: embed with a customer, build on a platform's models, prove value with numbers. Realistic for a fresher at AI startups with two deployed customer-style projects and

strong communication. The roadmap has a section on it.

104. Which project should I do after this? One user, one document type, one measurable claim. `COLLEGE-PROJECT-TRACKS.md` gives Level 1 (6–10 h: study-notes RAG, receipt extractor, prompt-eval dashboard), Level 2 (12–20 h: regulations assistant with injection testing, codebase RAG, Tamil scheme assistant) and Level 3 (semester: observability dashboard, RAG+tools with approval gates, department knowledge service), each with minimum evidence.

105. What interview questions should I be able to answer? The roadmap lists them. Fundamentals: what a token is, what happens between prompt and first word, why output is dearer than input. Building: RAG end to end and where it breaks; right chunk retrieved but wrong answer — diagnose. Judgement: how do you know it works, cost per user and how to halve it, agent or workflow and why, what would stop you shipping.

106. Should I learn LangChain next? After you have built the raw version, yes as a convenience — you will recognise every piece. Frameworks, rerankers, hybrid search, LoRA/QLoRA and tracing tools are the honest next steps in Learning Guide §7.2.

107. How do I turn the capstone into a final-year project? Add a baseline you beat, a 100-question eval set a person wrote, one documented attack and mitigation as the security chapter, and the cost arithmetic. The sentence a panel wants: here is the metric, here is the baseline, here is where it fails.

108. Is “prompt engineer” a job to aim at? Treat it as unlikely; it is being absorbed into other roles. Evaluation, retrieval and shipping are the durable skills.

109. Do I need a master’s for AI roles? For research, usually. For application engineering and forward-deployed roles, a deployed project with numbers matters more. Data science roles sit in between and want classical ML depth, which the roadmap’s weeks 1–2 address.

110. Is a fintech/KYC background relevant to what you taught? Directly: document extraction with schemas (S3), regulators asking where prompts go (S5’s local-model argument), human gates on anything that moves money (S6). Those are the same three slides that carry the instructor’s own war stories.

3.6 Curveballs

111. Is it conscious? [unknown] There is no agreed test and no mechanism in the chain that anyone has identified as consciousness. What is known: no persistent state, no goals of its own, identical machinery for right and wrong answers. Say that, decline to speculate, and redirect to something measurable.

112. When is AGI coming? [unknown] Forecasts from serious people range from a few years to never, and the term has no agreed definition. What the course can say: capability has followed compute predictably, the data wall is real, and test-time compute is the current axis. Anyone giving a date is guessing.

113. Will AI take our jobs? It changes what juniors are hired for. The person who can specify, test and debug an AI system is more employable, not less — which is the design of the course. Honest uncertainty about the aggregate; concrete advice about the individual.

114. How much energy does one query use? [contested] Published per-query estimates span an order of magnitude; it depends on model size, batching and hardware, and providers rarely disclose. Order of magnitude: a small model answer is comparable to a few seconds of a laptop; a long reasoning answer is much more. Smaller models, caching and capped output are the levers you control.

115. Who owns the copyright on what it generates? [contested] Jurisdiction-dependent and unsettled; several regimes decline to grant copyright to purely machine-generated work, and training-data cases are live in multiple countries. Not legal advice. The engineering consequence: keep provenance, disclose AI-generated content, and do not build a product whose value depends on owning generated text.

116. Can someone clone my mother’s voice? [yes] A few seconds of audio suffices with freely available tools; detection is an arms race. The defence that works is out of band: a family password. Tell your parents this weekend. Point at “Your mother’s voice is no longer proof of your mother.”

- 117. Can I use this to cheat in exams?** You can use it to generate answers; you cannot use it to know whether they are right, which the spot-the-lie game just showed. The skill the course teaches — measuring — is the one an exam is trying to test. And if a next-token predictor passes the paper, that is a question for the paper, which is the hot take.
- 118. How good is it in Tamil? [contested]** Uneven. Translation and simple QA are reasonable; formal register, handwriting, speech with code-switching and rare proper nouns fail more, and every sentence costs more tokens. The Lab 1 Tamil stretch, the Lab 3 handwriting part and `count_tokens` give you data points from this room; quote them rather than a general claim.
- 119. Could it be lying to us on purpose?** It has no goals, so “on purpose” does not apply; it produces the most plausible continuation under its training. Models trained with RLHF do exhibit sycophancy — agreeing with the user — because agreement was preferred by raters. That is a measurable bias, not intent.
- 120. Why does it apologise so much?** Preference tuning: raters preferred polite, hedged answers, so the model learned the register. Hedging is a trained behaviour and is not evidence that the model knows what it does not know.
- 121. Are open-weight models from China safe to use?** Weights are numbers; they cannot phone home. The concerns are licence terms, training-data provenance, and behaviour differences, which your eval set measures. Run it locally with Ollama and the data never leaves the machine.
- 122. Could the model be secretly running code when it “calls a tool”?** Not for tools you define — the call is a JSON request you execute or refuse. Provider-hosted tools (code execution, search) run on the provider’s side; that is a different trust boundary and the course names it.
- 123. Isn’t a 10-question eval laughably small?** Yes — ± 30 points. It is a smoke test and a way to learn the discipline; real systems use hundreds. The course says the honest number on the arena’s depth panel.
- 124. What if a student pastes something private into the free tier?** Stop, note the time, and treat it as a lesson rather than a disaster: the data may be retained for product improvement, cannot be recalled by the student, and this is exactly why the labs use own notes. Report it to the student and, if it is college data, to whoever owns it. Part 4 has the room line.
- 125. Is the pre-class Thirukkural model a real GPT?** Yes: a character-level GPT with about 164k parameters trained on the 1,330 kurals (and on sonnets for comparison), with real weights running in the browser, a recorded loss curve and samples at each step. It produces nonsense at that size, which is the point — coherence grows smoothly with data and parameters, and facts appear when the training text contains them.
- 126. What is the single most important thing to remember?** The chain in Part 1 and one consequence: the model does not know when it is wrong, so you measure. Everything else is technique.
-
-

Part 4 — Failure playbook for the room

Each entry: symptom · 20-second diagnosis · recovery · the teaching line that turns it into content.

- 1. 429s across the room.** Symptom: several pairs see “Rate limited — waiting 20s” at once. Diagnosis: the gateway’s per-project quota, not a bug; check whether it is per-minute (clears in a minute) or per-day (does not). Recovery: stagger — half the room runs while half writes test questions; on a daily cap, move the pair to the instructor demo or the mock path. Line: *“You just met the quota check on the API slide. Production has this too; it is why every notebook has a retry with backoff.”*
- 2. 404 on the model alias.** Symptom: `NOT_FOUND` for `gemini-flash-lite-latest`. Diagnosis: the alias moved or was retired, or the account is on a project without access — run `client.models.list()` in your own runtime and see what resolves. Recovery: change the one `MODEL` line to a working id you saw in the list (3.5 Flash-Lite’s dated id, or the flash alias as a last resort with a warning about thinking

tokens); announce the string once, write it on the board. Line: *“One variable per notebook, and this is why. Names change monthly; the mechanics do not.”*

3. Key creation blocked. Symptom: “Get API key” loops or shows nothing. Diagnosis: college-managed account (Workspace admin has disabled AI Studio), a phone-verification loop, or an existing Google Cloud account that needs a project imported. Recovery: personal account for the first two; import a project for the third; after five minutes, pair them with a working machine — both still type. Do not share keys or create extra accounts. Line: *“Identity and quota live in the key; that is also why you cannot borrow one.”*

4. Colab disconnect at lunch loses Lab 4’s store. Symptom: `chunk_vecs` undefined in Lab 5. Diagnosis: runtime recycled; no Drive mount. Recovery: run the reload cell if Drive was mounted; otherwise re-run Lab 4 Cells 1–4 (three batched embed calls, ~12k tokens) and mount Drive this time. Line: *“Runtime disk is ephemeral; a vector store is state. Persistence is the first thing ‘notebook to product’ changes.”*

5. Scanned PDF extracts nothing. Symptom: Cell 2 raises “Too little readable text”. Diagnosis: image-only PDF; `pypdf` reads text layers. Recovery: swap to a text PDF or `sample-os-notes.txt` at minute five; a fast pair can run pages through S3’s vision extraction first. Line: *“RAG starts with text. Extraction is its own pipeline, and Session 3 is where you built it.”*

6. The injection attack does not land. Symptom: every attack in Lab 6 Cell 3 bounces; Cell 5 also bounces. Diagnosis: model version has stronger refusal or instruction-hierarchy training. Recovery: Cell 3 bouncing is the expected finding — say so and move to Cell 5; if Cell 5 bounces, use the second payload (an instruction framed as a helpful note to the assistant, or roleplay inside the document), and if that bounces, state that the class is mitigated on this model today, not closed. Line: *“A defence that works today is not a proof. OWASP still ranks this first, and the trifecta is how you cap damage when the next payload lands.”*

7. A demo returns nonsense or refuses. Symptom: the live call gives an off-topic, empty or blocked reply. Diagnosis: read `usage_metadata` and the finish reason — a safety block returns no text; a thinking model may have spent its output budget; a wrong `MODEL` may be serving. Recovery: re-run once at T=0; if blocked, show the finish reason and move on with the canned example the slide already carries. Line: *“Same machinery, different sample. This is why the eval runs three times and why the deck’s demos are canned — the real one is your lab.”*

8. A student pastes private data into the free tier. Symptom: you see an ID number, a password or a confidential document in a prompt. Diagnosis: policy, not a technical failure. Recovery: stop the cell, tell the student plainly that free-tier data may be retained for product improvement and cannot be recalled, have them change the password if it was one, and note it for the data owner if it was college data. Line: *“This is the front-page rule from hour one. Nothing you put in a prompt is yours again.”*

9. Projector washes out the deck; dark-theme toggle. Symptom: the diffusion slider’s silhouette-then-detail ordering is invisible; the dot-grid bands; the HUD is unreadable. Diagnosis: projector gamma and contrast. Recovery: the `dark` toggle in the HUD flips every deck (and the index) to the shared `tce-genai-theme`; test both themes on the projector before students arrive and pick the one where the diffusion slider reads. Line: none needed — fix it before 9 a.m.

10. A faculty member challenges a number. Symptom: “that price is wrong / that figure is outdated / where is that from?” Diagnosis: they are probably right about drift; the volatile table exists because of it. Recovery: agree immediately, name the date the figure was verified (`fact-check.md`), give the current number if you have it (Part 2 §S6 has the September prices), and promise the corrected slide. Never defend an illustrative widget as measured. Line: *“Everything on screen is a faithful cartoon — simplified to be visible, never simplified to be wrong — and the numbers that move are in a table we re-check before every run.”*

11. Temperature cell shows no variance. Symptom: three runs at 1.5 are identical. Diagnosis: the parameter is deprecated on 3.x and is now ignored. Recovery: switch the demonstration to run-the-eval-three-times and show the score move. Line: *“The provider took the sampling knob away. The softmax is still there; you just no longer hold the dial — which is the whole ‘names change, mechanics don’t’ lesson in one cell.”*

12. Embedding store has three rows. Symptom: `vector store: (3, 3072)` for 60 chunks; every search returns the same chunks. Diagnosis: bare string list aggregated into one embedding; the fix (one `Content` per text, `output_dimensionality=768`, count assertion) is missing from that pair’s copy. Recovery: paste the fixed `embed()`. Line: *“An API that changes its contract fails silently. The assertion is there so it fails loudly — that is what a test is.”*

- 13. Ollama demo fails on college Wi-Fi.** Symptom: `ollama run` starts downloading. Diagnosis: the model was not pulled on this machine. Recovery: `gemma3:4b` if that was pulled earlier; otherwise the on-slide “Simulate the race” with the words “this is a simulation” said aloud. Line: “*The lesson is the file size arithmetic, not the download — $\text{parameters} \times \text{bits} \div 8$, and the pull is the one thing you do online.*”
- 14. The room is behind at the lab-brief slide.** Symptom: the pace badge reads behind by more than five minutes when you reach the lab slide. Diagnosis: talk overran. Recovery: do not shorten the lab; cut stretch goals and, in S1, Part D — never Part E. Line: “*Checkpoints are what the next session assumes; stretch goals are what fast pairs do.*”
- 15. Too many pairs for three-minute demos.** Symptom: headcount at S6 gives more than twelve pairs. Diagnosis: $3 \text{ min} \times \text{pairs}$ exceeds the ~35-minute slot. Recovery: announce 60–90-second lightning demos before Part A; keep the stopwatch visible; the failure story is the graded part, so ask for the failure first. Line: “*Lead with the problem, show the failure, name the fix. That is the demo.*”
- 16. The whole network dies.** Symptom: nothing resolves. Diagnosis: college uplink. Recovery: decks are single offline files — teach the slides, run the local model if pulled, convert the lab to a paper walkthrough (write the prompts, predict the outputs, run it at home), and say that this is what incident response looks like. Line: “*The model is on another continent. Your laptop was only ever posting letters.*”

Part 5 — Delivery craft specific to these decks

Presenter mode

Press `S` for the presenter panel: session timer, this slide’s budget, cumulative target, a pace badge (on-time / behind / ahead), the speaker note, and the next-slide preview. `D` opens every `<|deeper|>` panel on the current slide (and closes them); `0` is the overview grid; `F` is fullscreen; a number then Enter jumps to a slide. The S2 lab slide also has `L` to start/pause the 50:00 countdown and `R` to reset it (amber at 10:00 left, red at 2:00). The pace badge compares elapsed time to the cumulative budget; it is only as good as the DATA budgets, so know the two places it lies: S1’s slide numbering changed this week (regenerate the slide guide before trusting the table) and S6’s “Six sessions, one throughline” carries the whole last hour.

Compressible slides, in the order to cut them

The deck flags `► compressible` slides; the prep packs give a considered order. Merged:

SESSION	CUT FIRST → LAST	NEVER COMPRESS
S1	Famous failures (four cards → two) · AI/ML/GenAI/LLM rings (90 s) · Why ChatGPT ≠ Gemini ≠ Claude (60 s, do not read the table) · the hot take · the instructor bio	Those odds aren’t magic · Watch the network think · the pipeline stepper · lab Part E
S2	Four prompt crimes (two cards) · Few-shot (Tanglish example alone) · Six ideas recall · Demand a format · the hot take · “It worked when I tried it”	One of these is a confident lie · Watch an eval run (run it twice) · Same answer, three verdicts · the arena’s Q5

SESSION	CUT FIRST → LAST	NEVER COMPRESS
S3	Madurai seeds (one, not three) · Place your bets (three items) · Five ideas recall · Image generation fine print · the hot take	Diffusion slider · All of it is one API call · the Day 1 close and overnight task
S4	You've been using RAG all along (one card) · Six ideas recall · Keyword search misses meaning (one example) · Vector databases · RAG vs paste-it-all vs fine-tuning · the hot take	Search playground query 2 and the trap query · RAG stepper stage 5 · the grounded template's A/B toggle
S5	Which tool should it call (three items) · Six ideas recall · Defend your vote (two items) · MCP · Multi-agent · the hot take · API vs owned · Local vs API	The model never executes anything (read twice) · Why long agent chains collapse · Workflow or agent? One question decides
S6	Jailbreaks & leaks (one door) · Capstone brief (90 s) · Everything in 90 seconds (four items) · UX patterns · Responsible AI · Whose law · Where to go from here · the hot take	Indirect injection · Defense in depth toggle · the demos

The six hot takes and the strongest counter to each

Each is read once, slowly, followed by three seconds of silence and exactly one counter-argument; the strongest one is named at the close with credit. Know the best counter so you can steel-man the room if nobody offers it.

If autocomplete can pass your exam, your exam was never testing understanding. Counter: recall is a prerequisite for transfer and a written exam is the cheapest scalable check of it; the fact that a model passes says the model has recall, not that recall was the wrong thing to test. Your reply: fair — the claim is about what the paper *discriminates*, and a paper that cannot discriminate a student from a next-token predictor is no longer discriminating the thing it was designed for.

Hallucination is an expected failure mode. That does not make it acceptable. Counter: humans confabulate too, and we do not call a person “unacceptable” for occasional error; the bar should be comparative, not absolute. Your reply: agreed on the bar — which is why the session measures rather than moralises; the difference is that a human’s confidence carries information and the model’s does not.

Your mother’s voice is no longer proof of your mother. Counter: provenance and detection technology will restore trust faster than social habits change; passwords do not scale to institutions. Your reply: detection is an arms race and provenance requires adoption on both ends; a shared secret works today and costs nothing, and institutions already use exactly this (OTP is an out-of-band secret).

Your final-year project needs a user, a baseline, and a number. Counter: research-oriented projects legitimately explore without a user or a baseline; insisting on a number kills exploratory work. Your reply: exploration is fine — but a claim needs a measurement, and “we explored” is not a claim; the project tracks have a Level 4 research track with its own evidence bar.

Most production “AI agents” are a while-loop in a trench coat. Counter: the loop is trivial; the hard parts — tool design, state management, evaluation, guardrails — are real engineering that the slogan dismisses. Your reply: exactly, and the slogan is aimed at the pricing and the mystique, not at the engineering; the guarded loop on the next slide is that engineering.

There is no perfectly secure AI agent. Build one whose blast radius you can survive. Counter: no system is perfectly secure, so the claim is vacuous; conventional security engineering has the same posture. Your reply: yes, and that is the point — the difference is that conventional systems have a parser boundary you can reason about and a transformer does not, so the *only* defences that hold are the conventional ones on the tool side.

The capstone rubric and how to score it

Working demo 40 · honest eval numbers 25 · failure analysis 15 · technique fit 10 · presentation 10 (S6 slide, `certificate.html`, `course-plan.md` all agree). The demo line asks for a live run on the free tier surviving one unrehearsed input — the certificate makes that mandatory, the “How to demo” slide calls the wild-card optional; reconcile aloud: pre-run your best example, then take one input from the room.

Scoring guidance. Demo (40): runs live and answers a room input, 32–40; runs live on the pre-run example only, 22–30; runs from screenshots or a recording of an earlier run, 10–18; does not run, 0–8. Eval (25): ten examples, real scores including the unflattering ones, with n stated, 20–25; five examples or scores without failures shown, 10–16; “we tested it a lot”, 0–5. Failure analysis (15): a failure traced to a stage (retrieval / prompt / model / tool) with the mitigation applied and re-measured, 12–15; failure named but not traced, 6–9; none, 0. Technique fit (10): two or more techniques chosen because they fit — a human gate on a side-effect tool, a schema on extraction, retrieval on a changing corpus — 8–10; techniques bolted on to impress, 3–5. Presentation (10): three minutes, problem first, failure shown proudly, 8–10.

The pair whose app does not run but whose failure analysis is excellent. Score the demo line honestly and low — 0–8 — and score the other four lines on their merits, which can reach 60. Say aloud why: the rubric weights the working demo because shipping is the skill, and a rigorous failure analysis of a broken app is still worth more than a polished demo with no numbers. A 55 with a traced failure beats a 50 with a fragile “perfect” run, and the room should hear that comparison made explicitly. If the app failed *during* the demo on the unrehearsed input, that is a live failure analysis — score it under failure analysis, not as a demo penalty, provided the pair diagnoses it on the spot.

Demo-time arithmetic

Three minutes per pair plus thirty seconds of changeover. Ten pairs is 35 minutes — the slot. Twelve pairs is 42. Above twelve, announce lightning demos (60–90 seconds: problem, failure, fix) *before* Part A starts so pairs prepare for that format; above twenty, split into two rooms or select demos by volunteers plus a random draw. Write the hard stop on the board and hold it.

Running the red-team swap

At the start of Part C, pairs physically swap laptops with the pair beside them — not across the room, to save time. Each attacks the other’s capstone for ten minutes: a direct injection in a question, an indirect one by adding a poisoned line to a document and re-ingesting, and a tool that fires when it should not. The attacker writes one sentence describing the hole; then laptops swap back and each pair patches its own. Nobody sees their own blind spot; that is why the swap is mandatory, and it is what the certificate’s “red-teamed by a classmate” line refers to. A pair with no working app red-teams the naive bot from Cell 2 and hardens that; the checkpoint is the hole plus the fix, not the app.

Part 6 — Reading list for the ten days

One or two an evening; abstract, figures, results table, limitations, in that order, and stop when you have what you came for.

1. **Vaswani et al. 2017, “Attention Is All You Need.”** The architecture. Read §3 for the Q/K/V equations and the $\sqrt{d_k}$ justification; skip the translation results.
2. **Hoffmann et al. 2022, the Chinchilla paper.** Compute-optimal scaling; the 20-tokens-per-parameter figure and why “bigger” stopped being the plan. Figure 1 and Table 3 are enough.
3. **Kaplan et al. 2020, “Scaling Laws for Neural Language Models.”** The power-law curves the Chinchilla paper corrected; read for the shape of the argument.
4. **Ouyang et al. 2022, InstructGPT.** SFT → reward model → PPO with a KL penalty; the “finishing school” slide is a summary of Figure 2.
5. **Rafailov et al. 2023, “Direct Preference Optimization.”** Why the reward model can be eliminated; read the derivation of the loss (§4) once so you can say “same objective, closed form.”
6. **Liu et al. 2023, “Lost in the Middle.”** The U-shaped accuracy curve over position in context; the empirical basis for the third tax on pasting and for context growth in agents.
7. **Lewis et al. 2020, “Retrieval-Augmented Generation.”** The name and the original architecture; note that the course’s RAG is the frozen-retriever, prompt-stuffing variant the industry converged on, not the jointly trained original.
8. **Karpukhin et al. 2020, “Dense Passage Retrieval.”** The bi-encoder; the reason reranking exists.
9. **Yao et al. 2022, “ReAct.”** Reason-then-act interleaving; the ancestor of every agent loop, and a clear picture of why trajectories need evaluating.
10. **Schick et al. 2023, “Toolformer,”** plus the Gemini function-calling docs page. Tool use as a trained behaviour; the docs page for the current wire format and the `function_response` shape you will demo.
11. **Greshake et al. 2023, “Not what you’ve signed up for.”** Indirect prompt injection formalised; the taxonomy behind the poisoned-document slide.
12. **Simon Willison, “The lethal trifecta” (2025).** Short blog post; the three-legged model the S6 depth panel uses. Read his prompt-injection series index if you have an hour.
13. **OWASP Top 10 for LLM Applications, 2025 edition.** Confirm LLM01 is still prompt injection; skim the other nine so you can name them if asked.
14. **Zheng et al. 2023, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena.”** Position, verbosity and self-enhancement bias measured; §3 is the part you will quote.
15. **Anthropic, “Building effective agents” (December 2024).** The workflow-vs-agent distinction in the industry’s own words; the five workflow patterns are a useful vocabulary for Lab 5 Part D.
16. **Gemini docs: the “thinking” page and the “embeddings” page.** Read both properly. The thinking page is where `thinking_level` vs `thinking_budget` and per-model support live; the embeddings page is where the aggregated-embedding behaviour, `output_dimensionality` and task types live. These two pages are the September-2026 truth for the labs and they move; read them again on Thursday 17.

Optional, if an evening is free: Dosovitskiy et al. 2020 (ViT) for the patch-as-token claim; Ho et al. 2020 (DDPM) §3 for the noise-prediction objective; Schaeffer et al. 2023 (“Are emergent abilities a mirage?”) for the honest version of emergence; Brown et al. 2020 (GPT-3) §3 for in-context learning as originally reported.

If you can tell the Part 1 chain in three minutes with no slides, derive every Part 2 formula on a whiteboard, and answer any ten questions from Part 3 cold, you are ready. The prep packs carry the slides; this carries you.

Session 1 — Instructor Prep Pack

How Machines Learned to Talk · 29 slides · 2 hours (115.5 min budgeted, ≈4 min reserve)

What this file is. The level under every slide, so you are never caught one question deep in front of a room of final-year CSE students and their faculty. Read it once end to end (about 40 minutes), then re-skim the slide table the night before. The run sheet (`session-1-notes.md`) is the delivery-day document; this one is for the days before.

Slide numbers here are the deck's own numbers — generated from `presentations/session-1-how-machines-learned-to-talk.html` , so slide 14 below is slide 14 on screen. Press **S** in the deck for presenter mode; the timings in this file are the deck's own budgets.

1 · The spine — the session as one argument

Seven beats. If you lose the thread mid-session, come back to this list and pick up the next one.

1. **There are two kinds of AI, and this course is about the second** (slides 4–5) — judges label what exists, creators make new content.
2. **Creation is next-token prediction, repeated** (6–9) — one idea, said once, then demonstrated three different ways until nobody in the room can un-see it.
3. **The odds come from learned weights, not a lookup** (10–13) — this is the myth-killing block, and it is where the room's model of “AI” actually changes.
4. **The machinery of one pass** (14–15) — tokens, then the whole loop in one stepper. Embeddings and attention are no longer slides of their own: the pre-class page teaches both with a real model, and slide 15 carries a two-paragraph depth panel for a room that has not read it.
5. **Three dials of intelligence** (17–20) — scale, finishing school, thinking time.
6. **The consequences are predictable** (22–23) — every famous failure falls out of beats 2–4.
7. **You can drive it today** (25–27) — API anatomy, then the lab.

Readiness test. Tell this story out loud, no slides, in three minutes. Record it on your phone and listen back. If beat 3 lands flat, the whole session sags — that is the beat that changes minds, and it is the one most often rushed.

2 · Slide by slide

Each entry: **on screen** (what they see) · **the move** (how to deliver it) · **one level under** (so you can improvise) · **hardest question** · **landmine** (what not to say).

Slides 1–3 · Opening, credibility, the promise · 3.5 min total

On screen. Title; “I sat in these seats — TCE CSE, class of 2009”; the six artifacts they will build.

The move. Open with energy: *“By the end of today, everyone here writes code that talks to a frontier AI model — for free.”* Keep the bio under 45 seconds — one line of credibility, one line of humility. The failed-things line gets a laugh; let it land, then move. Walk the six artifacts fast and plant the capstone seed: *“Session 6 you demo — start thinking about what you’d build.”*

One level under. The bio exists to buy you permission to be believed, not to impress. The six-artifact slide exists so that a student who is bored in slide 12 can see a reason to still be there at slide 27.

Hardest question. *“Will this get me a job?”* — Do not promise. Say: this course gets you something to show and the vocabulary to defend it, which is what an interview actually tests. Point at `PLACEMENT-ROADMAP.md`.

Landmine. Do not oversell yourself and do not stack credentials — the faculty in the room calibrate you on this slide, and the work sells you better than the bio does.

Slide 4 · Two kinds of AI: judges vs creators · 3 min · `trim`

On screen. A classify game — the room shouts *judges* or *creates* before each reveal.

The move. Run it as a game, about 25 seconds per item. The gotcha is keyboard autocomplete — it is generative, and most rooms call it a judge.

One level under. Discriminative models learn $P(\text{label} \mid \text{input})$; generative models learn $P(\text{next} \mid \text{all previous})$. That is a boundary in feature space versus a distribution you can sample from. Same maths family, different training target, genuinely different capability.

Hardest question. *“Google Maps predicts arrival time — isn’t that prediction generation?”* — It predicts a property of existing reality; it cannot produce a novel artifact. The line is *“labels or quantifies what exists”* versus *“constructs something new.”*

Landmine. Do not let discriminative AI sound obsolete. It still runs fraud detection, ranking and face unlock. It ran the world from 2012 to 2022 and it has not retired.

Slide 5 · AI, ML, GenAI, LLM — who lives inside whom · 2.5 min

On screen. Nested rings, clicked outside-in.

The move. Land one line: **app $\neq$ model**. ChatGPT is WhatsApp; GPT is the network.

One level under. A model artifact is literally tensors of floating-point weights plus an architecture config. “Deep” means many stacked layers. Not every LLM is generative in the loose sense, but every modern frontier LLM is.

Hardest question. *“Is Gemini one model?”* — No: a family, Flash and Pro, plus versions. Mechanics identical, size and cost different.

Landmine. Never state parameter counts for GPT-5.6, Gemini 3.5 or Claude — not public. Say “estimated hundreds of billions to trillions” and move; the labs do not disclose, and mixture-of-experts makes the count ambiguous anyway.

Slide 6 · ChatGPT is autocomplete at scale · 1.5 min

On screen. The one-sentence thesis of the whole course.

The move. This is **the** slide. Say it slowly: one job — predict the next token, append, repeat. Pre-empt *“but it reasons!”* by promising you will come back to it at scale (slide 17).

One level under. Every LLM does exactly one thing: given text, produce a distribution over the next token. Everything else — chat, code, agents — is that loop plus scaffolding.

Hardest question. *“So it’s just a fancy T9?”* — Same objective, incomparable machinery, and the difference is what slides 14–15 (and the pre-class page) are for. Do not concede more than that here.

Landmine. Do not say “just” autocomplete in a dismissive tone. You are making an architectural claim, not a capability ceiling — and slide 16 picks that fight deliberately later.

Slides 7–9 · Play the model; count the words; watch the network · 9 min

On screen. A guess-the-next-word game; the bigram tally table; the animated 21-neuron network.

The move. Slide 7 is a game — the room shouts before the reveal. The *idli* + ____ example is chosen because the room disagrees, which is the distribution made audible. The finale, *Thiagarajar College of* ____ at 97%, puts them inside a distribution they can feel. Slide 8 is the demystifier: click **Build the table** and narrate that training is tallying which word follows which, then **Sample** and let the room shout a word while you roll; **Take the top** contrasts greedy with sampling. Slide 9 is the money slide — play it once in silence, then reset and step through, narrating tokens → numbers → layers mix → distribution → dice → **append**.

One level under. Slide 8 is a genuine bigram Markov model — an n-gram with $n = 2$. It is the honest ancestor of an LLM and it fails for a specific reason worth naming: a bigram conditions on one token, so it cannot hold a subject across a clause. Real LLMs learn a compressed *function* over the whole context instead of storing a table. Historical anchor: Markov, 1906 → n-gram phrase models → transformers, 2017.

Hardest question. “*So an LLM is just a big lookup table?*” — No. That table would be astronomically large and could only ever replay what it has seen; the tally has a function that generalises to sequences it has never seen. Same interface, radically different engine.

Landmine. Do not oversell the toy — it knows adjacency, not meaning. Say so, because that gap is exactly the bridge from “you guessed the next word” (7) to “a network computes the guess” (9).

Slide 10 · Why the same question gives different answers · 3 min · [D]

On screen. A temperature slider with a roll tally.

The move. $T \rightarrow 0.05$, roll ten times (Chennai ten times). $T \rightarrow 2$, roll until *pizza* appears in the tally. Rule: facts low, creative high.

One level under. The last layer emits a raw score per vocabulary token — a **logit** — not a probability. Softmax turns logits into a distribution; temperature divides the logits *before* softmax, so low T sharpens and high T flattens. $T = 0$ is argmax.

Depth panel [D]. *Where the dice actually come from* — logits → softmax → temperature, plus top-k / top-p. Open it the moment a student asks where the probability comes from.

Hardest question. “*Why not always use $T = 0$?*” — For factual extraction, do. For anything where you want variety, $T = 0$ makes the model repetitive and prone to loops.

Landmine. Do not call temperature “creativity”. It is a sampling parameter; the model is not more imaginative at $T = 2$, only less discriminating.

Slide 11 · Wait — is it just searching a giant database? · 2.5 min

On screen. A search box returning zero results, then the model producing a pirate poem about jigarthanda.

The move. This kills myth number one. “*Nothing was stored. It computed that.*”

One level under. The weights are frozen at inference. There is no retrieval step, no index, no row. The poem is a path through a learned function, and it is the first time the room feels the difference.

Hardest question. “*Then how does it know facts?*” — Facts are compressed into the weights during training, lossily. That is also the reason it can be confidently wrong, which you will cash in on slide 22.

Landmine. Do not say “it doesn’t store anything” without qualification — it stores a great deal, just not as retrievable text. The precise claim is: no lookup at inference time.

Slides 12–13 · Just knobs; training vs using · 5.5 min

On screen. A fit-the-line widget with two sliders; then the cookbook rule and a true/false quiz.

The move. Invite a student to drag both sliders to *trained*. Narrate: w and b are **parameters**; a model is a file of knob values. Then the cookbook analogy — you write the recipe once, and cooking from it does not change the book.

One level under. Training adjusts parameters by gradient descent against a loss; inference runs the frozen function. The quiz item about the free tier training future models is the privacy warning, and it is the one students repeat to their friends afterwards.

Depth panel [D] (slide 12). *How anyone knows training is working* — cross-entropy loss and perplexity, the one number a lab watches for months.

Hardest question. “If it’s frozen, how did ChatGPT learn my name from earlier in the chat?” — It did not learn. The app re-sends the conversation every turn. This is the seed for slide 23.

Landmine. Do not imply weights update while chatting. That misconception, left alive, breaks Session 4 entirely.

Slide 14 · Models don’t read words. They read tokens. · 2.5 min

On screen. Tokenizer with a ₹ meter.

The move. Type a student’s name, then தமிழ் — the ratio jump is the moment, and the ₹ meter turns it into money: same meaning, bigger bill. Do not promise a number for the real gap: say “the real gap depends on the tokenizer — you’ll measure Gemini’s with `count_tokens` in the lab.” Gemini is comparatively efficient on Tamil, so the lab may show a narrower ratio than the toy.

One level under. Tokens are sub-word pieces from a learned BPE vocabulary; Indic scripts fragment harder because the vocabulary was fitted mostly to English, which is why the bill moves.

Hardest question. “Why doesn’t it just use whole words?” — Vocabulary explosion and no way to handle a word never seen. Sub-words trade sequence length for coverage.

Landmine. The on-slide tokenizer is a rule-based toy. Say so plainly — the lab shows real `count_tokens`. “A faithful cartoon: simplified to be visible, never simplified to be wrong.”

Slide 15 · You saw this machine on the pre-class page — here it is in one loop · 3 min · [D]

On screen. The pipeline stepper: text → tokens → embeddings → attention → probabilities → sample, driven by hand or auto-run.

The move. Open with “who did the pre-class page?” — hands up calibrates how fast you go. Then the hardest three minutes of the session: step slowly, narrate each stage (tokens → coordinates → attention → scores → dice → **append**), then auto-run and let the stat flash land: a 500-word answer is roughly 650 loops. The two middle stages are the ones the page taught with a real model. Do not re-teach them here; name them and move.

One level under. Embeddings are rows of a learned table — direction carries meaning, not position; idli sits next to dosa, and *king* – *man* + *woman* ≈ *queen* because meaning is now geometry. Attention computes, for every token, a weighted blend of every other token, and the weights are learned — it is what resolves “it” in “the trophy didn’t fit in the suitcase because it was too big”, and it is the T in GPT. Attention is order-blind, so position is added into each embedding first (positional encoding, today usually RoPE); and it costs n^2 , which is why Session 4 retrieves three paragraphs instead of pasting the textbook.

Depth panel [D]. *Embeddings and attention, in two paragraphs* — exactly the paragraph above, on screen, plus a pointer to the page’s chapters and to Session 4. Open it if a room that skipped the page asks “wait, what is attention?” — budget two extra minutes if you do, and

take them from slides 21–22.

Hardest question. “*Why does it need attention if the embeddings already carry meaning?*” — A word’s embedding is fixed; its meaning in a sentence is not. Attention is how *bank* becomes river-bank or money-bank from context.

Landmine. The stepper’s attention weights and coordinates are illustrative numbers. Say so. The real ones are on the pre-class page and, for embeddings, in Session 4’s lab with real cosine scores.

Slide 16 · If autocomplete can pass your exam... · 1.5 min · trim

On screen. The ink slide. The fight, picked deliberately.

The move. Read it once, slowly, then be quiet for three seconds. Say it looking at the faculty row, and smile. Invite disagreement and take exactly one counter-argument now; park the rest for the close.

One level under. The claim is about assessment design, not about student ability. Keep it there and the room stays with you; let it drift into “exams are worthless” and you lose the faculty.

Landmine. Do not win this argument. You are opening it on purpose; slide 29 is where you name the best counter-argument the room gave you.

Slides 17–20 · Scale, base models, finishing school, reasoning · 8.5 min

On screen. A 10M → 1T parameter slider; raw base model versus after finishing school; the three-step diagram; the reasoning dial.

The move. Drag the slider and pause at 10B: “*translation appeared — nobody programmed it.*” Then click **Raw base** on “2+2” for the laugh, and **After finishing school** for the contrast. Three finishing steps, one line each, landing on: ChatGPT’s 2022 win was better finishing school, not a smarter brain. On slide 20, use the dial metaphor — scale, finishing school, thinking time — and land the cost rule: never pay thinking prices for capital-city questions.

One level under. Emergence is real but contested; researchers disagree about how sudden it is, and some apparent jumps are artifacts of the metric. Say “researchers argue about how sharp this is” and you are both honest and safe. Reasoning models are not different magic — they are trained to spend more tokens before answering.

Depth panel [D] (slide 17). *Scale is not one slider — it is three:* parameters, data and compute have to move together. This is the Chinchilla correction.

Hardest question. “*Is the o-series a different architecture?*” — No. More tokens spent thinking, trained to do it well. Same loop.

Landmine. Do not present emergence as magic or as settled. Both readings are wrong and one of them will be challenged.

Slide 21 · Why ChatGPT ≠ Gemini ≠ Claude · 1.5 min · trim · [D]

On screen. The kitchens-and-landscape map, labelled “names as of mid-2026 — they change monthly”.

The move. Use the biryani line. Two points only — closed versus open weights, and *names change, mechanics don’t*. Do not read the table aloud.

Depth panel [D] . *Why every lab ships a cheap fast one and an expensive slow one* — mixture-of-experts, distillation, quantization. Flash, Haiku and mini are the same idea three times.

Landmine. This slide ages fastest in the whole deck. Check [instructor-notes/fact-check.md](#) before you teach; the volatile rows were last live-verified 2026-08-03.

Slides 22–24 · Famous failures; the context window; eight words · 6.5 min

On screen. Reveal cards for four failures; a window filling green then overflowing red; the eight-word recall list.

The move. On slide 22 the class explains *why* before each click — strawberry is tokens, arithmetic is prediction, stale news is the cutoff, the fake citation is plausibility. On 23, fill the window with chat and notes, then drop a textbook in and watch it go red; switch to 1M and the textbook fits — “*the wall moves; the physics doesn’t.*” Then the twist: no memory, the app re-sends everything, which is what motivates RAG. Slide 24 is active recall — the room says each definition aloud **before** you click.

One level under. All four failures fall out of beats 2–4, which is why this slide works: they are not four bugs, they are one mechanism seen four times.

Depth panel [D] (slide 23). *Then why isn’t re-sending everything unbearably slow?* — the KV cache, prefill versus decode. This is the single most useful piece of plumbing an engineer can know, and it pays off again in Session 6.

Hardest question. “*Why can’t it count letters if it can write essays?*” — It never sees letters. It sees token ids. `strawberry` may be three tokens, and nothing in the input distinguishes the r’s.

Landmine. Do not say hallucination is a bug being fixed. It is the same machinery that produces the right answers — Session 2 is built on that.

Slides 25–26 · API anatomy; the lab kit · 4.5 min

On screen. A dot travelling code → key → GPUs → response; then the A→E lab strip.

The move. Press **Send** and narrate the journey. Land: “*the model does not run on your laptop.*” Then show the lab kit once — it applies to all six labs. Point at intrepidkarthi.com/genai and make everyone bookmark it now, phones out. State the pairs rule once: *both partners have a key, one drives per part, swap at each checkpoint* — it settles every “who types?” argument for the weekend. Read the Lab 3 line aloud: put a receipt and a page of your own handwriting in Drive now. Say plainly that checkpoints are for them, not for marks.

Landmine. Do not skip the “not on your laptop” line. It pre-empts an hour of setup anxiety in the lab.

Slide 27 · Lab 1: your first AI API call · 50 min

On screen. The five steps and `aistudio.google.com`.

The move. Two-minute brief, then fifty minutes of lab. Five steps on the board. Pairs — both have a key, one drives per part, swap at each checkpoint; five-minute rule before asking a neighbour. Circulate; do a checkpoint sweep at +40. The links are on screen — point at them rather than reading them.

Lab shape (`labs/session-1/lab-handout.md`): A confirm access and get a key (10) · B first call (10) · C the five prompts (10) · D one prompt, three models (10) · E build your test set (10) = 50. Part E is not optional — Session 2 consumes it.

Landmine. Do not let Part E get squeezed. If you are behind, cut Part D, never Part E.

Slides 28–29 · Three things; the close · 7 min · trim

On screen. No homework — the hand-off to Session 2.

The move. Slide 28 is budgeted five minutes and show and tell comes first: two or three pairs’ model differences from Part D, about three minutes. Then the three cards — confirm every pair actually wrote the ten-question test set in Part E, tease the lie-detector lab, hand off. Slide 29 is a two-minute close: name the strongest counter-argument the room made to slide 16 — out loud, with credit — remind them to bring their own documents for Day 2 and the two photos for Lab 3, and close: “*You’ve learned the trick; next you drive it.*” There is no homework; Session 2 starts after a short stretch.

3 · The depth layer in this deck

Six `<|deeper|>` panels, collapsed by default so they cost the clock nothing. Press **D** on a slide to open every panel on it.

SLIDE	PANEL	OPEN IT WHEN
10	Where the dice actually come from — logits, softmax, temperature, top-k/top-p	someone asks where the probability comes from
12	How anyone knows training is working — cross-entropy loss and perplexity	someone asks how a lab measures progress
15	Embeddings and attention, in two paragraphs — meaning as coordinates, what “it” points to, positional encoding, n^2	the room skipped the pre-class page and someone asks what attention is
17	Scale is three sliders, not one — the Chinchilla correction	someone asks why not just add parameters
21	Why every lab ships a cheap one and an expensive one — MoE, distillation, quantization	someone asks what Flash actually is
23	Why re-sending everything isn’t slow — the KV cache, prefill vs decode	someone asks about latency or cost

Open one because a student earned it, never because it is there. The main line is the promise; this is the ceiling.

4 · Q&A bank — the ones that actually get asked

- “Is it conscious / does it understand?” — Not a question this course can settle, and say so. What you can say: it has no persistent state between calls, no goals, and no model of itself. Then redirect to something testable.
- “Will AI take my job?” — The honest answer is that it changes what juniors are hired for. The person who can specify, test and debug an AI system is more employable, not less. That is the whole design of this course.
- “Which model should I use?” — For this course, the one pinned in the notebooks. In general: start cheap and fast, escalate only when an eval says you must. That ordering is Session 2’s material.
- “Can I run this offline?” — Yes, with open-weights models via Ollama, and you will see it in Session 5. Quality per unit of hardware is the trade.

“Is the free tier really free?” — Yes for this course's volume, with rate limits. No card required on the designed path. Availability and quotas change — check the live account.

“Why Python?” — Because the SDKs and the ecosystem are there. The ideas are language-independent.

5 · Misconceptions, and the sentence that kills each

THEY BELIEVE	SAY THIS
It searches the internet	Nothing is looked up. The weights are frozen; the answer is computed (slide 11)
It learns from our chat	The weights never change while you use it. The app re-sends the history (13, 23)
It understands like a person	It predicts tokens. Whether that adds up to understanding is not a question we settle today
Hallucination is a bug	It is the same machinery as the correct answers. Session 2 is about measuring it
Bigger is always better	Three dials, not one — and cost is a real constraint (17, 20)
The tokenizer on screen is real	It is a labelled toy. The lab shows real token counts

6 · Timing pressure map

Slide budgets total **115.5 minutes** in a 120-minute block: hook + talk (slides 1–26) **58.5 min**, lab (slide 27) **50 min**, wrap (slides 28–29) **7 min** including show and tell. That leaves about four minutes of reserve — enough to absorb one good question, not a late start. The talk was cut from 68 to 58.5 minutes by removing the standalone embeddings and attention slides (the pre-class page teaches both with a real model; slide 15’s depth panel is the fallback) and trimming half a minute each from slides 8, 12, 14, 22, 23 and 24. Where to take further time from, in order:

1. **Slide 22** famous failures — four cards can become two.
2. **Slide 5** terminology rings — can be 90 seconds.
3. **Slide 21** kitchens — do not read the table; 60 seconds is enough.
4. **Slide 24** eight words — say four, click four.
5. **Lab Part D** one prompt three models — the first thing to cut inside the lab hour.

Never compress: slides 8–9 (the demystifier and the network), slide 15 (the stepper), lab Part E (the test set Session 2 needs), or the show and tell on slide 28.

The deck flags slides 2, 4, 8, 16, 17, 20, 21, 28 and 29 **compressible** — press **S** and you will see `compressible` on them. That is the deck's own suggestion; the ordered list above is the considered one. Checkpoints on the clock: slide 16 (the hot take) should start by **0:36**; slide 25 (API anatomy) by **0:54**; laptops open by **0:58**. If you are past slide 16 at more than 40 minutes, cut from that list rather than rushing the stepper. If the lab starts late, compress it to Parts A–C and run D as the opening of Session 2 — never drop Part E.

7 · The page students may arrive having already read

`how-llms-work.html` — “*We taught a machine to write Thirukkural*” — is a complete, interactive walk through how a language model works, on one worked example: a small GPT trained from scratch on the 1,330 couplets of Thirukkural (and, for comparison, on Shakespeare's sonnets), running live in the browser with its real weights. Twelve chapters, about thirty-five minutes: next-character prediction, the training data, tokenization and BPE, embeddings, attention, the transformer block and where the parameters live, logits → softmax → sampling with temperature, the pretraining loop with the recorded loss curve and samples at each step, supervised fine-tuning, RLHF (a preference game, a stand-in reward, and a measured reinforcement loop), the whole machine end to end, and the ~120 lines of PyTorch to build one for any text. It overlaps Session 1 on purpose: the page is the full pipeline on one example, the session is behaviour-first with a hosted model. A room that has read it arrives with the vocabulary; use it. The prose companion is `LEARNING-GUIDE.md` **Part 9**.

What overlaps your deck. Tokens (slide 14), the sampling stepper (15), temperature (10) and base-vs-finished (18) all appear on the page with the *kural* model and real numbers. Embeddings and attention are taught **only** on the page now (plus Session 4 for embeddings): the deck has no standalone slides for them, just the two-paragraph depth panel on slide 15. Ask the room “*who has seen the attention map on the page?*” at slide 15 to calibrate — a room that has not read it gets the panel opened, and you pay for it from slides 21–22.

Vocabulary. The page and the deck use the same words: token, embedding, attention, logits, softmax, temperature, pretraining, fine-tuning, RLHF. If a student says “the reward rule” they mean the page's stand-in reward model; the real thing is a second network trained on human preferences, which the page also says.

The question it will send you. “*If the kural model produces nonsense with 164k parameters, at what size does meaning appear?*” Honest answer: there is no threshold; coherence grows smoothly with data and parameters, and *facts* appear when the training text contains them. The page's chapter 11 table is the reference.

Its prose companion is Learning Guide **Part 9**.

8 · Rehearsal protocol — about two hours, ideally two evenings before

Evening one (45 min). Read this file end to end. Then tell the seven-beat spine out loud with no slides, recorded. Listen back for beat 3.

Evening two (60 min). Drive every interactive element yourself, in order: the classify game (4), the next-word game (7), build-the-table and sample (8), the network animation (9), the temperature slider (10), the fit-the-line widget (12), the tokenizer with a Tamil word (14), the stepper on both step and auto-run (15), the scale slider (17), base versus finished (18), the failure cards (22), the context window fill and overflow (23), and the API send animation (25). Open all six depth panels once so nothing surprises you — including the new one on slide 15, which you may need to read aloud.

The morning of (15 min). Technical smoke test — run the first cell of `labs/session-1/session_1_lab.ipynb` on a real key; confirm the model id in the notebook still resolves; check the deck opens offline; confirm the projector renders the deck's dot-grid without banding.

Delivery standard. Nothing on screen is defended as real when it is illustrative. The one-line shield, if challenged: *“Everything on screen is a faithful cartoon — simplified to be visible, never simplified to be wrong.”*

Session 1 — Run Sheet

How Machines Learned to Talk · 29 slides · 1:56 total (≈4 min reserve)

Delivery-day document. The concepts are in `session-1-prep.md` ; this is the clock. Highest-stakes session of the six: it sets whether the room believes you. Beats 8–9 and 15 are the ones that must not be rushed. Embeddings and attention no longer have slides of their own — they are on the pre-class page and behind **D** on slide 15.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET	
0:00	1	How machines learned to talk	1 min	
0:01	2	I sat in these seats — TCE CSE, class of 2009.	1 min	trim
0:02	3	12 hours. 6 things you'll build.	1.5 min	
0:04	4	Two kinds of AI: judges vs creators	3 min	trim
0:06	5	AI, ML, GenAI, LLM — who lives inside whom	2.5 min	
0:09	6	ChatGPT is autocomplete at scale. That one idea goes surprisingly far.	1.5 min	
0:10	7	Play the model: guess the next word	3.5 min	
0:14	8	Those odds aren't magic — you just count words.	2.5 min	trim
0:16	9	Watch the network think	3 min	
0:20	10	Why the same question gives different answers	3 min	D
0:22	11	Wait — is it just searching a giant database?	2.5 min	
0:25	12	What's inside a model? Just knobs.	3 min	D
0:28	13	Training vs using: the cookbook rule	2.5 min	

AT	#	SLIDE	BUDGET	
0:30	14	Models don't read words. They read tokens.	2.5 min	
0:33	15	You saw this machine on the pre-class page — here it is in one loop	3 min	D
0:36	16	If autocomplete can pass your exam, your exam was never testing understanding.	1.5 min	trim
0:38	17	The loop is old. The scale is new.	2 min	trim D
0:40	18	A freshly trained model won't answer you. It just keeps writing.	2.5 min	
0:42	19	Finishing school, in three steps	2 min	
0:44	20	Reasoning models: think longer, not just train bigger	2 min	trim
0:46	21	Why ChatGPT ≠ Gemini ≠ Claude	1.5 min	trim D
0:48	22	Famous failures — you explain them	2.5 min	
0:50	23	The context window: its entire working memory	2 min	D
0:52	24	Eight words you own now	2 min	
0:54	25	What actually happens when you call an API	2.5 min	
0:56	26	Your lab kit, and the rhythm	2 min	
0:58	27	Lab 1: your first AI API call	50 min	
1:48	28	Three things before the next session	5 min	trim
1:54	29	You've learned the trick. Next: you learn to drive it.	2 min	trim
1:56		end		

trim = the deck flags this slide **compressible** (press **S** and you will see **compressible** on it) — these are the first things to shorten when you are behind, not beats you must land. **D** = a **<|deeper|>** panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

0:00 · 1 · How machines learned to talk — Open with energy. “By the end of today, everyone here writes code that talks to a frontier AI model — for free.”

0:01 · 2 · I sat in these seats — TCE CSE, class of 2009. — Under 45 seconds. One line of credibility, one line of humility. The failed-things line gets the laugh — let it land, then move on. The work sells you better than the bio.

0:02 · 3 · 12 hours. 6 things you’ll build. — Walk the 6 artifacts fast. Plant the capstone seed: “Session 6 you demo — start thinking what you’d build.”

0:04 · 4 · Two kinds of AI: judges vs creators — GAME. Class shouts judges/creates before each click. Gotcha = keyboard autocomplete (generative). ~25s/item.

0:06 · 5 · AI, ML, GenAI, LLM — who lives inside whom — Click rings outside-in AI → LLM. Land app ≠ model: ChatGPT is WhatsApp, GPT is the network.

0:09 · 6 · ChatGPT is autocomplete at scale. That one idea goes surprisingly far. — THE slide. Say slowly: one job — predict the next token, repeat. Pre-empt “but it reasons!” → hold for scale.

0:10 · 7 · Play the model: guess the next word — GAME. Class shouts the next word BEFORE reveal. Ex2 (idli+___) room disagrees = spread distribution. Finale: Thiagarajar College of ___ → 97% — the room is sitting inside a distribution. Bridge: who picks? dice.

0:14 · 8 · Those odds aren’t magic — you just count words. — THE demystifier. Click Build the table — narrate: training is just tallying which word follows which. Then Sample from <|start|>: room shouts a word, you roll. Hit Take the top to contrast greedy vs sampling. Land it: the SAME three steps as Claude — only step 2 and training differ. It stops when it samples <|end|> — that is the EOS token. Compress by skipping the greedy toggle.

0:16 · 9 · Watch the network think — THE money slide. Play it once in silence — let the room watch. Then Reset and Step through, narrating: tokens → numbers, layers mix, distribution, dice, APPEND. Land the punch: it never plans a sentence. It only ever picks one next token — 650 times for a 500-word answer.

0:20 · 10 · Why the same question gives different answers — SLIDER. $T \rightarrow 0.05$, Roll 10× (Chennai ×10). $T \rightarrow 2$, Roll 10× until pizza shows in the tally. Rule: facts low, creative high. Answers “why different answers?” [D] deeper: logits → softmax → temperature divides the logits, so $T=0$ is argmax; plus top-k / top-p. Open it if anyone asks “but where does the probability come from?”

0:22 · 11 · Wait — is it just searching a giant database? — Kills #1 myth. Search → 0 results; Ask model → pirate jigarthanda poem. “Nothing stored. It computed that.”

0:25 · 12 · What’s inside a model? Just knobs. — WIDGET. Invite a student to drag both sliders to ‘trained’. Narrate: w,b are PARAMETERS; a model = a file of knob values. [D] deeper: cross-entropy loss + perplexity — the one number a lab watches for months. Good answer to “how do they know it’s working?”

0:28 · 13 · Training vs using: the cookbook rule — Cookbook analogy: write once, cooking doesn’t change the book. Q3 (free-tier trains future) = the privacy warning.

0:30 · 14 · Models don’t read words. They read tokens. — Type a student name, then தமிழ். The ratio jump is the moment — and the ₹ meter turns it into money: same meaning, bigger bill. Honest: illustrative tokenizer; lab shows real counts.

0:33 · 15 · You saw this machine on the pre-class page — here it is in one loop — STEPPER — the hardest 3 min. Ask “who did the pre-class page?” — then Step slowly, narrating each stage: tokens → coordinates → attention → scores → dice → APPEND. Auto-run ends on the stat flash; let it land. ‘500-word answer ≈ 650 loops.’ Embeddings and attention are NOT taught on their own slides any more — they

live on the page and come back in Session 4. [D] deeper: the two-paragraph version (idli/dosa, king-man+woman, what “it” points to, the T in GPT, positional encoding, n^2). Open it only if someone asks “wait, what is attention?”

0:36 · 16 · If autocomplete can pass your exam, your exam was never testing understanding. — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Say it looking at the faculty row — and smile. Invite disagreement — take one counter-argument now, park the rest for the break.

0:38 · 17 · The loop is old. The scale is new. — Drag 10M → 1T. Pause at 10B: ‘translation appeared — nobody programmed it.’ Emergence; researchers argue how sudden it is. [D] deeper: scaling laws + the Chinchilla correction (params and data together), and the data wall. Pairs with slide 20 on test-time compute.

0:40 · 18 · A freshly trained model won’t answer you. It just keeps writing. — Click Raw base on 2+2 (laugh), then After finishing school. Point: raw pretraining ≠ assistant.

0:42 · 19 · Finishing school, in three steps — Three steps, one line each. Land: ‘ChatGPT’s 2022 win was better finishing school, not a smarter brain.’

0:44 · 20 · Reasoning models: think longer, not just train bigger — Dial metaphor. 1 scale, 2 finishing school, 3 thinking time. Land the cost rule: never pay thinking prices for capital-city questions. Pre-empts ‘is o-series different magic?’ — no, more tokens.

0:46 · 21 · Why ChatGPT ≠ Gemini ≠ Claude — Biryani line. Two points only: closed vs open weights, and ‘names change, mechanics don’t.’ Don’t read table. [D] deeper: MoE, distillation, quantization — why Flash-class models exist and why parameter count stopped predicting cost. Sets up the S6 routing decision.

0:48 · 22 · Famous failures — you explain them — REVEAL CARDS. Class explains WHY before each click: strawberry → tokens, math → predict, news → cutoff, citation → plausible.

0:50 · 23 · The context window: its entire working memory — Fill window with chat+notes (green), then textbook → red overflow. Then 1M: textbook fits — ‘the wall moves; the physics doesn’t.’ Twist: no memory — app re-sends everything. Motivates RAG. [D] deeper: the KV cache: prefill vs decode, TTFT vs tokens/sec, and why context caching is ~90% off. This is the mechanism behind three later cost claims — worth opening if time allows.

0:52 · 24 · Eight words you own now — ACTIVE RECALL. Class says each definition aloud, THEN click. Now includes Parameter + Inference.

0:54 · 25 · What actually happens when you call an API — Press Send. Dot travels code → key → GPUs → response. ‘The model does NOT run on your laptop.’ De-mystifies the lab.

0:56 · 26 · Your lab kit, and the rhythm — Shown ONCE, applies to all 6 labs. Point at A → E strip. Point at intrepidkarthi.com/genai — everyone bookmarks it now, phones out. Pairs rule: both have a key · one drives per part · swap at each checkpoint. Read the Lab 3 line aloud: put a receipt + a handwriting page in Drive now. ‘Checkpoints are for you, not marks.’

0:58 · 27 · Lab 1: your first AI API call — 2 min brief, then 50 MIN LAB. 5 steps on board + aistudio.google.com. Pairs: both have a key, one drives per part, swap at checkpoints; 5-min rule. Circulate; checkpoint sweep at +40. Links are on screen — point at them.

1:48 · 28 · Three things before the next session — After lab · 5 min incl. show & tell. Show & tell first (~3): 2–3 pairs’ model differences from Part D. Then the three cards: the 10-question test set is written in the last 10 min of lab (Part E) — confirm every pair has it, it feeds Session 2; the lie-detector teaser; the next-session hand-off.

1:54 · 29 · You’ve learned the trick. Next: you learn to drive it. — Close (2 min). ‘Learned the trick; next you drive it.’ Name the strongest hot-take counter-argument from the room, with credit. Remind: bring your own documents (PDF/notes) for Day 2, and the two photos for Lab 3. Links are on screen — point at them.

The lab hour

Slide 27, 50 minutes. Two-minute brief, then hands off. Pairs rule, said once at the brief: both partners have a key, one drives per part, swap at each checkpoint.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** (“what did you expect?”), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Slides 28–29, 7 minutes. Show and tell first (~3 min, two or three pairs’ model differences from Part D), then confirm every pair wrote the 10-question test set (lab Part E) — Session 2 consumes it. Name the room’s best counter-argument to slide 16, then the hand-off. No homework: Session 2 starts after a short stretch.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md` ; if the whole provider is down, the deck’s widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the opening of the next block.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.
- **Running long** — the prep pack’s timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Session 2 — Instructor Prep Pack

Talking to AI, and Catching Its Lies · 21 slides · 2 hours

What this file is. The level under every slide. Read it end to end once (about 30 minutes), re-skim the slide table the night before.

`session-2-notes.md` is the delivery-day run sheet.

Slide numbers are the deck's own — generated from `presentations/session-2-talking-to-ai-and-catching-its-lies.html`. Press **S** for presenter mode.

This is the session that changes the course. Session 1 taught them what the machine is. This one teaches the discipline that separates using AI from engineering it, and the artifact they build here — an eval harness — is reused in Sessions 4, 5 and 6 and is the single most employable thing in twelve hours.

1 · The spine — the session as one argument

1. **Your asks are wishes, not instructions** (4–9) — a prompt has anatomy, and each part is a dial.
2. **A polished prompt still lies beautifully** (10–12) — the turn; polish is not correctness.
3. **So you measure** (13–14) — test set + scorer + score = one honest number.
4. **The scorer can be the liar too** (15) — the second turn, and the one that earns the room's respect.
5. **One number is not a verdict** (16–17) — five questions cannot settle A versus B.
6. **The loop is the job** (18) — prompt → eval → read failures → fix one thing → re-run.

Readiness test. Say beats 2 → 3 → 4 out loud in ninety seconds. That is the argument. If a student can only take one thing from twelve hours, it is this.

2 · Slide by slide

Slides 1–3 · Opening, recap quiz, the two artifacts · 4.5 min

On screen. Title; a recap quiz on Session 1; the two artifacts — prompt playbook and eval harness.

The move. Open with: *“You can now call the model. But your asks are wishes, not instructions. Today we make them instructions — and catch the lies.”* Run the recap fast for energy; a wrong answer gets a one-line re-teach, never a lecture. Then: *“This one artifact is the difference between using AI and engineering it.”*

Landmine. Do not re-teach Session 1 here. The quiz is a temperature check, not a revision lecture; if three answers are wrong, note it and fix it inside the relevant slide later.

Slide 4 · The prompt makeover: five upgrades · 4 min

On screen. A prompt evolving v0 → v5 with a quality meter climbing from about 10% to 85%.

The move. The centrepiece of the first hour. Before each **Improve** click, ask the room “*what’s still wrong with it?*” Let them find the flaw before the slide shows it. The v5 constraint is the punchline — the constraint is what stops the model inventing an award.

One level under. The five upgrades are role, task specificity, context, format and constraint. Each one narrows the distribution the model samples from. The meter is a teaching device, not a measurement — say so if asked.

Hardest question. “*Isn’t this just writing clearly?*” — Largely yes, and that is the point: the skill is unglamorous. The part that is *not* just clear writing is the constraint, which anticipates a failure mode you have seen before.

Landmine. Do not defend the quality meter as a real score. It is illustrative; the real number arrives on slide 13.

Slide 5 · Anatomy of a prompt: six switches · 2.5 min

On screen. Toggleable prompt parts with a live token count.

The move. Toggle parts on and off and watch the token count climb. Task is mandatory; everything else is a dial. Name the trap: over-stuffing dilutes attention.

One level under. More context is not monotonically better. Attention is a weighted blend over everything present, so irrelevant context competes with relevant context for weight — and you pay for it per token.

Hardest question. “*So should I always give lots of context?*” — No. Give what is needed to disambiguate the task. Session 4 is the disciplined version of this question.

Slides 6–8 · Few-shot; step-by-step; demand a format · 7 min · **trim** at 8

On screen. Zero-shot versus few-shot on a Tanglish review; a ₹472 arithmetic problem direct versus stepped; a format-enforcing prompt where the word ONLY does the work.

The move. On slide 7, make the room compute ₹472 on paper first — give them thirty seconds — *then* reveal both outputs. Direct is confidently wrong; stepped is auditable. On slide 8, point at the word **ONLY** and say: “*apps don’t read prose — your 2 a.m. parser dies without it.*”

One level under. Few-shot works because the examples pin both format and edge cases; two to five is the usual sweet spot. Step-by-step works because intermediate tokens are computation the model can condition on — it is not “thinking harder”, it is having more room to work. Format instructions reduce variance but do not guarantee validity, which is why Session 3 introduces **response_schema**.

Hardest question. “*If step-by-step is better, why not always?*” — Tokens cost money and latency. For a lookup, it is waste.

Landmine. Do not promise that asking for JSON gives you valid JSON. It usually does; Session 3 shows the version that actually enforces it.

Slide 9 · The four classic prompt crimes · 3 min

On screen. Crime cards; the class guesses each fix before the reveal.

The move. Let them guess. The kitchen-sink crime previews Session 5 workflows — name that forward link.

Slides 10–12 · The turn: it lies beautifully · 8 min · **trim** at 12

On screen. Two real cases; a three-round spot-the-lie game with a tone meter; then the hot take.

The move. Read the lawyer case and the Air Canada case **straight**, no editorialising. The room will go quiet — let it. “*These were not bad prompts. They were unmeasured ones.*” Then the game: three rounds, the room votes A/B/C before each reveal. The lies are **C** (the award

is invented), **B** (India lost that match) and **A** (the Nobel was for the photoelectric effect, not relativity). The tone meter shows identical confidence bars every round: **tone tells you nothing**. Then slide 12, the hot take — read it once, slowly, be quiet for three seconds, take exactly one counter-argument and park the rest for the break.

One level under. *Mata v. Avianca* (2023) — lawyers sanctioned for six fabricated citations, roughly \$5,000. *Moffatt v. Air Canada* (Feb 2024) — the airline held liable for its chatbot's invented refund policy, BC Civil Resolution Tribunal. Both are real and both are checkable; the “unlike the lawyer, I checked mine” receipts line on the slide is doing deliberate work.

Hardest question. “Doesn't RAG fix this?” — It reduces it and gives you a citation to check. It does not remove the mechanism. Promise Session 4 and keep the promise.

Landmine. Do not let the room conclude the technology is untrustworthy and therefore useless. The conclusion you want is: untrusted *and therefore measured*. That distinction is the whole session.

Slides 13–14 · You measure; watch an eval run · 5.5 min

On screen. Test set + scorer + score; dots filling to “7/10 — argument over”; then a live eval, run twice.

The move. Land the line: “**A prompt without an eval is a superstition.**” Then run the eval and *immediately* run it again — the score moves 7 → 8. Ask which is true. Answer: neither. Set temperature to 0, run three times, average. Then read the failures aloud.

One level under. Three components, and students conflate them: the **test set** (inputs plus expected outputs), the **scorer** (the function that decides pass or fail) and the **score** (one number). Changing any one changes the number, which is why you report the whole configuration, not just the percentage.

Hardest question. “How many questions do I need?” — More than five, and slide 16's depth panel gives the rule. Ten is the teaching floor; a real system wants a few hundred.

Landmine. Do not skip the run-twice move. Watching the number move by itself is what makes the variance lesson land.

Slide 15 · Same answer, three verdicts · 3 min · [D]

On screen. One correct answer judged by three scorers; exact-match fails it.

The move. This is the aha. Exact-match rejects a *correct* answer. “Your scorer can be the liar too.” Contains-match is what today's lab uses.

Depth panel [D]. *One number is usually the wrong number* — precision, recall and F1, plus the AI judge's position and verbosity bias, and the fix (swap the order and re-run).

One level under. Exact match has high precision and terrible recall on free text. Contains is permissive and can pass an answer that also contains a contradiction. An LLM judge is flexible and introduces its own biases. There is no free scorer.

Hardest question. “So which scorer should I use?” — The cheapest one that does not lie about your task. For short factual answers, contains. For anything structured, parse it. For prose, a judge with the position-bias fix.

Slides 16–17 · The arena; “it worked when I tried it” · 5 min · trim at 17

On screen. Prompt A versus Prompt B, B wins 4–2 but Q5 beats both; then the hot take about evidence.

The move. Run the fight. Then point at Q5 — the question both prompts failed — and say the loop never ends, it converges, and then you grow the test set. On 17: demo is the best case, eval is the expected case.

Depth panel [D] (slide 16). *B won 4–2. Did B actually win?* — the widget renders A 2/5, B 4/5, and a two-question gap on five questions is still inside the noise; the $\pm 1/\sqrt{n}$ rule, paired comparison, and separating model from prompt from scorer. Open this whenever a student

treats a small win as settled.

One level under. With five questions the finest distinction you can resolve is coarse; a 4–2 split is well inside noise. Paired comparison — same questions, both prompts, same scorer — is what makes small n usable at all.

Landmine. Do not let “B won” stand unchallenged, even for a minute. The whole point of the slide is that it did not.

Slides 18–19 · The loop; six ideas · 5 min

On screen. The eval-driven development cycle, running on its own; then recall flips.

The move. Let the pipeline animate without narrating over it. **“Failures are the syllabus.”** Then active recall — the class says each of the six ideas before you click: anatomy, few-shot, step-by-step, hallucination, test set, the loop.

Slide 20 · Lab 2: the lie detector · 52 min

On screen. The lab brief.

The move. Two-minute brief, then fifty minutes. Press **L** for the countdown — it paces the room. Three things to enforce while circulating: expected strings must be **short**, every **X** must be diagnosed rather than deleted, and the documentation is the deliverable — say it at the brief: **three iterations documented well beats five documented badly**. They A/B on the ten questions they wrote in Session 1.

Lab shape (labs/session-2/lab-handout.md): A prompt makeover (15) · B your first eval (15) · C the arena (15).

Landmine. Students will “fix” a failing test by loosening the expected string until it passes. Name this at the brief, before it happens — that is the single most common failure of the hour, and it is exactly the behaviour the session exists to prevent.

Slide 21 · You can now prove whether AI is right · 3 min

The move. Show and tell: best improvement, best caught lie, and name the strongest counter-argument the room made to slide 12. Ask of any failure: *model, scorer, or question?* Then the teaser — next session AI gets eyes, and they should bring photos.

3 · The depth layer in this deck

SLIDE	PANEL	OPEN IT WHEN
15	One number is the wrong number — precision, recall, F1, judge bias	someone asks which scorer to use, or proposes an LLM judge
16	Did B actually win? — the $\pm 1/\sqrt{n}$ rule, paired comparison	someone treats a small eval win as settled

Two panels only; this deck spends its depth in the main line.

4 · Q&A bank

- “Can’t I just use a better model instead of better prompts?” — Sometimes. You cannot know without an eval, which is the point. And the eval outlives the model choice.
- “Isn’t an LLM judging an LLM circular?” — Partly, and it has measurable biases (position, verbosity). It is still useful when the alternative is no measurement. Fix what you can: swap the order and re-run.
- “How is this different from unit testing?” — Same instinct, different assertion. Outputs are non-deterministic, so you assert on a distribution of behaviour rather than an exact value, and you accept a threshold rather than a pass/fail.
- “Ten questions seems tiny.” — It is. It is enough to catch gross regressions and to teach the discipline. Say the honest number: real systems use hundreds, and the depth panel explains why.
- “What if my expected answer is wrong?” — Then your eval lies with total confidence, which is why you read failures instead of trusting the score.
-
-

5 · Misconceptions

THEY BELIEVE	SAY THIS
A better prompt fixes hallucination	It reduces frequency, not the mechanism (slides 10–12)
A high score means it works	It means it works <i>on that test set, with that scorer</i> (13–16)
Confident tone signals correctness	The tone meter is identical on every round, including the lies (11)
The scorer is neutral infrastructure	Exact-match fails correct answers (15)
One run is a result	Run it twice on stage and watch the number move (14)

6 · Timing pressure map

- Slide budgets total 102.5 minutes including the 52-minute lab, leaving about 18 minutes of reserve — slack that Session 1 does not have. Spend it on slides 10–16, which are the session, and on a longer show and tell.
- Cut in this order: slide 9 crime cards (four can be two) · slide 6 few-shot (the Tanglish example alone carries it) · slide 19 recall flips.
- Never compress: slide 11 (the game), 14 (run it twice), 15 (the scorer turn), or 16’s Q5 moment.

The deck flags slides 8, 12 and 17 **compressible** (press **S** to see the marker). If you reach slide 12 later than 40 minutes in, cut from the list above.

7 · Day-before checklist

Run the makeover through all five clicks. Play all three rounds of the spot-the-lie game and check you can state *why* each is false without reading. Run the eval twice and confirm the score moves. Open both depth panels. Then open `labs/session-2/session_2_lab.ipynb`, run it end to end on a real key, and confirm the model id still resolves — see `fact-check.md`, volatile rows last live-verified 2026-08-03.

Have your own ten-question test set ready as the worked example, in case a pair arrives without one.

Session 2 — Run Sheet

Talking to AI, and Catching Its Lies · 21 slides · 1:42 total

Delivery-day document. The concepts are in `session-2-prep.md` ; this is the clock. The discipline session. If you deliver only one thing well this weekend, make it slides 13–16.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET
0:00	1	Talking to AI, and catching its lies	1 min
0:01	2	Still true after the break?	2 min
0:03	3	Two artifacts in two hours	1.5 min
0:04	4	The prompt makeover: five upgrades	4 min
0:08	5	Anatomy of a prompt: six switches	2.5 min
0:11	6	Few-shot: show, don't tell	2.5 min
0:14	7	Step-by-step beats straight-to-answer	2.5 min
0:16	8	Demand a format, or parse chaos forever	2 min
0:18	9	The four classic prompt crimes	3 min
0:21	10	Your polished prompt still lies beautifully.	2 min
0:23	11	One of these is a confident lie	4.5 min
0:28	12	Hallucination is an expected failure mode. That does not make it acceptable.	1.5 min
0:29	13	"How do you know it's right?" You measure.	2 min
0:31	14	Watch an eval run	3.5 min

AT	#	SLIDE	BUDGET	
0:34	15	Same answer, three verdicts	3 min	D
0:38	16	Prompt A vs Prompt B: the arena	3.5 min	D
0:41	17	“It worked when I tried it” is not evidence.	1.5 min	trim
0:42	18	Eval-driven development: the loop	2.5 min	
0:45	19	Six ideas you own now	2.5 min	
0:48	20	Lab 2: the lie detector	52 min	
1:40	21	You can now prove whether AI is right.	3 min	
1:42		end		

trim = the deck flags this slide **compressible** (press **S** and you will see **compressible** on it) — these are the first things to shorten when you are behind, not beats you must land. **D** = a **<|deeper|>** panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

- 0:00 · 1 · Talking to AI, and catching its lies** — ‘You can now call the model. But your asks are wishes, not instructions. Today: make them instructions, and catch the lies.’
- 0:01 · 2 · Still true after the break?** — Recap quiz. Fast energy. Wrong answers = one-line re-teach, don’t lecture.
- 0:03 · 3 · Two artifacts in two hours** — Two artifacts: prompt playbook + eval harness. ‘This one artifact is the difference between using AI and engineering it.’
- 0:04 · 4 · The prompt makeover: five upgrades** — CENTERPIECE. Makeover v0 → v5. Before each Improve click ask ‘what’s still wrong?’ v5 constraint = the punchline (can’t invent awards). Quality meter climbs 10% → 85%.
- 0:08 · 5 · Anatomy of a prompt: six switches** — Toggle prompt parts live — watch the ~token count climb. Task mandatory, rest are dials; over-stuffing dilutes attention.
- 0:11 · 6 · Few-shot: show, don’t tell** — Zero vs few-shot. The Tanglish review lands well. ‘2–5 examples pin format + edge cases — the model imitates.’
- 0:14 · 7 · Step-by-step beats straight-to-answer** — Let class compute ₹472 on paper FIRST (30s), then reveal both. Direct = confident wrong; steps = auditable.
- 0:16 · 8 · Demand a format, or parse chaos forever** — The word ONLY does real work. ‘Apps don’t read prose — your 2 a.m. parser dies without it.’

0:18 · 9 · The four classic prompt crimes — Crime cards. Class guesses each fix before the click. Kitchen-sink previews S5 workflows.

0:21 · 10 · Your polished prompt still lies beautifully. — The turn. Read the lawyer + Air Canada cases straight. Room goes quiet — let it. ‘Not bad prompts. Unmeasured ones.’ Point at the receipts line: ‘Unlike the lawyer, I checked mine.’

0:23 · 11 · One of these is a confident lie — GAME. 3 rounds, vote A/B/C before each reveal. Lies: C (award invented) · B (India LOST that match) · A (Nobel was for photoelectric, not relativity). Tone-o-meter: identical bars every round. ‘Tone tells you nothing. Only checking does.’

0:28 · 12 · Hallucination is an expected failure mode. That does not make it acceptable. — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Invite disagreement — take one counter-argument now, park the rest for the break.

0:29 · 13 · “How do you know it’s right?” You measure. — Test set + scorer + score = one honest number. Dots fill to ‘7/10 — argument over.’ Then land the line: ‘A prompt without an eval is a superstition.’

0:31 · 14 · Watch an eval run — Run eval, then IMMEDIATELY Run-again — score 7 → 8. ‘Which is true? Neither. T=0, ×3, average.’ Read failures.

0:34 · 15 · Same answer, three verdicts — Exact-match FAILS a correct answer — the aha. ‘Your scorer can be the liar too.’ Contains = today’s lab scorer. [D] deeper: precision / recall / F1, and the AI judge’s position + verbosity bias (fix: swap the order and run twice). Open it if anyone proposes an LLM judge.

0:38 · 16 · Prompt A vs Prompt B: the arena — Arena. Fight. B wins 4–2 but Q5 beats both. ‘The loop never ends; it converges. Then grow the test set.’ [D] deeper: why 5 questions cannot settle A vs B: the $\pm 1/\sqrt{n}$ rule, paired comparison, and separating model variance from test-set variance.

0:41 · 17 · “It worked when I tried it” is not evidence. — ‘It worked when I tried it’ = not evidence. Demo = best case; eval = expected case.

0:42 · 18 · Eval-driven development: the loop — The loop = the job. Prompt → eval → read failures → fix ONE thing → re-run. The pipeline cycles on its own — let it. ‘Failures are the syllabus.’

0:45 · 19 · Six ideas you own now — Recall flips. Class says each before clicking. Six ideas: anatomy, few-shot, step-by-step, hallucination, test set, the loop.

0:48 · 20 · Lab 2: the lie detector — 2 min brief, then 50 MIN LAB. Press L — the 50:00 countdown paces the room. Expected strings SHORT; diagnose every *X*; documentation is the deliverable — three iterations documented well beats five documented badly. A/B on their 10 Qs. Links are on screen — point at them.

1:40 · 21 · You can now prove whether AI is right. — Show & tell + break. Best improvement + best caught lie + name the strongest hot-take counter-argument. ‘Model, scorer, or question?’ Next: AI gets eyes — have photos. Links are on screen — point at them.

The lab hour

Slide 20, 52 minutes. Two-minute brief, then hands off.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** (“what did you expect?”), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Ask of every failure: model, scorer, or question? Tell them to bring photos.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md`; if the whole provider is down, the deck's widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the opening of the next block.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.
- **Running long** — the prep pack's timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Session 3 — Instructor Prep Pack

AI Beyond Text: Eyes, Ears and a Paintbrush · 17 slides · 2 hours

What this file is. The level under every slide. `session-3-notes.md` is the delivery-day run sheet. Slide numbers are the deck's own, generated from `presentations/session-3-ai-beyond-text.html`.

This session runs after lunch and closes Day 1. Two consequences: energy management is a real part of the job, and slide 17 is not a normal close — it is the Day 1 close and it carries the overnight task that Session 4 depends on.

1 · The spine

1. **Same loop, new kinds of tokens** (3–4) — nothing from yesterday is discarded; an image becomes tokens and enters the same attention loop.
2. **It doesn't just see — it reads** (5–6) — and capability is not permission.
3. **Reading is prediction; making is un-destruction** (7–8) — diffusion reversed.
4. **Every capability comes with fine print** (9–11) — deepfakes, voice, and the fight worth picking.
5. **It is all one API call** (12) — everything from Sessions 1 and 2 transfers unchanged.
6. **It fails in the same shapes as before** (13) — counting is multiplication in pixels; blur-invention is hallucination in pixels.

The line that makes the session cohere: *“the course rhymes.”* Every failure today is a failure they already met, wearing new clothes.

2 · Slide by slide

Slides 1–3 · Opening, recap, the bridge · 6 min

On screen. Title; a recap quiz whose hallucination question sets up today; then the bridge slide.

The move. Post-lunch. If the room is flat, open with the Seoul story for forty-five seconds *before* the title. The story, in three lines — *[Karthik: replace with your own K-Startup Seoul story; this is a placeholder shape, not your biography]*: you are in a Seoul accelerator pitch room, a founder demos an app that reads a photographed menu and answers questions about it, and half the room assumes it is a separate “vision AI”; you ask to see the code and it is the same text model with an image in `contents` — one API call. The point of the story is the bridge sentence that follows it, so end on *“same machine, new kind of token.”* Then the bridge, which is the most important sentence of the first ten minutes: **“new eyes wired into the same brain.”** Everything from yesterday transfers.

Landmine. Do not let this feel like a new subject. If the room thinks vision is a separate topic, slide 12's payoff dies.

Slide 4 · How a model reads a picture · 5 min · `[D]`

On screen. An image being patchified into a grid, then into tokens; hovering a token pill highlights its patch.

The move. Click grid → tokens → “*same attention loop.*” Hover a pill and its patch together. Note aloud that this gopuram scene returns in the diffusion demo — the callback pays off on slide 8.

One level under. A vision transformer cuts the image into fixed patches, linearly projects each into a vector, adds a positional embedding and feeds the sequence to the same attention stack. A patch is a token in every sense that matters, including billing.

Depth panel [D] . *A patch is a token, so an image has a token bill* — resolution drives token count drives cost.

Hardest question. “*Does it see the whole image at once?*” — Every patch attends to every other patch, so effectively yes, but there is no retina and no fovea. Resolution is decided before the model sees anything.

Slides 5–6 · It reads; place your bets · 8.5 min

On screen. A KYC card being read; then a betting game on what the model will and will not read.

The move. The KYC card is your fintech credibility beat — most rooms have not seen a model read a form. On slide 6 the room votes on each item. The CAPTCHA item is a deliberate trick: the model *can*, and the answer is that **capability is not permission**. Say those exact words and flag them forward to Session 6.

Landmine. Do not let the CAPTCHA item become a how-to. The point is the distinction, and it is an ethics beat, not a technique.

Slides 7–8 · Reading is prediction; diffusion · 8.5 min · [D]

On screen. The framing sentence; then the diffusion slider running static → image, with a Generate button.

The move. “*Reading is prediction; making is un-destruction.*” The GPT-4o token-by-token aside is one breath — do not derail into it. Then the centrepiece: drag the slider slowly left to right and narrate that **shapes arrive before details** — silhouette around step 15, sun around 30, window around 45. Then press Generate.

One level under. Training adds known noise to a real image and asks the network to predict the noise that was added. At generation time you start from pure noise and subtract predicted noise repeatedly. That is why coarse structure resolves first: the low-frequency content is the easiest signal to recover.

Depth panel [D] . *What the network is actually trained to output* — it predicts the noise, not the picture. This is the twist that makes diffusion trainable and it is the single most satisfying idea in the session.

Landmine. The canvas fakes the pixels. The coarse-before-fine *ordering* is real and is the teaching point; say which part is honest before anyone asks.

Slides 9–11 · Fine print; speech; the fight · 9.5 min · trim at 9 and 11

On screen. Generation limits and ethics; three speech statistics; then the hot take.

The move. Deliver the camera line slowly — “*plausible, not verified — now in pixels.*” On speech, hit the three numbers first: three seconds of audio, roughly zero cost, unlimited output. Then the family-password beat, delivered straight, no jokes: **tell your parents this weekend**. Then slide 11, the hot take: read once, slowly, three seconds of silence, take one counter-argument, park the rest.

One level under. Voice cloning from a few seconds is a real, deployed capability. The defence that actually works is a shared secret out of band — a family password — because it does not depend on detecting the fake.

Landmine. Do not turn this into fear. The room should leave with one action they can take, not with anxiety.

Slide 12 · All of it is one API call · 4 min · [D]

On screen. Four lines of code; `contents=[img, question]`; a highlight finds the total and JSON lands.

The move. This is the payoff of the whole session. Run it. *“Prompting, format, evals — all apply unchanged.”* Then the new beat: `response_schema`.

Depth panel `[D]`. *Why a schema beats begging for JSON* — in the lab they do it both ways, and the prompt-only version works until it doesn't.

One level under. A schema constrains decoding so a malformed *shape* is unreachable, rather than merely discouraged. That is a different guarantee from asking politely, and it is the honest answer to Session 2's format slide. It is not a guarantee about *values* (a confidently wrong total parses fine — allow `null`) and truncation at `max_output_tokens` can still cut JSON mid-object, so the lab keeps its `try/except`.

Slides 13–15 · Where vision fails; Madurai seeds; five ideas · 11.5 min

On screen. Failure flips; local project seeds; recall flips.

The move. Run the dot flash on the counting failure and let the room estimate too — they will also be wrong, and that is the point. Name the rhyme out loud: counting is Session 1's multiplication disease in pixels; blur-invention is Session 2's hallucination in pixels. Then read two or three Madurai seeds with genuine enthusiasm — *“steal any of these for your capstone.”*

One level under. Counting fails because there is no iterative counting procedure; the model estimates from a single forward pass, exactly as it does with arithmetic. Blur-invention fails because the prior fills what the pixels do not determine.

Slide 16 · Lab 3: interrogate your photos · 53 min

The move. Three-minute brief, then fifty minutes. Demo the folder upload **once** yourself first — it is the single biggest time sink. Push Part B until `json.loads` actually passes; that is the checkpoint that matters. Collect two best inventions for show and tell.

Lab shape: A interrogation ladder (10) · B document → JSON (10) · the `response_schema` production way (5) · C handwriting (10) · D break it (10).

Slide 17 · Day 1: understand the machine. Day 2: arm it · 14 min

The move. Do not rush this. Show and tell, then recap the four artifacts built across Day 1, then name the strongest counter-argument the room gave to slide 11. Then **one** overnight task: bring two or three real documents. **Say it twice.** Session 4 does not work without them.

Landmine. If you let this slide compress, Session 4 opens with half the room having nothing to ingest. It is the highest-leverage two minutes of Day 1.

3 · The depth layer in this deck

SLIDE	PANEL	OPEN IT WHEN
4	A patch is a token, so an image has a token bill	someone asks what an image costs

SLIDE	PANEL	OPEN IT WHEN
8	What the network is actually trained to output — it predicts the noise	someone asks how diffusion is trained
12	Why a schema beats begging for JSON	someone asks why their JSON parse failed

4 · Q&A bank

- “Can it read Tamil handwriting?” — Sometimes, unreliably, and the lab’s Part C is where they find out on their own documents. Honest answer beats a promise.
- “Is the image model the same model?” — Usually a different model in the same family, sometimes the same multimodal model. The API surface is what you teach; the packaging changes.
- “Can I detect a deepfake?” — Detection is an arms race and you should not build a product on it. Provenance and out-of-band verification are the defences that hold.
- “Why is my JSON sometimes broken?” — Because you asked rather than constrained. Use `response_schema` — slide 12’s depth panel.
- “How much does an image cost?” — Resolution-dependent, because resolution sets patch count sets tokens. Slide 4’s depth panel has the arithmetic.

5 · Misconceptions

THEY BELIEVE	SAY THIS
Vision is a different kind of AI	Patches become tokens and enter the same attention loop (3–4)
Diffusion draws like a person	It removes predicted noise, repeatedly; shapes before details (8)
It can count what it can see	Counting is one forward pass, not a procedure — same disease as arithmetic (13)
Asking for JSON gives valid JSON	A schema constrains; a prompt requests (12)
It can read the image, so it may	Capability is not permission — flagged here, cashed in Session 6 (6)

6 · Timing pressure map

Budgets total 120 minutes including a 53-minute lab, so this session is **tight** — the same shape as Session 1.

Cut in this order: slide 14 Madurai seeds (read one, not three) · slide 6 betting game (three items, not all) · slide 15 recall flips.

Never compress: slide 8 (the diffusion drag), slide 12 (the payoff), or slide 17 (the Day 1 close and the overnight task).

The deck flags slides 9 and 11 **compressible** (press **S** to see the marker).

7 · Day-before checklist

Drag the diffusion slider end to end and confirm the silhouette-then-detail ordering is visible on the projector — it is subtle on a washed-out screen. Slide 12's demo is a canned run (the highlight and the JSON are scripted, not a live call) — the live version is Lab 3 Part B, so run the notebook's extraction cell on a real key with your own receipt the day before. Have two or three of your own photos ready, including one bill and one piece of handwriting, so you can demo the lab yourself if uploads fail. Confirm `response_schema` still behaves on the pinned model — `fact-check.md`, volatile rows verified 2026-08-03.

Session 3 — Run Sheet

AI Beyond Text · 17 slides · 2:00 total

Delivery-day document. The concepts are in `session-3-prep.md` ; this is the clock. Post-lunch, and it closes Day 1. Watch energy, and protect the overnight task on the last slide.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET
0:00	1	AI beyond text: eyes, ears and a paintbrush	1.5 min
0:02	2	Still true after the break?	2.5 min
0:04	3	Same loop. New kinds of tokens.	2 min
0:06	4	How a model reads a picture	5 min D
0:11	5	It doesn't just see. It reads.	3.5 min
0:14	6	Will it read it? Place your bets	5 min
0:20	7	Reading pictures is prediction. Generating them reverses noise.	2.5 min
0:22	8	Diffusion: a picture emerges from static	6 min D
0:28	9	Image generation: magic with fine print	3.5 min trim
0:32	10	Speech: useful, but not uniformly solved	4.5 min
0:36	11	Your mother's voice is no longer proof of your mother.	1.5 min trim
0:38	12	All of it is one API call	4 min D
0:42	13	Where vision quietly fails	5 min
0:46	14	Vision ideas that belong to Madurai	3.5 min

AT	#	SLIDE	BUDGET
0:50	15	Five ideas you own now	3 min
0:53	16	Lab 3: interrogate your photos	53 min
1:46	17	Day 1: you understand the machine. Day 2: you arm it.	14 min
2:00		end	

trim = the deck flags this slide **compressible** (press **S** and you will see **compressible** on it) — these are the first things to shorten when you are behind, not beats you must land. **D** = a **<|deeper|>** panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

- 0:00 · 1 · AI beyond text: eyes, ears and a paintbrush** — Post-lunch — if flat, open with the Seoul story (45s) first. 'Same machine, new kinds of tokens.'
- 0:02 · 2 · Still true after the break?** — Recap quiz. Q on hallucination-in-pixels sets up today.
- 0:04 · 3 · Same loop. New kinds of tokens.** — THE bridge: 'new eyes wired into the same brain.' Everything from yesterday transfers.
- 0:06 · 4 · How a model reads a picture** — Patchify. Click grid → tokens → 'same attention loop.' Hover a pill ↔ its patch. Note: this gopuram scene returns in the diffusion demo (callback). [D] deeper: ViT patches as tokens — resolution costs tokens quadratically, small text falls off a cliff, counting is structurally hard. Explains the failures on slide 13 in advance.
- 0:11 · 5 · It doesn't just see. It reads.** — It doesn't just see — it READS. The KYC card is your fintech credibility beat.
- 0:14 · 6 · Will it read it? Place your bets** — GAME. Vote each. CAPTCHA item is a trick question — it plants 'capability ≠ permission' — say those words (Session 6 flag).
- 0:20 · 7 · Reading pictures is prediction. Generating them reverses noise.** — 'Reading is prediction; making is un-destruction.' Set up diffusion. The 4o token-by-token aside is one breath — don't derail.
- 0:22 · 8 · Diffusion: a picture emerges from static** — CENTERPIECE. Drag the slider slowly left → right — narrate 'shapes arrive before details': silhouette ~15, sun ~30, window ~45. THEN Generate. Honest: canvas fakes the pixels, the coarse-to-fine direction is true. [D] deeper: the network predicts the noise , not the image; steps / guidance / seed; latent diffusion. Open it when someone asks what the sliders in an image tool do.
- 0:28 · 9 · Image generation: magic with fine print** — Honest limits: subtle details, style ethics, deepfakes. Land the camera line slowly. 'Plausible not verified — now in pixels.'
- 0:32 · 10 · Speech: useful, but not uniformly solved** — Hit the three stats first — 3 s, ~₹0, ∞. Family-password beat is serious — deliver straight. 'Tell your parents this weekend.'
- 0:36 · 11 · Your mother's voice is no longer proof of your mother.** — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Invite disagreement — take one counter-argument now, park the rest for the break.

0:38 · 12 · All of it is one API call — The payoff: 4 lines. contents=[img, question]. Run it — the highlight finds the total, then the JSON lands. ‘Prompting, format, evals — all apply unchanged.’ New beat: response_schema = guaranteed JSON — lab does both ways. [D] deeper: constrained decoding — a schema masks illegal tokens to $-\infty$ at the decoder, so json.loads cannot fail. Pairs with lab cell 4b.

0:42 · 13 · Where vision quietly fails — Failure flips. Counting = S1 multiplication disease in pixels — run the dot flash, the room estimates too. Blur-invention = S2 hallucination in pixels. The course rhymes.

0:46 · 14 · Vision ideas that belong to Madurai — Read 2–3 Madurai project seeds with enthusiasm. ‘Steal any for your capstone.’

0:50 · 15 · Five ideas you own now — Recall flips. Class says each before clicking.

0:53 · 16 · Lab 3: interrogate your photos — 3 min brief, then 50 MIN LAB. Demo the folder-upload ONCE first. Push Part B until json.loads passes. Collect 2 best inventions. Links are on screen — point at them.

1:46 · 17 · Day 1: you understand the machine. Day 2: you arm it. — Show & tell + DAY 1 CLOSE — don’t rush. Recap 4 artifacts built. Name the strongest hot-take counter-argument. ONE overnight task: 2–3 real documents. Say it twice. ‘Sleep. Tomorrow we build for real.’ Links are on screen — point at them.

The lab hour

Slide 16, 53 minutes. Two-minute brief, then hands off.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** (“what did you expect?”), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Do not rush the close. ONE overnight task: bring 2–3 real documents. Say it twice.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md`; if the whole provider is down, the deck’s widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the opening of the next block.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.

- **Running long** — the prep pack's timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Session 4 — Instructor Prep Pack

Giving AI Your Own Knowledge · 20 slides · 2 hours

What this file is. The level under every slide. `session-4-notes.md` is the run sheet. Slide numbers are the deck's own, from `presentations/session-4-giving-ai-your-own-knowledge.html`.

Say this out loud in the first minute: this is the most employable session of the six. RAG is what most “AI engineer” job descriptions are actually describing, and the lab builds a working one in about sixty lines.

It depends on students' own documents. They were told at the Day 1 close to bring two or three real ones. Have `labs/session-4/sample-os-notes.txt` open in a tab before the room fills — it is the instructor sample the notebook's Cell 2 falls back to, and the deck's demo queries (attendance, pass marks, the OS content) retrieve from it — because some fraction of the room will arrive empty-handed and they cannot sit out the build. **Minute 0 of the lab: the Drive-mount reminder.** Lab 4 Cell 1 mounts Google Drive and saves `chunk_vecs.npy` / `chunks.json` under `MyDrive/genai/`; say out loud that Labs 5 and 6 reload those files, so a pair that skips the mount will re-embed under a rate limit this afternoon.

1 · The spine

1. **It cannot know your notes, and it will not say so** (3) — the villain.
2. **Pasting everything is priced, and the price is real** (4) — three taxes: window, meter, middle.
3. **Send the right three paragraphs** (5) — the whole idea, in one line.
4. **Keyword search cannot find them; meaning can** (6–8) — and they already own the tool.
5. **How you cut the book decides everything** (9) — more RAG failures come from chunking than model choice.
6. **The pipeline, end to end** (11–12) — and the escape hatch is what 9 a.m. was missing.
7. **It breaks in four specific ways** (13) — with a debug order.

2 · Slide by slide

Slides 1–2 · Welcome; did it survive the night? · 3 min

The move. Day 2 opens. *“The model read the internet — but not YOUR notes. Today we fix that.”* The recap quiz's fourth question, on the absence of memory, is the deliberate setup: **“that gap is what we fix this morning.”**

Slide 3 · Watch it not know — loudly · 3 min

On screen. The villain slide: a question about the syllabus, an answer typing out, then a stamp.

The move. Ask it, let the answer type, and let the stamp land at the end. Then the line that carries the session: *“It cannot know. Notice that it didn't say that.”*

One level under. If a student's own phone hedges politely on the same question, that is not a counter-example worth conceding — it still invents a *typical* answer. Point out that hedging is a trained behaviour, not knowledge of its own ignorance.

Landmine. Do not mock the model here. The point is structural — it has no mechanism for knowing what it does not know — and mockery makes it sound like a fixable defect.

Slides 4–5 · Price the paste; send three paragraphs · 4 min

On screen. A cost slider; then the insight.

The move. Drag the slider to 380 pages — the Galvin textbook — and **read the ₹/month figure aloud**. Name the three taxes: the window (it may not fit), the meter (you pay per token every call), and the middle (attention dilutes across irrelevant text). Then the insight: send the right three paragraphs. Open-book exam with a good index.

Hardest question. *“But context windows are huge now — isn’t this obsolete?”* — This is the best question in the session and it recurs in Session 6. Cost, latency, freshness and citations all still favour retrieval. Concede the honest case: a small static corpus queried rarely is genuinely fine to paste.

Slides 6–8 · Keyword misses meaning; embeddings for paragraphs; the playground · 8.5 min ·

[D]

On screen. A keyword search failing on zero shared words; the embedding map at paragraph scale; a four-query search playground.

The move. On slide 6, pause on the zero-shared-words example and say *“you already know a machine that maps meaning to coordinates...”* — the room shouts **embeddings**. On slide 8, run all four queries and watch the query dot drop onto the map. Query 2 is teaching gold: **two** chunks are relevant, which is why you take top-k and not top-1. End on the trap query where every bar is under 0.35 — that is the “nothing relevant” case, and it is what the escape hatch on slide 12 exists for.

Depth panel [D] (slide 7). *Why that one line is allowed to be that short* — pooling, why normalisation turns a matrix multiply into cosine similarity, and query-versus-document task types.

One level under. Retrieval is `chunk_vectors @ question_vector` and then `argsort`. It is one numpy line because the vectors are normalised, which makes the dot product a cosine.

Slides 9–10 · Chunking; vector databases · 5 min · trim at 10

On screen. Three chunking strategies; then vector DBs in plain words.

The move. **“More RAG failures come from chunking than from model choice.”** Say it twice — it is the most actionable sentence in the session. Then deflate the database question: what they need today is a numpy array. Do not add infrastructure until you have an infrastructure-sized problem.

Depth panel [D] (slide 9). *Four upgrades once paragraph-with-overlap stops being enough* — structure-aware splitting, small-to-search/big-to-read, metadata and filter-before-rank.

Landmine. Do not let a student leave believing they need Pinecone to do this. That belief costs them a working prototype.

Slides 11–12 · RAG end to end; the grounded prompt · 6.5 min · [D]

On screen. The RAG stepper; then the grounded prompt template with an A/B toggle.

The move. Step the pipeline slowly. **Stage 5, Augment, is the money screen** — *“the whole industry is this prompt.”* At stage 6 click the [1] and let the citation flash its source chunk. Then slide 12: three load-bearing lines in the template, and the A/B toggle — delete the

escape hatch and watch the invention come back. Call back to the villain: “**the escape hatch is what 9 a.m. was missing.**”

Depth panel [D] (slide 11). *The stage production adds between retrieve and augment* — reranking.

One level under. The escape hatch is the instruction that permits refusal: answer only from the context, and say you don't know if it isn't there. Without it the model treats the retrieved context as a hint rather than a boundary.

Slides 13–15 · Where RAG breaks; vs fine-tuning; you've been using it · 6.5 min · trim at 14 and 15 · [D]

On screen. Four failure flips; the three-way comparison; then familiar products.

The move. Give them the **debug order** and make them write it down: retrieval → chunks → prompt → model. Name the nastiest failure: a stale index gives confident, *cited*, wrong answers — the citation makes it worse, not better. Then make the room repeat: “**fine-tuning teaches behaviour; RAG provides knowledge.**” On 15, point at the scheme-assistant and regulations-bot cards as capstone seeds.

Depth panel [D] (slide 13). *Two scores, never one* — recall@k and MRR for retrieval, faithfulness for generation. The four failures split cleanly into retrieval failures and generation failures, and one number cannot tell them apart.

Slides 16–18 · The fight; lab architecture; six ideas · 5 min · trim at 16

The move. Slide 16 is the hot take — read once, slowly, three seconds of silence, one counter-argument, park the rest. Then the two pipelines: ingest once, query forever, about sixty lines, and every stage is something they have already touched this weekend.

Slide 19 · Lab 4: chat with YOUR notes · 50 min

The move. Two-minute brief, then the big build. The number one time sink is **garbage PDF extraction** — have them swap the document or use Session 3's vision extraction rather than fight it. If search returns junk, the answer is almost always chunking. Push the capstone framing while circulating.

Lab shape: A ingest (12) · B search sanity (8) · C full RAG (15) · D break it honestly (10).

Landmine. Do not let a pair spend twenty minutes on a PDF that will not extract. Swap the document at minute five; the lesson is retrieval, not parsing.

Slide 20 · Your AI now knows what you know · 3 min

The move. Show and tell: two honest failures and their fixes. Remind them to **save the notebook** — Session 5 builds on it. “*It knows what you know. After lunch, it gets hands.*”

3 · The depth layer in this deck

SLIDE	PANEL	OPEN IT WHEN
7	Why that one line is allowed to be that short — pooling, normalisation, task types	someone asks how similarity works
9	Four upgrades past paragraph-with-overlap	someone's chunking is failing and they want the real answer
11	The stage production adds — reranking	someone asks how to improve retrieval quality
13	Two scores, never one — recall@k, MRR, faithfulness	someone asks how to evaluate a RAG system

4 · Q&A bank

- “**Why not fine-tune on our documents?**” — Fine-tuning teaches behaviour and style, not facts, and it cannot be updated when a circular changes. RAG can, by re-embedding one file.
- “**How big should a chunk be?**” — Start with a paragraph plus overlap. Too small loses context; too large dilutes the signal and costs more. Then measure — slide 13's depth panel.
- “**What if the answer spans two chunks?**” — That is exactly why top-k is greater than 1, and it is what playground query 2 demonstrates.
- “**Do I need a vector database?**” — Not today. A numpy array is a vector database with fewer moving parts.
- “**How do I know retrieval is the problem and not the model?**” — Hand the model the correct chunk yourself. If it answers correctly, the bug is upstream in retrieval.
- “**My search returns the same three chunks for every query**” (pushback you will get from at least one pair) — the `gemini-embedding-2` batching gotcha. Passing a plain list of strings as `contents` returns **one aggregated embedding for the whole list**, not one per string, and no error is raised; `chunk_vecs` ends up with a handful of rows and `argsort` returns the same chunks every time. Each input has to be wrapped in its own `types.Content(parts=[types.Part.from_text(text=t)])`. The notebook's `embed()` does this and asserts `len(res.embeddings) == len(texts)` — if a pair rewrote the helper, that assertion is the first thing to check.

5 · Misconceptions

THEY BELIEVE	SAY THIS
Fine-tuning is how you add knowledge	Behaviour vs knowledge — make them repeat it (14)

THEY BELIEVE	SAY THIS
Long context makes RAG obsolete	Cost, latency, freshness, citations (4, and Session 6)
A citation means the answer is right	A stale index cites its source and is still wrong (13)
Chunking is a detail	More failures come from chunking than model choice (9)
You need infrastructure to start	A numpy array (10)

6 · Timing pressure map

Budgets total 95.5 minutes plus the lab — this deck has genuine slack. Spend it on slides 8, 11 and 12.

Cut in this order: slide 15 (one card, not four) · slide 18 recall flips · slide 6 (the zero-shared-words example alone carries it).

Never compress: slide 8's query 2 and trap query, slide 11 stage 5, or slide 12's A/B toggle.

The deck flags slides 10, 14, 15 and 16 **compressible** (press **S** to see the marker).

7 · Day-before checklist

Run the full lab notebook on a real key with **your own** documents. Prepare the fallback document set for students who arrive without any. Run all four playground queries and confirm the trap query still shows every bar under 0.35. Toggle the escape hatch off and on and confirm the invention actually returns — it is the session's best single demo. Confirm the embedding model still resolves: `gemini-embedding-2` with `output_dimensionality=768`, and that `embed()` returns one vector **per chunk** (the count assertion in Cell 4 must pass — see the batching gotcha in §4). Have `labs/session-4/sample-os-notes.txt` open and confirm the pre-filled queries retrieve from it.

Session 4 — Run Sheet

Giving AI Your Own Knowledge · 20 slides · 1:36 total

Delivery-day document. The concepts are in `session-4-prep.md` ; this is the clock. The most employable session. Depends on students having brought documents — have a fallback set.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET
0:00	1	Giving AI your knowledge	1 min
0:01	2	Did it survive the night?	2 min
0:03	3	Watch it not know — loudly	3 min
0:06	4	“Just paste all my notes!” — let’s price that	2.5 min
0:08	5	Don’t send everything. Send the right 3 paragraphs.	1.5 min
0:10	6	Keyword search misses meaning	2.5 min
0:12	7	Embeddings, now for whole paragraphs	2.5 min
0:15	8	The search playground	3.5 min
0:18	9	Chunking: how you cut the book	3 min
0:22	10	Vector databases, in plain words	2 min
0:24	11	RAG, end to end	4 min
0:28	12	The grounded prompt template	2.5 min
0:30	13	Where RAG breaks in the wild	3 min
0:33	14	RAG vs paste-it-all vs fine-tuning	2 min

AT	#	SLIDE	BUDGET	
0:35	15	You've been using RAG all along	1.5 min	trim
0:36	16	Your final-year project needs a user, a baseline, and a number.	1.5 min	trim
0:38	17	Lab architecture: two pipelines	2 min	
0:40	18	Six ideas you own now	2.5 min	
0:42	19	Lab 4: chat with YOUR notes	50 min	
1:32	20	Your AI now knows what you know.	3 min	
1:36		end		

trim = the deck flags this slide **compressible** (press **S** and you will see compressible on it) — these are the first things to shorten when you are behind, not beats you must land. D = a <|deeper|> panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

- 0:00 · 1 · Giving AI your knowledge** — Day-2 welcome. ‘The model read the internet — but not YOUR notes. Today we fix that. This is the most employable session.’
- 0:01 · 2 · Did it survive the night?** — Recap quiz. Q4 (no memory) is the setup: ‘THAT gap is what we fix this morning.’
- 0:03 · 3 · Watch it not know — loudly** — Villain slide. Ask, let the syllabus type out — the stamp lands at the end. ‘It cannot know. Notice it didn’t say that.’ If a student’s phone hedges politely: it still invents a ‘typical’ syllabus — the hedge is not knowledge. Call back to the stamp all morning.
- 0:06 · 4 · “Just paste all my notes!” — let’s price that** — Cost slider to 380 (= the Galvin textbook) — read the ₹/month aloud. Three taxes: window, meter, middle.
- 0:08 · 5 · Don’t send everything. Send the right 3 paragraphs.** — The insight: send the right 3 paragraphs, not everything. Open-book exam = good index. RAG.
- 0:10 · 6 · Keyword search misses meaning** — Keyword vs semantic. Zero shared words — pause. ‘You know a machine that maps meaning to coordinates...’ (they shout embeddings).
- 0:12 · 7 · Embeddings, now for whole paragraphs** — Session 1’s map, now for paragraphs. Search = cosine top-k, one numpy line. [D] deeper: pooling, why normalization makes the matrix multiply be cosine, query-vs-document task types, dimension truncation. The “why is it one line” answer.
- 0:15 · 8 · The search playground** — Playground. Run all 4 queries — watch the query dot drop on the map. Q2 = teaching gold: TWO chunks relevant → why top-k not top-1. End on the trap query: every bar under 0.35 — the tool admits ignorance (foreshadows the RAG-

breaks slide).

0:18 · 9 · Chunking: how you cut the book — Chunking, all 3 knives. 'More RAG failures come from chunking than model choice.' Say it twice. [D] deeper: structure-aware splitting, small-to-search/big-to-read, metadata + filter-before-rank, context prefixing. The four upgrades after the boring default.

0:22 · 10 · Vector databases, in plain words — Vector DB in plain words. 'What YOU need today = a numpy array.' Don't add infra until infra-sized problem.

0:24 · 11 · RAG, end to end — CENTERPIECE. RAG stepper, step slowly. Stage 5 (Augment) = the money screen: 'the whole industry is THIS prompt.' Stage 6: click the [1] — the citation flashes its source chunk. Grounded, cited, checkable, live. [D] deeper: retrieve-20-then-rerank-4, bi-encoder vs cross-encoder, hybrid BM25 + semantic, query rewriting. The single biggest post-chunking upgrade — and a lab stretch.

0:28 · 12 · The grounded prompt template — Grounded template — three load-bearing lines. Then the A/B toggle: delete the escape hatch, watch the invention come back. Callback to villain: 'the escape hatch is what 9am was missing.'

0:30 · 13 · Where RAG breaks in the wild — Failure flips. Debug order: retrieval → chunks → prompt → model. Stale index gives confident, cited, wrong answers. [D] deeper: recall@k / MRR / faithfulness — two scores, never one, and the diagnosis order they give you. The most employable slide in the deck.

0:33 · 14 · RAG vs paste-it-all vs fine-tuning — RAG vs fine-tune. Make them repeat: 'Fine-tuning teaches behaviour; RAG provides knowledge.'

0:35 · 15 · You've been using RAG all along — 'You've been using RAG all along' — support bots, NotebookLM, AI search. 'Someone gets paid to build today's lab.' The scheme-assistant and regulations-bot cards are capstone seeds — tell them to steal one.

0:36 · 16 · Your final-year project needs a user, a baseline, and a number. — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Invite disagreement — take one counter-argument now, park the rest for the break. What makes it a project, not a demo: your own corpus + a 10-question eval set + one documented failure and fix.

0:38 · 17 · Lab architecture: two pipelines — Two pipelines: ingest once, query forever. ~60 lines. Every stage they've touched this weekend.

0:40 · 18 · Six ideas you own now — Recall flips. Class says each before clicking.

0:42 · 19 · Lab 4: chat with YOUR notes — 2 min brief, then 50 MIN LAB — the big build. #1 sink = garbage PDF extraction (swap doc / use S3 vision). Search junk = chunking. Push the capstone framing. Links are on screen — point at them.

1:32 · 20 · Your AI now knows what you know. — Show & tell + break. Two honest failures + fixes. 'Knows what you know. After lunch it gets hands.' Remind: SAVE the notebook. Links are on screen — point at them.

The lab hour

Slide 19, 50 minutes. Two-minute brief, then hands off.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** ("what did you expect?"), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Remind them to SAVE the notebook — Session 5 builds on it.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md`; if the whole provider is down, the deck's widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the opening of the next block.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.
- **Running long** — the prep pack's timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Session 5 — Instructor Prep Pack

Making AI Do Things · 23 slides · 2 hours

What this file is. The level under every slide. `session-5-notes.md` is the run sheet. Slide numbers are the deck's own, from `presentations/session-5-making-ai-do-things.html`.

This is the anti-hype session. The room arrives having heard “AI agents” everywhere. You are going to teach them tool use properly, then hand them the arithmetic that explains why most agent demos fail — using their own multiplication. Do not soften it and do not sneer at it; the honest number is more persuasive than either.

1 · The spine

1. **Your app is a brain in a jar** (3) — text in, text out, no hands.
 2. **The model never executes anything; it writes requests** (4) — the trick, and the thread to Session 6.
 3. **The description is the prompt** (6–7) — docstrings just became prompts.
 4. **The loop nobody writes an exit for** (9–10) — impressive, and exactly where danger lives.
 5. **Reliability compounds downward** (12–13) — this explains most failed agent demos.
 6. **One question decides workflow or agent** (11, 15) — do you know the steps in advance?
 7. **Cheapest fix first** (16–17) — and rung zero is “you didn't need AI”.
 8. **You can own the model** (18–20) — a trade, not a religion.
-
-

2 · Slide by slide

Slides 1–3 · Opening; recap; brain in a jar · 6 min

The move. “*Your RAG app is a brain in a jar — text in, text out. Today we wire it to the world. Carefully.*” The recap answers now advance on **your** click — let the room argue before you reveal. Then name the four disabilities: it cannot compute, cannot check today's date, cannot read a file, cannot email.

Slide 4 · The model never executes anything. It writes requests · 2 min

On screen. The central trick of the session.

The move. Read it twice. The model writes a request; *your* code executes it. “*Control stays with you*” — and that sentence is the thread you pull all the way through Session 6.

One level under. If challenged, be precise: this is strictly true for tools **you** define and execute. Provider-side tools (code execution, search) run on their infrastructure, and that is a different trust boundary. Concede that distinction immediately — a sharp student will find it and you gain more by naming it first.

Landmine. Do not say “the model runs your function”. It never does. The whole security posture of Session 6 rests on that sentence being exactly right.

Slide 5 · One tool call, step by step · 5 min

On screen. The tool stepper, then a malicious run.

The move. Step through it. Stress **stage 3, validation** — “*the args parse as maths, not DROP TABLE.*” Stage 5’s wording is exact: it got it right **because it delegated**. Then run the malicious version: the model proposes, you dispose.

One level under. The four stages are declare, propose, validate, execute, return. Validation is the only stage the model cannot influence, which is why it is the one you teach hardest.

Slides 6–7 · The description IS the prompt; MCP · 4.5 min · trim at 7 · [D]

The move. “*Docstrings just became prompts.*” A vague docstring routes to the wrong tool — lab Stretch 3 proves it, so name that forward. Then MCP in one breath: they just hand-wrote a declaration; MCP is that, standardised. The USB-C line lands by itself. **Do not demo MCP** — name it and move.

Depth panel [D] (slide 6). *What the SDK is actually sending, and why it works at all.*

Slides 8–10 · Which tool; the agent loop; five failures · 10.5 min · [D]

On screen. A tool-choice vote; a live agent log; five failure flips.

The move. On slide 8, item 4’s correct answer is **no tool** — over-tooling is a failure, and voting for restraint is a maturity beat. Item 5 chains into the next slide. On 9, run the agent log and land the stat: **zero lines of if/else decided that order**. “*Impressive — and exactly where the danger lives.*” On 10, flip four failures, and let the fifth, poisoned tool results, be the red card that bridges to Session 6: **tool results are untrusted input**.

Depth panel [D] (slide 9). *The loop condition nobody writes, and the score nobody keeps* — the trap is the naive `while response.wants_tool_call:` with nothing after it, whose only exit is the model deciding to stop; the on-slide sketch already carries the fix, `and steps < MAX_STEPS`, plus the allow-list and argument validation. Then: evaluate the trajectory, not just the destination.

Landmine. Do not let the agent log read as intelligence. The order emerged from a distribution, and slide 12 prices exactly what that costs.

Slides 11–13 · Workflow vs agent; why chains collapse; multi-agent · 9.5 min · trim at 13

On screen. The distinction; the reliability slider; the multi-agent anti-hype slide.

The move. Slide 11 is **the takeaway of the session** — say the distinction slowly, then the idli line, and let it land. Slide 12 is the centrepiece: drag the reliability slider. At 95% per step, ten steps is about **60%**. “*This explains most failed agent demos.*” At 99%, twenty steps is 82%. Then run the Monte-Carlo agent three times and let it die at a different step each time. On 13, ask the room first: two 90%-reliable agents talking — what is the joint reliability? **81% before any work happens**. Then name the legitimate pattern: parallel fan-out plus a judge.

One level under. The arithmetic is verified: $0.95^{10} = 0.5987$, $0.95^{20} = 0.3585$, $0.99^{20} = 0.8179$. Write one of them on the board rather than only showing the slider — students trust chalk.

Hardest question. “*But agents work in demos.*” — Demos are short chains under supervision. The compounding is a property of chain length, and it is why production systems shorten chains rather than lengthen them.

Slides 14–17 · The fight; one question; the ladder; defend your vote · 11.5 min · trim at 14

The move. Slide 14 is the hot take — once, slowly, three seconds, one counter-argument, park the rest. Slide 15 gives the single decision rule: **do you know the steps in advance?** Yes is a workflow, and that is most real business AI by a wide margin; no is an agent with a leash. Then the escalation ladder, clicking each rung, and the rule that governs it: escalate only when your **eval** says the cheap rung failed. Fine-tuning is rung five, never rung one. Slide 17 is effectively the exam — students vote and defend; item 5, the capital of France, is **rung zero**, because over-engineering is a failure mode too.

Slides 18–20 · API vs owned; Ollama; the trade · 7.5 min · trim at 18 and 20 · [D]

The move. Frame it with their own future: in fintech or KYC work, regulators ask where prompts go. Then **turn the Wi-Fi off** and run a local model live. “*Four billion knobs, on this laptop, no meter.*” Then land slide 20: local versus API is an engineering trade, not a religion, and production is usually hybrid — API for the hard 10%, local for the routine 90%.

Depth panel [D] (slide 19). *Will it fit on my laptop? Arithmetic, not vibes* — $\text{parameters} \times \text{bits} \div 8$.

Landmine. Have the model pulled **before** the session and verify it runs offline. This demo fails badly on college Wi-Fi if you rely on a download.

Slides 21–23 · Six ideas; Lab 5; the close · 63.5 min

The move. Recall flips with a cold-call — point at a student, they answer, *then* you flip. Then the lab: nudge strong pairs to the stretch, because RAG-as-a-tool is where the capstone assembles itself. Part C’s manual loop is what demystifies the whole thing; checkpoint 3 is verbal. Close with show and tell: best “called both tools”, best scenario defence, and name the strongest counter-argument to slide 14.

Lab shape: A calculator tool (12) · B two tools chained (12) · C see the machinery (8) · D scenario cards, paper and pen (10).

3 · The depth layer in this deck

SLIDE	PANEL	OPEN IT WHEN
6	What the SDK is actually sending	someone asks how the model “knows” the function
9	The loop condition nobody writes, and the score nobody keeps	someone asks how an agent decides to stop
19	Will it fit on my laptop? — $\text{params} \times \text{bits} \div 8$	someone asks what hardware they need

4 · Q&A bank

“Is this what AutoGPT was?” — Yes, and it is why AutoGPT disappointed. Slide 12 is the reason, in one multiplication.

“Can the model call a tool I didn’t give it?” — No. It can only propose from the declared menu, and your code decides whether to run it. That is the whole safety story.

“Should I use LangChain?” — Not to learn this. Write the loop once by hand — Part C — and then decide whether a framework is buying you anything.

“How many tools is too many?” — When the model starts choosing wrong, which your eval tells you. Over-tooling is slide 8, item 4.

“Is a multi-agent system better?” — Usually worse, and the arithmetic on slide 13 is why. Fan-out plus a judge is the pattern that survives.

5 · Misconceptions

THEY BELIEVE	SAY THIS
The model executes code	It writes a request; your code executes (4)
Agents are the default architecture	One question decides, and the answer is usually workflow (15)
More agents means more capability	Two 90% agents are 81% before doing anything (13)
Tool output is trustworthy	Tool results are untrusted input — the Session 6 bridge (10)
Local models are toys	It is a trade, not a religion; production is hybrid (20)

6 · Timing pressure map

Budgets total 120 minutes including a 50-minute lab. **Tight.**

Cut in this order: slide 8 (three items, not five) · slide 21 recall flips · slide 17 (two items, not five).

Never compress: slide 4 (read twice), slide 12 (the slider and the Monte-Carlo run), slide 15 (the decision rule).

The deck flags slides 7, 13, 14, 18 and 20 **compressible** (press **S** to see the marker).

7 · Day-before checklist

Pull the local model and test it with Wi-Fi off. This is the one demo that cannot be recovered live. Run the reliability slider and be able to state $0.95^{10} \approx 60\%$ without looking. Run the agent log until you have seen it choose a different order. Open the lab notebook and run the two-tool chain end to end. Confirm automatic function calling still behaves on the pinned model — `fact-check.md`.

Session 5 — Run Sheet

Making AI Do Things · 23 slides · 2:00 total

Delivery-day document. The concepts are in `session-5-prep.md`; this is the clock. The anti-hype session. The reliability slider does the persuading; you just have to not rush it.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET	
0:00	1	Making AI do things	1 min	
0:01	2	Still true after lunch?	3 min	
0:04	3	Your RAG app is a brain in a jar	2 min	
0:06	4	The model never executes anything. It writes requests. Your code executes.	2 min	
0:08	5	One tool call, step by step	5 min	
0:13	6	Function declarations: the description IS the prompt	3 min	D
0:16	7	MCP: USB-C for tools	1.5 min	trim
0:18	8	Which tool should it call?	4 min	
0:22	9	The agent loop: while it wants tools, feed it	3 min	D
0:24	10	Five ways tool use goes sideways	3.5 min	
0:28	11	Most problems don't need an agent. They need a boring workflow.	2.5 min	
0:30	12	Why long agent chains collapse	5 min	
0:36	13	Multi-agent: more agents, more compounding	2 min	trim
0:38	14	Most production "AI agents" are a while-loop in a trench coat.	1.5 min	trim

AT	#	SLIDE	BUDGET	
0:39	15	Workflow or agent? One question decides	2.5 min	
0:42	16	The escalation ladder: cheapest fix first	3.5 min	
0:45	17	Right approach? Defend your vote	4 min	
0:49	18	API models vs models you own	2 min	trim
0:51	19	Ollama: a model in your pocket	4 min	D
0:55	20	Local vs API: an engineering trade, not a religion	1.5 min	trim
0:56	21	Six ideas you own now	3 min	
1:00	22	Lab 5: give it hands	50 min	
1:50	23	Your AI knows, sees, and now acts.	10.5 min	
2:00		end		

trim = the deck flags this slide **compressible** (press **S** and you will see **compressible** on it) — these are the first things to shorten when you are behind, not beats you must land. **D** = a **<|deeper|>** panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

- 0:00 · 1 · Making AI do things** — ‘Your RAG app is a brain in a jar — text in, text out. Today we wire it to the world. Carefully.’
- 0:01 · 2 · Still true after lunch?** — Recap quiz. Reinforces embeddings + grounding from this morning. Answers advance on YOUR click now — let them argue first.
- 0:04 · 3 · Your RAG app is a brain in a jar** — Brain in a jar: can’t compute, can’t check today, can’t read a file, can’t email. Text in, text out.
- 0:06 · 4 · The model never executes anything. It writes requests. Your code executes.** — THE trick, read twice. Model writes requests; YOUR code executes. ‘Control stays with you’ — the thread to Session 6. If challenged: strictly true for tools YOU define — providers also offer hosted tools (code execution, search) that run on THEIR servers. Same request-based principle, different executor.
- 0:08 · 5 · One tool call, step by step** — Tool stepper. Step through; stress stage 3 (validate ‘args parse as math, not DROP TABLE’). Stage 5 exact ‘because it delegated.’ Then the malicious run: the model proposes, your code disposes — plant the Session 6 seed.
- 0:13 · 6 · Function declarations: the description IS the prompt** — ‘Docstrings just became prompts.’ Vague docstring → wrong tool. (Lab Stretch 3 proves it.) Name-drop MCP: the standardized menu format — same idea, industry-wide. [D] deeper: the JSON schema the

SDK actually sends, the four message roles, parallel tool calls, and why tool results eat your context budget.

0:16 · 7 · MCP: USB-C for tools — One breath. They just hand-wrote a declaration — MCP is that, standardized. USB-C line lands by itself. Don't demo; name it and move.

0:18 · 8 · Which tool should it call? — GAME. Vote each tool. Item 4 = NO tool (maturity: over-tooling is a failure). Item 5 = chain → next slide. You control the pace — click advances.

0:22 · 9 · The agent loop: while it wants tools, feed it — Run agent log. 'Model chose the order — nobody scripted it. Impressive, and exactly where danger lives.' Land the stat line: 0 lines of if/else deciding the order. [D] deeper: the missing loop guard (max steps + allow-list + arg validation), context growth over a long run, and trajectory eval: tool-choice accuracy, step count, wasted calls.

0:24 · 10 · Five ways tool use goes sideways — Five failure flips. Flip 4; the 5th (poisoned tool results) is the red card — Session 6 bridge. 'Tool results are untrusted input.'

0:28 · 11 · Most problems don't need an agent. They need a boring workflow. — THE takeaway. Workflow (you fix steps) vs agent (model picks). Say the distinction slowly, then the idli line — let it land.

0:30 · 12 · Why long agent chains collapse — CENTERPIECE. Reliability slider. 95% → 10 steps ≈ 60%. 'This explains most failed agent demos.' 99% → 20 = 82%. Then run the Monte-Carlo agent three times — it dies at a different step each run. Let them feel it.

0:36 · 13 · Multi-agent: more agents, more compounding — Anti-hype with their own math. Ask the room: two 90%-reliable agents chatting — reliability? 81% before any work happens. Parallel fan-out + judge = the legit pattern.

0:38 · 14 · Most production "AI agents" are a while-loop in a trench coat. — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Invite disagreement — take one counter-argument now, park the rest for the break.

0:39 · 15 · Workflow or agent? One question decides — One question decides: do you know the steps in advance? Yes=workflow (most real business AI, by a wide margin). No=agent with a leash.

0:42 · 16 · The escalation ladder: cheapest fix first — Ladder, click each rung. Escalate only when your EVAL says the cheap rung failed. Fine-tune is rung 5, never rung 1.

0:45 · 17 · Right approach? Defend your vote — GAME = the exam. Vote + defend. Item 5 (capital of France) = rung ZERO. Over-engineering is a failure mode too. Click advances — pace the argument.

0:49 · 18 · API models vs models you own — API vs local: privacy, cost, offline. Your fintech/KYC reality: regulators ask where prompts go.

0:51 · 19 · Ollama: a model in your pocket — OLLAMA — Wi-Fi OFF, run live: ollama run gemma4:e4b (gemma3:4b still works). '4 billion knobs, on this laptop, no meter.' 8GB RAM runs it at home — no 8 GB machine? Same model in a free Colab runtime. If the live demo sulks, run the typing race — and call it a simulation. [D] deeper: $\text{params} \times \text{bits} \div 8$ — will it fit? Plus KV-cache headroom and why memory bandwidth, not compute, sets tokens/sec.

0:55 · 20 · Local vs API: an engineering trade, not a religion — Local vs API = a trade, not a religion. Production = hybrid: API for hard 10%, local for routine 90%.

0:56 · 21 · Six ideas you own now — Recall flips. Class says each before clicking. 'Pick one for me' = cold-call: point at a student, they answer, THEN flip.

1:00 · 22 · Lab 5: give it hands — 2 min brief, then 50 MIN LAB. Nudge strong pairs to Stretch (RAG-as-tool = capstone assembles itself). Part C manual loop de-mystifies. Checkpoint 3 is verbal. Links are on screen — point at them.

1:50 · 23 · Your AI knows, sees, and now acts. — Show & tell + last break (10.5 min). Best 'called both tools' + best scenario defense + name the strongest hot-take counter-argument. 'Knows, sees, acts. Finale: we break it all.' Bring the notebook. Links are on screen — point at them.

The lab hour

Slide 22, 50 minutes. Two-minute brief, then hands off.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** (“what did you expect?”), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Nudge the strong pairs toward RAG-as-a-tool; that is where the capstone assembles itself.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md`; if the whole provider is down, the deck's widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the opening of the next block.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.
- **Running long** — the prep pack's timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Session 6 — Instructor Prep Pack

Breaking It, Securing It, Shipping It · 21 slides · 2 hours

What this file is. The level under every slide. `session-6-notes.md` is the run sheet. Slide numbers are the deck's own, from `presentations/session-6-breaking-securing-shipping.html`.

This is the finale, and it has an unusual shape. Slide 19 is not a two-minute slide — it is a **76-minute block**: the 30-minute red-team/harden sprint, then ~35 minutes of demos (3 min × pairs), then the course close. The deck budgets $39.5 + 76 + 4.5 = 120$ minutes, so the presenter's pace badge stays honest through the finale. Plan the last 80 minutes around that, not around the slide count. **Headcount rule, decided before Part A starts:** three-minute demos fit at most 12 pairs in 35 minutes; with more than 12 pairs, announce 60–90-second lightning demos *before* the sprint begins, never at demo time.

Highest energy of the weekend. *"We attack everything you built, then defend it, then you demo."*

1 · The spine

1. **Every capability is a surface** (3) — the hat change.
 2. **There is no border between instructions and data** (4–6) — injection, direct then indirect.
 3. **No single fix, so layer them** (7–8) — defence in depth, and match trust to blast radius.
 4. **It works in Colab — now make the assumptions visible** (10) — cost, speed, reliability, security, observability.
 5. **Design for a component that is sometimes wrong** (13–14) — UX and accountability.
 6. **Your users decide whose law you are under** (15).
 7. **The gaps are your roadmap** (16) — then build, demo, close.
-

2 · Slide by slide

Slides 1–3 · The finale; 90-second recap; the hat change · 4.5 min

The move. Highest energy of the course. The recap is six fast true/false, one per session, no notes — S1 numbers, S2 evals, S3 multimodal, S4 grounding, S5 compounding, S6 tool results. Then **make the hat change physical** — actually pause. *"For five sessions you built. Now think like an attacker. Every capability is a surface."*

Slides 4–5 · Direct injection; indirect injection · 6 min

On screen. A bot induced to issue a fake certificate; then a poisoned document attacking a RAG pipeline.

The move. Run the direct injection and let the room react before you explain. *"There is no border between instructions and data. This is SQL injection reborn."* Hit **Try another payload** so it does not look like one lucky string. Then slide 5, the scary one: the payload is

invisible in the chunk until you reveal it — “*your RAG read what you couldn’t see.*” Land the asymmetry: **the attacker never talks to your bot — they just leave a document where you will fetch it.**

One level under. Injection has no escape character. Everything the model receives is one token stream, and the model has no mechanism to attribute provenance to a span. That is why filtering cannot be the fix — which is exactly slide 7’s depth panel.

Landmine. Do not present injection as solvable. The honest position is: mitigable, layered, and capped by what the tools can do.

Slide 6 · Jailbreaks and leaks: the two quieter doors · 2.5 min

The move. Flip both, then put the golden rule on screen and say it slowly: “**never put anything in a prompt you couldn’t survive seeing on the front page.**”

Slides 7–8 · Defence in depth; keep a human on anything that bites · 5.5 min · [D]

On screen. Toggleable defence layers; then the trust dial.

The move. Centrepiece. Toggle the layers: at zero to two the bot falls; at three to four it holds. “*No single wall is perfect — but four are not all weak at the same time.*” Say OWASP LLM01 once, precisely, without commentary. Then slide 8: **match trust to blast radius.** Read-only can relax; anything with a side effect gets a gate. Get that one dial right and most disasters never leave the building.

Depth panel [D] (slide 7). *Why filtering will never be the fix — and what is.* Open it when a student proposes a regex.

Hardest question. “*Can’t I just detect injection with a classifier?*” — You can raise the cost. You cannot close the class of attack, because the attacker adapts and you are filtering natural language. Layer it, and cap the damage at the tool boundary.

Slide 9 · There is no secure AI agent · 1.5 min · trim

The move. The hot take. Read once, slowly, three seconds of silence, take one counter-argument, park the rest.

Slides 10–12 · It works in Colab — now make the assumptions visible; cost; speed and reliability · 7 min · [D]

On screen. A 20/80 bar; a cost slider to 5,000 users; then the three production properties.

The move. “*It works in Colab. Now make the assumptions visible*” — the 20/80 bar makes it land, and the five things that change are cost, speed, reliability, security and observability. Drag the cost slider to 5,000 users and let the ₹/month climb; the jigarthanda line — *N glasses a day* — makes it visceral. Then flip **caching on** and watch input cost drop about 90%. On 12: speed means stream, because users **forgive slow and hate frozen**; reliability means retries; observability means log. Then the payoff line: **your Session 2 evals become the regression test.**

Depth panels [D] . Slide 11: *read the price list again — the two numbers aren’t the same* (\$1.50 in / \$9.00 out per 1M; output is dearer, and that changes what you optimise). Slide 12: *measure the tail, not the average* — TTFT versus tokens/sec, p50 versus p99.

One level under. The cost-slider constants and every derived figure were re-synced and Python-verified against the live widget; the ₹/\$ rate is 95.5 as of 2026-09; the rates on screen are Gemini 3.5 Flash (previous-gen Flash) list price, and the lab model, 3.5 Flash-Lite, is \$0.30 in / \$2.50 out. If you re-base the currency, `fact-check.md` names the single constant to change.

Slides 13–15 · Honest UX; four questions auditors ask; whose law · 6 min · trim at 13, 14, 15

The move. Slide 14 is a **rename, not a re-teach** — every card points at something they already did, and this slide is for the faculty in the room. Say EU AI Act and NIST AI RMF once, precisely, and move. Slide 15 is the globally useful one, so go slowly: read the four cards fast,

then land the rule — **you are judged where your users are, not where you are.**

Landmine. Do not give legal advice, and say so. Dates and thresholds move; the architectural consequences (data residency, retention, PII redaction before the prompt) do not.

Slides 16–18 · The ship-it checklist; the capstone; how to demo · 6.5 min

The move. Toggle the checklist honestly, including the items you would fail. **“The gaps are your roadmap, not your shame.”** Then the capstone brief: roughly 30 minutes of red-team and harden, then three-minute demos per pair. Say the thing that sets the tone: **the failure story matters more than the polish.** Then the demo rules — pre-run everything, lead with the problem, show the failure, and take **one unrehearsed input from the audience**: it is part of the 40 demo points on `certificate.html`, not a dare.

Landmine. Slide 18 exists because bad demos are avoidable. Do not skip it to save two minutes; it saves you thirty.

Slide 19 · Six sessions, one throughline · 76 min

This slide is the last 76 minutes. Inside it: the 30-minute red-team and harden sprint, then ~35 minutes of demos (3 min × pairs), then the course close.

The move. During the sprint, circulate and push pairs to attack a *classmate*’s bot, not their own — they cannot see their own blind spots. During demos, keep time hard at three minutes; the failure story is the graded part. **Lightning-demo rule:** more than 12 pairs means 60–90 seconds each — what it does, the one failure, the one fix — and you announce that before Part A, so nobody rehearses a three-minute story they cannot tell. Then close on the spine: **Predict → Measure → See → Know → Act → Ship**, one verb per session.

Scoring a pair whose app does not run. The rubric is 40 demo · 25 honest eval · 15 failure analysis · 10 right tool · 10 presentation. A pair whose app crashes on stage but whose 10-question eval set is real, whose failure analysis is specific, and whose hardening story is genuine can still earn up to about **50** — the 25 + 15 + 10 are independent of the demo running, and part of the 40 is the unrehearsed-input and failure story, which they can still narrate from a log. Score it that way, explicitly, and say so before the demos start: it is the only thing that stops pairs faking a green run.

Lab shape: A attack a naive bot (8) · B harden it (8) · C red-team a classmate (10) · D ship-readiness self-audit (5).

Slides 20–21 · Where to go from here; the close · 4.5 min · `trim` at 20

The move. Ship it for real — a live URL beats any certificate. Say the security line explicitly: key in secrets or an environment variable, **never in the pushed notebook**. Then close: *“You came as users. You leave as builders.”* And the line worth landing last: **“knowing how to use ChatGPT expires next semester — knowing why it breaks compounds.”** Point at the repo they keep.

3 · The depth layer in this deck

SLIDE	PANEL	OPEN IT WHEN
7	Why filtering will never be the fix — and what is	someone proposes a regex or a classifier

SLIDE	PANEL	OPEN IT WHEN
11	Read the price list again — input and output are not the same number	someone asks how to cut cost
12	Measure the tail, not the average — TTFT, p50 vs p99	someone asks why their app feels slow

4 · Q&A bank

- “**So AI apps can’t be secure?**” — Not in the sense of proof. You cap blast radius, layer defences, and keep a human on anything irreversible. That is the same posture as the rest of security engineering.
- “**Isn’t this just SQL injection?**” — Structurally yes, with one crucial difference: SQL has an escape character and a parser boundary. Natural language has neither.
- “**Who is liable when it goes wrong?**” — Air Canada is the case to name; the tribunal held the company liable for its bot. Session 2 covered it, and slide 14 is where it lands.
- “**Do I need to comply with the EU AI Act?**” — If you have EU users, it reaches you. Not legal advice — say that plainly — but that is the architectural assumption to build on.
- “**How do I keep the cost down?**” — Shorter context, cheaper model, caching, and streaming for perceived speed. The depth panel on slide 11 has the arithmetic.

5 · Misconceptions

THEY BELIEVE	SAY THIS
Injection is a bug to patch	There is no escape character; you layer and cap damage (4–7)
A filter will stop it	Filtering raises cost, never closes the class (7 depth panel)
Working in Colab means it ships	It ran once, for you, on one input — make the assumptions visible (10)
Cost is one number	Output is dearer than input, and caching changes both (11)
Compliance is a legal problem	It is an architecture problem, decided early (15)

6 · Timing pressure map

Slide budgets total 120 minutes — 39.5 of talk, then slide 19's 76-minute build-and-demo block, then 4.5 to close. **The whole risk in this session is demos overrunning.** Hold three minutes per pair (or 60–90 seconds with more than 12 pairs, announced before Part A), out loud, with a visible timer.

Cut in this order: slide 6 (one door, not two) · slide 17 (the brief can be 90 seconds) · slide 2 (four recap items, not six).

Never compress: slide 5 (indirect injection), slide 7 (the layer toggle), or the demos.

The deck flags slides 9, 13, 14, 15 and 20 **compressible** (press **S** to see the marker).

7 · Day-before checklist

Run both injections and have a second payload ready in case the first is fixed by a model update. Toggle all defence layers and confirm the bot actually falls at zero to two layers. Drag the cost slider to 5,000 and check the ₹ figure against `fact-check.md`. Re-read the jurisdiction cards — that slide was trimmed once for overflow, so confirm it still fits your projector at 900px. Have a stopwatch for the demos, and decide your hard stop before the room talks you out of it.

Session 6 — Run Sheet

Breaking It, Securing It, Shipping It · 21 slides · 2:00 total

Delivery-day document. The concepts are in `session-6-prep.md` ; this is the clock. The finale. Slide 19 is a 76-minute block (30-min sprint + ~35-min demos + close), not a slide. Demos overrun unless you hold the clock; more than 12 pairs means 60–90 s lightning demos, announced before Part A.

Generated from the deck, so the timings are the deck's own budgets. Press **S** in the deck for presenter mode — it shows the same numbers with a live timer.

Timing map

AT	#	SLIDE	BUDGET	
0:00	1	Breaking it, securing it, shipping it	1 min	
0:01	2	Everything, in 90 seconds	2 min	
0:03	3	For five sessions you built. Now: think like an attacker.	1.5 min	
0:04	4	Prompt injection: data becomes commands	3 min	
0:08	5	Indirect injection: the document attacks your RAG	3 min	
0:10	6	Jailbreaks & leaks: the two quieter doors	2.5 min	
0:13	7	Defense in depth: no single fix, so layer them	3.5 min	D
0:16	8	Keep a human on anything that bites	2 min	
0:18	9	There is no perfectly secure AI agent. Build one whose blast radius you can survive.	1.5 min	trim
0:20	10	It works in Colab. Now make the assumptions visible.	1.5 min	
0:22	11	Cost: every token is a coin	3 min	D
0:24	12	Fast, resilient, and observable — speed, reliability, observability	2.5 min	D
0:27	13	UX patterns for honest AI products	1.5 min	trim

AT	#	SLIDE	BUDGET	
0:28	14	Responsible AI: the four questions auditors now ask	2 min	trim
0:30	15	Your users decide whose law you're under	2.5 min	trim
0:33	16	The ship-it checklist	2.5 min	
0:36	17	Capstone: the sprint & the demo	2 min	
0:38	18	How to demo without dying	2 min	
0:40	19	Six sessions, one throughline	76 min	
1:56	20	Where to go from here	2.5 min	trim
1:58	21	You came as users. You leave as builders.	2 min	
2:00		end		

trim = the deck flags this slide **compressible** (press **S** and you will see **compressible** on it) — these are the first things to shorten when you are behind, not beats you must land. **D** = a **<|deeper|>** panel lives on this slide; press **D** to open it.

Slide beats

One line per slide — what you actually do. Fuller reasoning is in the prep pack.

- 0:00 · 1 · Breaking it, securing it, shipping it** — The finale. Highest energy. ‘We attack everything you built, then defend it, then you demo.’
- 0:01 · 2 · Everything, in 90 seconds** — Final recap — the whole weekend. No notes. Six fast T/F, one per session: S1 numbers, S2 evals, S3 multimodal, S4 grounding, S5 compounding, S6 tool results.
- 0:03 · 3 · For five sessions you built. Now: think like an attacker.** — Hat change — make it physical, pause. ‘Five sessions you built. Now think like an attacker. Every capability is a surface.’
- 0:04 · 4 · Prompt injection: data becomes commands** — CENTERPIECE. Run direct injection. Bot makes a fake certificate. Let the room react. ‘No border between instructions and data. SQL injection reborn.’ Hit ‘Try another payload’ — injection is a family (override, roleplay, translation), not one gimmick.
- 0:08 · 5 · Indirect injection: the document attacks your RAG** — Run indirect injection — the scary one. The payload is invisible in the chunk until you reveal it: ‘your RAG read what you couldn’t see.’ ‘Attacker never talks to your bot — this is the app you built this morning.’
- 0:10 · 6 · Jailbreaks & leaks: the two quieter doors** — Jailbreak + leak flips. Golden rule (now big on-screen): ‘never put anything in a prompt you couldn’t survive seeing on the front page.’

0:13 · 7 · Defense in depth: no single fix, so layer them — CENTERPIECE. Toggle defense layers (equal weight now). 0–2 layers the bot falls; 3–4 and it holds. ‘No single wall is perfect — four aren’t all weak at once.’ OWASP #1, no complete fix. [D] deeper: why there is no escape character in a token stream, and the three-legged framing — Simon Willison’s ‘lethal trifecta’ (private data + untrusted content + a way out) — remove one leg and exfiltration dies. Strongest security idea in the course.

0:16 · 8 · Keep a human on anything that bites — ‘Match trust to blast radius.’ Read-only relax, side-effects gate. Get this one dial right and most disasters never leave the building.

0:18 · 9 · There is no perfectly secure AI agent. Build one whose blast radius you can survive. — Pick the fight deliberately. Read it once, slowly, then be quiet for 3 seconds. Invite disagreement — take one counter-argument now, park the rest for the break.

0:20 · 10 · It works in Colab. Now make the assumptions visible. — ‘It works in Colab — now make the assumptions visible.’ The 20/80 bar makes it land: a demo runs once, for you, on one input. Five things change: cost, speed, reliability, security, observability.

0:22 · 11 · Cost: every token is a coin — Cost slider to 5000 users. Watch \$/month climb — and the jigarthanda line (‘N glasses a day’) makes it visceral. Then flip caching ON — watch input cost drop 90%; that’s the ‘cache repeated questions’ lever, real Gemini feature. Cheap-by-default: free local model for the boring 90%, API only for the hard 10%. [D] deeper: why output costs ~6× input (prefill vs decode), prefix caching, batch endpoints, and the cost-per-user formula that decides the architecture.

0:24 · 12 · Fast, resilient, and observable — speed, reliability, observability — Speed=stream (‘forgive slow, hate frozen’). Reliability=retries. Observability=log. ‘Your S2 evals become the regression test.’ [D] deeper: TTFT vs tokens/sec, p50 vs p99, retry jitter, idempotency, and exactly what to log.

0:27 · 13 · UX patterns for honest AI products — UX for honest AI: sources, retry, uncertainty, escape-to-human. Design for a component that’s sometimes wrong.

0:28 · 14 · Responsible AI: the four questions auditors now ask — Rename, don’t re-teach. Every card points at something they DID. Faculty in the room: this slide is for them. EU AI Act + NIST AI RMF said once, precisely.

0:30 · 15 · Your users decide whose law you’re under — The globally-useful slide — go slowly, faculty care about this one. Read the four cards fast, then land the rule: you are judged where your USERS are, not where you are. The EU reaches you through the output; the US has no single law but ‘the AI decided’ is not a defence; India’s binding law is data protection, so ask whose data is in the context window; the Gulf’s real constraint is data residency → callback to Ollama in S5. Close on the last line: they already built all four duties this weekend. Say once, precisely: dates move, this isn’t legal advice, check the current text.

0:33 · 16 · The ship-it checklist — Ship checklist, toggle honestly. ‘The gaps are your roadmap, not your shame.’ Sets up lab Part D.

0:36 · 17 · Capstone: the sprint & the demo — Capstone brief. ~30 min red-team+harden, then 3-min demos per pair (~35 min total). ‘The failure story matters more than the polish.’ Links are on screen — point at them.

0:38 · 18 · How to demo without dying — Demo rules: pre-run, lead with the problem, show the failure. Critical — bad demos are avoidable.

0:40 · 19 · Six sessions, one throughline — This slot is the finale: 30-min red-team/harden sprint + ~35 min demos (3 min × pairs) + close — 76 min budgeted. If there are more than 12 pairs, announce 60–90 s lightning demos before Part A starts, not at demo time. Only then the course close: the six-session spine — Predict → Measure → See → Know → Act → Ship, each verb igniting in its session’s colour. ‘You didn’t learn to use ChatGPT — you learned how these systems work, where they fail, and how to build safely.’ That’s the difference between a user and an engineer.

1:56 · 20 · Where to go from here — Where to go from here. Ship it for real (Streamlit/Vercel — a live URL beats any certificate; key in secrets or an env var, never in the pushed notebook). Turn it into the final-year project: department RAG assistant + 100-question eval, measured against a simpler baseline, one attack + mitigation as the security chapter — ‘here is the metric, here is the baseline, here is where it fails.’ The meta-skill: you own the fundamentals under the buzzwords — tokens, evals, retrieval, tools, safety.

1:58 · 21 · You came as users. You leave as builders. — Thanks. ‘You came as users. You leave as builders.’ ‘Knowing how to use ChatGPT expires next semester — knowing why it breaks compounds.’ Point to the repo they keep. ‘Ship something.’ Links are on screen

— point at them.

The lab hour

Slide 19, 76 minutes: 30-minute red-team/harden sprint, ~35 minutes of demos (3 min × pairs; 60–90 s lightning demos if more than 12 pairs — announced before Part A), then the close. Two-minute brief, then hands off. A pair whose app does not run can still score up to ~50 on a real eval set, failure analysis and hardening story — say that before demos start.

Your four moves, the same every lab: **circulate** (never sit), **ask before answering** (“what did you expect?”), **checkpoint sweep** at +40 minutes, **collect two artifacts** for show and tell.

The five-minute rule: stuck for five minutes, ask a neighbour before asking you. Say it at the brief.

Close

Hold demos to 3 minutes each. The failure story is the graded part.

Fallbacks

- **API down or rate-limited** — switch to the fallback model named in `fact-check.md`; if the whole provider is down, the deck's widgets run offline and the lab becomes a paper walkthrough.
- **No internet** — every deck is offline-capable. Open the deck, run the widgets, and compress the lab to Parts A–C; run the remaining parts as the post-weekend stretch.
- **Projector washes out the dot-grid** — press **F** for fullscreen; if it is still bad, the decks are legible at 100% browser zoom on a laptop passed around.
- **Running long** — the prep pack's timing pressure map lists what to cut, in order. Cut from the top of that list, never from a checkpoint.

Lab Facilitation Guide — all six labs

What this file is. How to run the lab hour. It is the same hour six times, and the shape does not change — which is the point, because a room that knows the rhythm stops asking about the rhythm and starts asking about the work.

Taught once, on Session 1’s “*Your lab kit, and the rhythm*” slide. After that you only have to point at it.

The universal rhythm

Every lab is the same five moves, in the same order:

Brief	2–3 min	What you are building, and the one thing that must work by the end
Build	~50 min	Pairs — both partners have a key, one drives per part, swap at each checkpoint
Sweep	at +40	You walk the room and check every pair against the checkpoint list
Collect	last 5	Two artifacts for show and tell — one that worked, one that failed usefully
Show	next slide	Two or three pairs, three minutes total

Pairs, not groups of four. Both partners create their own API key in Part A; one drives per lab part and they swap at each checkpoint, so both hands have typed every kind of cell by the end. The moment one person is only watching, they have stopped learning, and by Session 4 they cannot catch up.

Your four moves during the hour

- 1 • Circulate. Never sit down.** The single highest-value thing you do in twelve hours is walk. A stuck pair will not raise a hand; they will quietly fall behind and leave.
- 2 • Ask before you answer.** “*What did you expect to happen?*” and “*What did it actually do?*” Most bugs solve themselves in the gap between those two sentences, and the student keeps the skill.
- 3 • Checkpoint sweep at +40 minutes.** Not before — earlier and you interrupt flow; later and you cannot rescue anyone. Walk the room with the checkpoint list and ask each pair which one they are on. This is also how you find the pair that has been stuck for twenty minutes and said nothing.
- 4 • Collect two artifacts.** One thing that worked, one thing that failed interestingly. Show and tell without collection is dead air; collection makes it two minutes of the best content in the session.

The five-minute rule

Say this at every brief: **stuck for five minutes** → **ask your neighbour** → **then ask me**. It halves your queue and it is how the room starts teaching itself. By Session 4 you will hear pairs debugging each other, which is the moment the weekend is working.

Timing template for a 50-minute lab

0:00–0:03	Brief. The goal, the checkpoints, the five-minute rule. Links on screen — point, do not read
0:03–0:10	Setup and first cell. Your busiest window — most failures are environmental and happen here
0:10–0:40	Build. Circulate continuously
0:40–0:45	Checkpoint sweep
0:45–0:50	Stretch goals for the fast pairs; rescue for the stuck ones; collect artifacts

If you are behind, cut a stretch goal. Never cut a checkpoint — the checkpoints are what the next session assumes.

Per-lab specifics

Lab 1 • Your first AI API call. A confirm access and key (10) · B first call (10) · C five prompts (10) · D one prompt, three models (10) · E build your test set (10) — 50 total. *The sink:* API key setup. Have the five steps on the board before anyone opens a laptop (and the “I already have a Google Cloud account → import a project first” line with them). *Protect:* Part E — it is the hand-off to Lab 2, which consumes those ten questions. Part D is the designated cut: if you are behind at the +30 checkpoint, drop D and go straight to E.

Lab 2 • The lie detector. A prompt makeover (15) · B first eval (15) · C the arena (15). *The sink:* students loosening the expected string until the test passes. Name it at the brief, before it happens. *Protect:* diagnosing every **X**. The documentation is the deliverable, not the score.

Lab 3 • Interrogate your photos. A interrogation ladder (10) · B document → JSON (10) · `response_schema` (5) · C handwriting (10) · D break it (10). *The sink:* the folder upload. Demo it once yourself before they start. *Protect:* pushing Part B until `json.loads` actually passes. That is the checkpoint that matters.

Lab 4 • Chat with YOUR notes. A ingest (12) · B search sanity (8) · C full RAG (15) · D break it honestly (10). *The sink:* garbage PDF extraction. Swap the document at minute five rather than fighting it, or use Session 3’s vision extraction. *Protect:* Part B. If search is junk, the answer is chunking, and no amount of prompt work will save it.

Lab 5 • Give it hands. A calculator tool (12) · B two tools chained (12) · C see the machinery (8) · D scenario cards, paper (10). *The sink:* over-tooling — pairs adding a third and fourth tool and then wondering why routing broke. *Protect:* Part C, the manual loop. It is what demystifies agents. Nudge strong pairs to the RAG-as-a-tool stretch — that is where the capstone assembles itself.

Lab 6 • Break it, then ship it. A attack a naive bot (8) · B harden it (8) · C red-team a classmate (10) · D ship-readiness audit (5). *The sink:* pairs attacking their own bot. Make them swap — nobody sees their own blind spot. *Protect:* Part C. The classmate's attack is the lesson.

If the internet or a service dies

1. **One pair affected** — pair them with a working machine. Both still type.
2. **Rate limits across the room** — switch to the fallback model named in `fact-check.md`, or stagger: half the room runs while half writes their test set.
3. **The provider is down** — flip the room to `MOCK = True` in Cell 1: every cell runs on canned `[MOCK]` responses, so the checkpoints still happen at reduced fidelity. If even Colab is gone, the deck widgets all run offline: convert the lab to a paper walkthrough — they write the prompts and predict the outputs — and run the live version as the opening of the next block (after S6 there is no next block, so for S6 it becomes the post-weekend stretch). Say plainly that this is what production incident response looks like.
4. **No internet at all** — every deck opens offline. Teach the slides, demo the local model from Session 5 if you have it pulled, and move the lab.

Have the labs downloaded before you arrive. Colab needs the network, but the notebooks and handouts do not.

The one metric that matters

Not how many finished. **How many can explain what they built to the pair next to them.**

At the checkpoint sweep, ask one pair to explain their code to another pair. If they can, the hour worked. If the notebook runs and they cannot explain it, they copied it, and Session 4 will find that out the hard way.